

1.智能体

1.什么是agent

在大模型应用开发中，智能体通常指一种以大语言模型为推理与决策核心，结合记忆、工具调用与环境交互能力，能够进行规划决策并执行复杂任务以达成目标的软件系统。

Agent的关键能力：

- 理解用户问题
- 如何拆解任务
- 判断是否需要工具
- 需要调用哪些工具
- 如何利用好工具结果生成回答&推进任务

2.Agent的核心组件

实际开发中几个要素并不需要同时出现，

- 必须的：行动（Action）
- 几乎总是存在的：工具（Tool）
- 有条件存在的：规划决策（Planning）
- 最容易被省略的：记忆（Memory）

3.Agent创建与调用

1) 历史上的调用

在 LangChain 0.x 时代，框架内的 Agent 系统经历了“碎片化”阶段。当时的设计理念是“针对场景设计特定 Agent”：

- 如果你要实现思维链推理（ReAct），就用 `create_react_agent` ；
- 如果需要结构化输出，就用 `create_structured_chat_agent` ；
- 要工具调用，则用 `create_tool_calling_agent` 。

举例：❌ v0.x 的复杂方式

```
In [ ]: # 需要多个步骤
from langchain.chat_models import init_chat_model
from langchain.agents import AgentExecutor, create_react_agent
from langchain_core.prompts import PromptTemplate
from dotenv import load_dotenv
import os

# 1. 模型初始化
load_dotenv(override=True)
load_dotenv()
model = init_chat_model(
```

```

model="deepseek-v4-flash",
model_provider="deepseek",
api_key=os.getenv("DEEPSEEK_API_KEY"),
# extra_body={
#     "thinking":{
#         "type": "disabled"
#     }
# }
)

# 2. 创建提示词模板
prompt = PromptTemplate.from_template("""
    You are a helpful assistant.

    Tools: {tools}
    Tool Names: {tool_names}

    {agent_scratchpad}
    """)

# 3. 创建 agent
agent = create_react_agent(
    llm=model,
    tools=tools,
    prompt=prompt
)

# 4. 创建 executor
executor = AgentExecutor(
    agent=agent,
    tools=tools,
    verbose=True
)

# 5. 调用
result = executor.invoke({"input": "问题"})

```

这种方式灵活，但也带来了三个明显问题：

- 1. 心智负担高——每种 Agent 都要单独记忆 API 与参数；
- 2. 可组合性差——多个 Agent 之间无法统一调度；
- 3. 生态碎片化——不同模块难以复用或协同演化。

2) 全新的调用

LangChain 在 1.0 版本后，团队做出了彻底重构：将所有 Agent 的创建方式统一为一个入口：

`create_agent()`。它取代了旧版本中的 `create_react_agent`、`create_json_agent`、`create_tool_calling_agent` 等多种分支函数，真正让开发者用一行代码即可创建任何类型的智能体。

同时在底层通过“中间件机制（Middleware）”和“标准模型接口（`invoke / stream`）”实现全局统一。这 让框架更轻、更稳，也更易于被集成到其他 Agent 平台中。

举例：✅ v1.x 的简洁方式：

```
In [ ]: from langchain.chat_models import init_chat_model
        from langchain.agents import create_agent

        # 1. 初始化模型
        model = init_chat_model("gpt-4o-mini", model_provider="openai")

        # 2. 创建 agent (一步完成)
        agent = create_agent(
            model=model,
            tools=[tool1, tool2],
            system_prompt="Agent 的行为指令" # 可选
        )

        # 3. 调用
        result = agent.invoke({
            "messages": [{"role": "user", "content": "问题"}]
        })
```

2.Agent的基本用法1：模型的传入方式

在 LangChain 1.2 中，`create_agent` 是构建智能体的核心方式，底层基于LangGraph 实现。

`create_agent` 完整参数：

```
In [ ]: from langchain.agents import create_agent

        agent = create_agent(
            model: str | BaseChatModel, # 必需：聊天模型
            tools: List[BaseTool], # 必需：工具列表
            *,
            system_prompt: str = "", # 系统提示词
            middleware: Sequence[AgentMiddleware[StateT_co, ContextT]] = (), # 中间件
            interrupt_before: List[str] = None, # 在某些工具前暂停（人机协作）
            interrupt_after: List[str] = None, # 在某些工具后暂停
            debug: bool = False, # 调试模式
            name: str | None = None, # 设置模型名称
        )
```

Agent在创建时，涉及到模型(Agent使用的模型)、可调用工具、系统提示词等参数的设置。

更多参数参考：

https://reference.langchain.com/python/langchain/agents/factory/create_agent

Agent中模型的传入方式：

本节我们只关注模型的传入。

模型是 Agent 的“大脑”，负责决策和推理。根据模型传入agent方式的不同，分为两种方式。

2.1 传入模型字符串

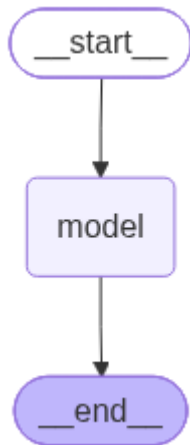
Agent根据传入的模型字符串，自主创建模型对象

```
In [1]: from langchain.agents import create_agent
from dotenv import load_dotenv

load_dotenv(override=True)
agent = create_agent("deepseek-v4-flash")
print(type(agent))

from IPython.display import Image, display
display(Image(agent.get_graph().draw_mermaid_png()))
```

```
c:\Users\huan.zheng\AppData\Local\miniconda3\envs\langchain1\Lib\site-
packages\langgraph\checkpoint\base\__init__.py:17: LangChainPendingDeprecationWarning:
The default value of `allowed_objects` will change in a future version. Pass an explicit value
(e.g., allowed_objects='messages' or allowed_objects='core') to suppress this warning.
from langgraph.checkpoint.serde.jsonplus import JsonPlusSerializer
<class 'langgraph.graph.state.CompiledStateGraph'>
```



由上可知，agent本质上是LangGraph的CompiledStateGraph实例，底层实现是一个图结构。

通过上述代码最后一行可以看到agent的图结构。

2.2 传入模型对象

```
In [3]: from langchain.chat_models import init_chat_model
from langchain.agents import create_agent
from langchain_deepseek import ChatDeepSeek
from dotenv import load_dotenv
import os

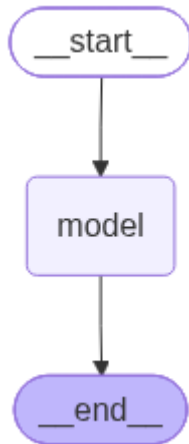
load_dotenv(override=True)
# 以ChatDeepSeek为例
# model = ChatDeepSeek(model="deepseek-v4-flash")
# 以init_chat_model为例
model = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
```

```

base_url=os.getenv("DEEPSEEK_BASE_URL")
)
agent = create_agent(model)
print(type(agent))
from IPython.display import Image, display
display(Image(agent.get_graph().draw_mermaid_png()))

```

<class 'langgraph.graph.state.CompiledStateGraph'>



3.Agent的基本用法2：如何调用Agent

agent.invoke() 是Agent 最基本的同步调用方法，它会阻塞程序执行直到返回最终结果。
具体的：

- **输入**：传入的参数为字典类型，字典内通过messages字段传递消息列表。即：“
{"messages":[{"role": "...", "content": "..."}]} ”
- **输出**：通过invoke调用Agent，底层可能会经历多轮交互，返回的是完整的消息列表，被封装在字典中，是messages字段的值。

```

In [ ]: response = agent.invoke({"messages": [...]})
# response 是字典类型
{
  "messages": [
    HumanMessage(...), # 用户问题
    AIMessage(...), # AI 工具调用
    ToolMessage(...), # 工具返回结果
    AIMessage(...) # 最终回答 ← 通常取这个
  ]
}

# 获取最终回答
final_answer = response['messages'][-1].content

```

举例1：

```

In [1]: from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
import os
load_dotenv(override=True)
from rich import print as rprint

```

```

# 以init_chat_model为例
model = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL")
)

agent = create_agent(model=model)
response = agent.invoke({"messages": ["你好"]}) # 默认是HumanMessage
print(type(response))
rprint(response)

```

c:\Users\huan.zheng\AppData\Local\miniconda3\envs\langchain1\Lib\site-packages\langgraph\checkpoint\base__init__.py:17: LangChainPendingDeprecationWarning: The default value of `allowed\_objects` will change in a future version. Pass an explicit value (e.g., allowed_objects='messages' or allowed_objects='core') to suppress this warning.

```

from langgraph.checkpoint.serde.jsonplus import JsonPlusSerializer
<class 'dict'>

```

```

{
  'messages': [
    HumanMessage(
      content='你好',
      additional_kwargs={},
      response_metadata={},
      id='12380fe9-6a03-423b-a457-d4ba0f0f1ebb'
    ),
    AIMessage(
      content='你好！很高兴见到你😊 有什么我可以帮你的吗？',
      additional_kwargs={
        'refusal': None,
        'reasoning_content': 'We need answer in Chinese.
Need be careful. User says "你好" (hello). We
should respond friendly. Maybe introduce as AI assistant. Need not
overdo.'
      },
      response_metadata={
        'token_usage': {
          'completion_tokens': 49,
          'prompt_tokens': 84,
          'total_tokens': 133,
          'completion_tokens_details': {
            'accepted_prediction_tokens': None,
            'audio_tokens': None,
            'reasoning_tokens': 34,
            'rejected_prediction_tokens': None
          },
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0},
          'prompt_cache_hit_tokens': 0,
          'prompt_cache_miss_tokens': 84
        },
        'model_provider': 'deepseek',
        'model_name': 'deepseek-v4-flash',
        'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
        'id': '33820836-bb32-4e25-a613-5bf96567d5f8',
        'finish_reason': 'stop',
        'logprobs': None
      },
      id='lc_run--019ff3d1-0f59-7350-878d-7d3940a949bb-0',
      tool_calls=[],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 84,
        'output_tokens': 49,
        'total_tokens': 133,
        'input_token_details': {'cache_read': 0},
        'output_token_details': {'reasoning': 34}
      }
    )
  ]
}

```

举例2：

invoke调用的核心就是输入一系列消息（messages），每条消息通常包含 role（如 "user", "assistant","system", "tool"）和 "content"。

我们也可以在message列表的开头加入"system"角色的消息来定义Agent的行为。

```
In [2]: from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
import os

load_dotenv(override=True)
from rich import print as rprint

# 以init_chat_model为例
model = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL")
)

agent = create_agent(model)

resp = agent.invoke({
    "messages":[
        {"role":"system","content":"你是一个小学数学老师，耐心，幽默，讲解深入浅出"},
        {"role":"user","content":"100+50等于多少？"}
    ]
})

rprint(resp)
```

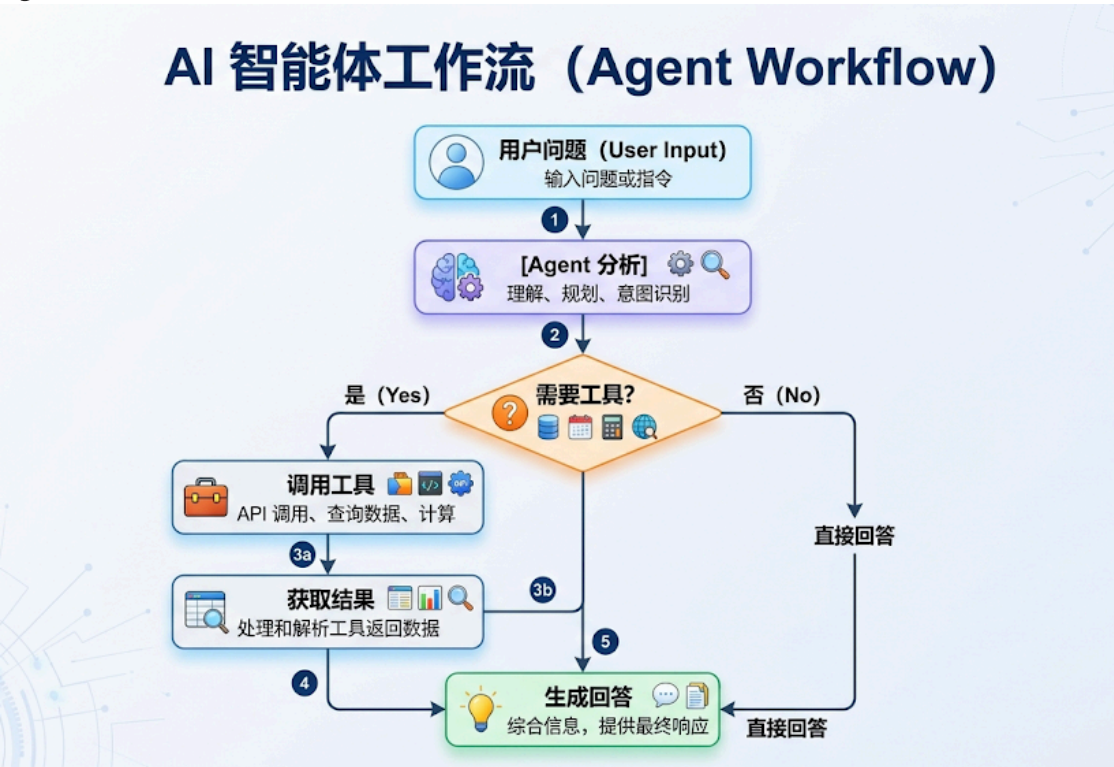
```

{
  'messages': [
    SystemMessage(
      content='你是一个小学数学老师，耐心，幽默，讲解深入浅出',
      additional_kwargs={},
      response_metadata={},
      id='2c53804b-4eba-4fda-b2da-b7dbef447777'
    ),
    HumanMessage(
      content='100+50等于多少？',
      additional_kwargs={},
      response_metadata={},
      id='6650bd5a-50de-4309-a3a5-e3ee9f164bad'
    ),
    AIMessage(
      content='100+50等于150呀，你看，就像你有100颗糖，再得到50颗，
      一共就有150颗糖，甜到心里去啦！😊',
      additional_kwargs={
        'refusal': None,
        'reasoning_content':
        '我们要求解100+50。这是简单的加法。答案150。注意用户问的是小学数学，但问题简单，
        直接回答即可。可以幽默一点。'
      },
      response_metadata={
        'token_usage': {
          'completion_tokens': 67,
          'prompt_tokens': 102,
          'total_tokens': 169,
          'completion_tokens_details': {
            'accepted_prediction_tokens': None,
            'audio_tokens': None,
            'reasoning_tokens': 32,
            'rejected_prediction_tokens': None
          },
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0},
          'prompt_cache_hit_tokens': 0,
          'prompt_cache_miss_tokens': 102
        },
        'model_provider': 'deepseek',
        'model_name': 'deepseek-v4-flash',
        'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
        'id': 'dc3b161d-8453-42ac-adbe-f34f929c4dbe',
        'finish_reason': 'stop',
        'logprobs': None
      },
      id='lc_run--019ff3d6-a865-76b2-81da-f74a59c813b8-0',
      tool_calls=[],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 102,
        'output_tokens': 67,
        'total_tokens': 169,
        'input_token_details': {'cache_read': 0},
        'output_token_details': {'reasoning': 32}
      }
    )
  ]
}

```

4.Agent的基本用法3：绑定工具

只有接入了一些工具，create_agent完成Agent创建才算完整。
Agent支持静态和动态绑定工具，后者需要用到中间件，后面会讲。在执行时：



这里的工具可以是LangChain内置的，也可以是自定义的。LangChain生态中已经内置集成了非常多的实用工具，开发者可以快速调用这些工具完成更加复杂工作流的开发。

LangChain内置工具列表：<https://docs.langchain.com/oss/python/integrations/tools>
其中典型的工具如下：

领域	工具名称	功能说明
搜索引擎与信息检索	DuckDuckGo Search	免费搜索引擎，返回网页标题、摘要与链接，常用于联网问答。
	Brave Search API	提供隐私友好的网页搜索结果，可配置输出深度与过滤。
	Bing Search Tool	微软 Bing 搜索接口，支持精确检索与多语言查询。
网页浏览与内容提取	Playwright Browser Toolkit	提供浏览器自动化访问、网页截图与文本抓取功能。
	Selenium Toolkit	类似于 Playwright，但偏向传统网页 DOM 操控。
生产力工具集成	Slack Toolkit	允许 Agent 在 Slack 中发送、读取或总结消息。
	Jira Toolkit	用于查询、创建或更新 Jira 项目工单。
	Google Drive / Docs Toolkit	操作 Google 文档、表格与文件搜索。

领域	工具名称	功能说明
数据库与数据操作	SQLDatabase Toolkit	连接 SQL 数据库，自动生成并执行 SQL 查询。
	Faiss / Chroma / Milvus VectorStore Tools	进行向量相似度搜索，用于 RAG 检索。
代码与终端执行	Python REPL Tool	执行 Python 代码片段并返回结果（沙盒环境）。
	Shell Tool (Bash)	允许执行 Shell 命令（需安全限制）。
	Requests Wrapper Tool	发起 HTTP 请求、API 调用，用于外部数据交互。
多模态工具	DALL-E Tool	生成图像，可结合文本描述创作内容。
	SerpAPI Image Search	图片搜索引擎，可返回图片 URL 与元信息。
	OpenAI Whisper Tool	音频转录工具，将语音文件转换为文本。
知识问答与嵌入检索	VectorStoreRetrieverTool	基于嵌入模型检索文档片段，用于 RAG 问答。
	MultiQueryRetrieverTool	生成多种查询变体以提升检索覆盖率。
云与AI服务	OpenAI Functions / Azure Functions Tool	让 Agent 调用 OpenAI 或 Azure 定义的函数。
	Anthropic ToolKit	集成 Claude 模型工具调用能力。
文件系统与数据处理	CSV Toolkit	读取、过滤与分析 CSV 文件数据。
	Pandas DataFrame Agent Tool	直接在 DataFrame 上执行查询与统计任务。

1) 基本用法

Agents支持绑定一或多个工具。

举例1：绑定一个工具

调用查询天气工具进行天气查询：

```
In [4]: from langchain.agents import create_agent
from langchain.tools import tool
from rich import print as rprint

@tool(parse_docstring=True)
def get_weather(city: str) -> str:
    """
    天气查询工具

    Args:
```

```

    city:城市名称
    """
    return f"{city}的天气为晴朗，25摄氏度"

agent = create_agent(
    model = model,
    tools = [get_weather]
)

resp = agent.invoke({
    "messages":[
        {"role":"system","content":"你是一个天气查询助手，只回答天气相关的问题，其他问题请直接回答：我不清楚这问题答案。"},
        {"role":"user","content":"上海的天气怎么样？"},
        #{"role":"user", "content": "100+50等于多少？"}
    ]
})

rprint(resp)

```

```

{
  'messages': [
    SystemMessage(
      content='你是一个天气查询助手，只回答天气相关的问题，其他问题请直接回答：我不清楚这问题答案。',
      additional_kwargs={},
      response_metadata={},
      id='a8fed6f7-ee8d-4f23-a257-46cfcde4b9ce'
    ),
    HumanMessage(
      content='上海的天气怎么样？',
      additional_kwargs={},
      response_metadata={},
      id='05b4eb55-58d2-438b-9ab9-e8c402d7148d'
    ),
    AIMessage(
      content='',
      additional_kwargs={
        'refusal': None,
        'reasoning_content': 'The user is asking about the weather in Shanghai. This is a weather-related question, so I should use the weather tool. Let me query the weather for Shanghai.'
      },
      response_metadata={
        'token_usage': {
          'completion_tokens': 78,
          'prompt_tokens': 384,
          'total_tokens': 462,
          'completion_tokens_details': {
            'accepted_prediction_tokens': None,
            'audio_tokens': None,
            'reasoning_tokens': 33,
            'rejected_prediction_tokens': None
          },
          'prompt_tokens_details': {'audio_tokens': None},
          'cached_tokens': 0,
          'prompt_cache_hit_tokens': 0,
          'prompt_cache_miss_tokens': 384
        },
        'model_provider': 'deepseek',
        'model_name': 'deepseek-v4-flash',
        'system_fingerprint': 'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
        'id': '8c796141-1eae-404b-9a88-35fff8bdee5d',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff3e2-a0c7-71a0-a83e-13840483897a-0',
      tool_calls=[
        {
          'name': 'get_weather',
          'args': {'city': '上海'},
          'id': 'call_00_BLnMZlj5urreVyRHdHib0648',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 384,
        'output_tokens': 78,
        'total_tokens': 462,
        'input_token_details': {'cache_read': 0},

```

```

        'output_token_details': {'reasoning': 33}
    },
    ToolMessage(
        content='上海的天气为晴朗, 25摄氏度',
        name='get_weather',
        id='933d0816-4bb8-441e-99dd-7ee51c1b0b64',
        tool_call_id='call_00_BLnMZlj5urreVyRHdHib0648'
    ),
    AIMessage(
        content='上海目前天气晴朗, 气温为25摄氏度。',
        additional_kwargs={'refusal': None, 'reasoning_content':
''},
        response_metadata={
            'token_usage': {
                'completion_tokens': 11,
                'prompt_tokens': 481,
                'total_tokens': 492,
                'completion_tokens_details': {
                    'accepted_prediction_tokens': None,
                    'audio_tokens': None,
                    'reasoning_tokens': 0,
                    'rejected_prediction_tokens': None
                },
                'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 256},
                'prompt_cache_hit_tokens': 256,
                'prompt_cache_miss_tokens': 225
            },
            'model_provider': 'deepseek',
            'model_name': 'deepseek-v4-flash',
            'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
            'id': 'd6e26a29-5605-45ca-bb06-6b29d73bf2d1',
            'finish_reason': 'stop',
            'logprobs': None
        },
        id='lc_run--019ff3e2-a466-7283-ad1c-34c91e150a62-0',
        tool_calls=[],
        invalid_tool_calls=[],
        usage_metadata={
            'input_tokens': 481,
            'output_tokens': 11,
            'total_tokens': 492,
            'input_token_details': {'cache_read': 256},
            'output_token_details': {'reasoning': 0}
        }
    )
]
}

```

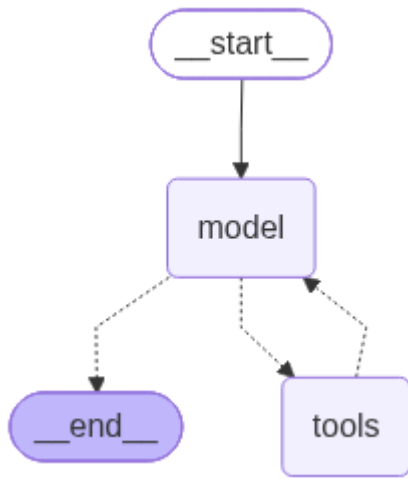
进一步：

```

In [5]: from IPython.display import Image, display

display(Image(agent.get_graph().draw_mermaid_png()))

```



举例2：接入内置工具

绑定内置的TavilySearch搜索工具，可以借助Tavily进行网络搜索和信息爬取。

这里我们需要先在tavily官网注册并获得API-KEY（每月有免费额度）：

<https://www.tavily.com/>。

```
In [6]: from langchain_tavily import TavilySearch
from dotenv import load_dotenv
import os

load_dotenv(override=True)
web_search = TavilySearch(
    tavily_api_key=os.getenv("TAVILY_API_KEY"),
    max_results=2
)

#这是一个高度封装的网络搜索工具，可以直接调用：
web_search.invoke("请问2026年足球世界杯有哪些参赛队？")
```

```

Out[6]: {'query': '请问2026年足球世界杯有哪些参赛队?',
'follow_up_questions': None,
'answer': None,
'images': [],
'results': [{'url':
'https://baike.baidu.com/item/2026%E5%B9%B4%E5%9B%BD%E9%99%85%E8%B6%B3%E8%81%94%E4%B8%96%E7%95%8C%E6%9D%AF/62863018',
'title': '2026年国际足联世界杯',
'content': '| 地区 | 球队 | 出线日期 | 参赛 次数 含2026 | 最好成绩 （年份） | \n --- --- \n | 东道主 (Host, 3支球队) | 美国国家男子足球队 | 2018年6月13日 | 12 | 季军 (1930) | \n | 加拿大国家男子足球队 | 2018年6月13日 | 3 | 小组赛 (1986、 2022年2次) | \n | 墨西哥国家男子足球队 | 2018年6月13日 | 18 | 八强 (1970、1986) | \n | 欧洲足球协会联盟 （UEFA, 16支球队） | 英格兰男子足球代表队 | 2025年10月15日 | 17 | 冠军 (1966) | \n | 法国国家男子足球队 | 2025年11月14日 | 17 | 冠军 (1998、2018 共2次) | \n | 克罗地亚国家男子足球队 | 2025年11月15日 | 7 | 亚军 (2018) | \n | 葡萄牙国家男子足球队 | 2025年11月16日 | 9 | 季军 (1966) | \n | 挪威国家男子足球队 | 2025年11月17日 | 4 | 十六强 (1998) | [...] | 德国国家男子足球队 | 2025年11月18日 | 21 | 冠军 (1954、1974、1990、2014 共4次) | \n | 荷兰国家男子足球队 | 2025年11月18日 | 12 | 亚军 (2010) | \n | 比利时国家男子足球队 | 2025年11月19日 | 15 | 季军 (2018) | \n | 奥地利国家男子足球队 | 2025年11月19日 | 8 | 季军 (1954) | \n | 西班牙国家男子足球队 | 2025年11月19日 | 17 | 冠军 (2010) | \n | 瑞士国家男子足球队 | 2025年11月19日 | 13 | 八强 (1934、1938、 1954) | \n | 苏格兰男子足球代表队 | 2025年11月19日 | 9 | 小组赛 | \n | 瑞典国家男子足球队 | 2026年4月1日 | 13 | 亚军 (1958) | \n | 土耳其国家男子足球队 | 2026年4月1日 | 3 | 季军 (2002) | \n | 捷克国家男子足球队 | 2026年4月1日 | 2 8+2 | 小组赛 (2006) 亚军 (1962、1934) | [...] | 突尼斯国家男子足球队 | 2025年9月8日 | 7 | 小组赛 (2022 等6次) | \n | 埃及国家男子足球队 | 2025年10月9日 | 4 | 埃及 (2018等4次) | \n | 阿尔及利亚国家男子足球队 | 2025年10月10日 | 5 | 十六强 (2014) | \n | 加纳国家男子足球队 | 2025年10月13日 | 5 | 八强 (2010) | \n | 佛得角国家男子足球队 | 2025年10月14日 | 1 | 首次 | \n | 南非国家男子足球队 | 2025年10月15日 | 4 | 小组赛 (2010等3次) | \n | 科特迪瓦国家男子足球队 | 2025年10月15日 | 4 | 小组赛 (2014等3次) | \n | 塞内加尔国家男子足球队 | 2025年10月15日 | 4 | 八强 (2002) | \n | 刚果民主共和国国家男子足球队 | 2026年4月1日 | 2 | 小组赛 (1974) | \n | 中北美洲及 加勒比海足球协会 (CONCACAF 3支球队) | 巴拿马国家男子足球队 | 2025年11月19日 | 2 | 小组赛 (2018) |',
'score': 0.8934678,
'raw_content': None,
'id': 'f0d4e9-00'},
{'url':
'https://zh.wikipedia.org/wiki/2026%E5%B9%B4%E5%9C%8B%E9%9A%9B%E8%B6%B3%E5%8D%94%E4%B8%96%E7%95%8C%E7%9B%83',
'title': '2026年國際足總世界盃',
'content': '| FIFA World Cup 26 Copa Mundial de la FIFA 2026 Coupe du Monde FIFA 2026 | \n | We Are 26 Somos 26 Nous Sommes 26 | \n | 賽事資料 | \n | 主辦國家／地区 | 加拿大 墨西哥 美國 | \n | 比賽日期 | 6月11日–7月19日 | \n | 參賽隊數 | 48 隊（來自6個大洲） | \n | 球場 | 16 個（位於16個城市） | \n | 最終成績 | \n | 冠軍 | 西班牙（第 2 次奪冠） | \n | 亞軍 | 阿根廷 | \n | 季軍 | 英格蘭 | \n | 殿軍 | 法國 | \n | 賽事統計 | \n | 比賽場數 | 104 場 | \n | 總入球數 | 308 球（場均 2.96 球） | \n | 入場人數 | 6,810,966 人（場均 65,490 人） | \n | 最佳射手 | 法國 基利安·麥巴比（10 球） | \n | 最佳球員 | 西班牙 羅德里 | \n | 最佳守門員 "守門員 (足球)" | 西班牙 烏奈·西蒙 | \n | 最佳年輕球員 | 西班牙 保·庫巴西 | [...] 本屆賽事首次由過往的32支参赛球队扩大至48支。作为主办国，三个东道主自动获得决赛周参赛资格。在入围名单中，佛得角、库拉索、约旦和乌兹别克斯坦均为首次晋身世界盃決賽周。本屆賽事的冠軍為西班牙。 \n \n ## 擴充 \n \n [编辑] \n \n 早在2013年，時任歐足聯主席米歇爾·普拉蒂尼就提出了世界盃擴充的構想。2016年，國際足聯主席基安尼·恩芬天奴也再次提出類似觀點。但有反對人士認為目前賽制的比賽場次已經過多，擴充將稀釋比賽的可觀性及含金量，並指責因凡蒂諾向更多國家承諾世界盃參賽權，背後動機是主席選舉拉票的政治因素而非體育考量。 \n \n 自本屆賽事起，世界盃決賽圈參賽隊伍擴充至48支，較此前七屆增加了16支。按國際足協委員會在2023年3月14日通過的方案，球隊將被分為12個小組，每組4隊，每組前兩名與8個成績最好的小組第三名將晉級全新的32強淘汰賽階段，這將是自1998年以來世界盃首次擴軍並更改賽制。 \n \n 比賽總場次將從64場增至104場，進入四強的球隊比賽總場次將從7場增至8場。整個賽事將持續39天，比2014年和2018年賽事的32天有所增加。每支球隊仍將進行3場小組賽。 [...] 出線球隊 \n \n 被淘汰的球隊 \n \n 取消比賽資格或自動退出比賽 \n \n 非國際足協會員 \n \n 48支取得參賽資格的球隊中，有26支也參加了2022年世界盃。其中： \n \n 佛得角、古拉索、約旦和烏茲別克均為首次晉級世界盃決賽圈。 \n 剛果民主共和國和海地在1974年唯一一次參賽後重返世界盃。 \n 奧地利、挪威和蘇格蘭在1998年後重返世界盃。 \n 南非自2002年以來首次通過預選賽晉級，它們在2010年作為東道主自動獲得參賽資格。 \n 土耳其自2002年獲得季軍後首次獲得參賽資格。 \n 捷克自2006年以來首次獲得參賽資格。 \n 上屆東道主卡塔

```

爾首次透過预选赛晉級世界盃決賽圈。\\n 值得一提的是，意大利在欧洲区外围赛附加赛决赛点球大战中输给波斯尼亚和黑塞哥维那，错失参赛资格，成为首支连续三届无缘世界杯决赛圈的前冠军。\\n 俄罗斯由於入侵烏克蘭，继续被取消参加此次外圍賽及決賽週。\\n\\n 以下是按地区划分的出线球队名单，括號內的數字代表賽前的FIFA 男子世界排名。';

```
'score': 0.88420963,
'raw_content': None,
'id': '78c9bf-01'}],
'response_time': 1.08,
'request_id': '8be2c33d-ee3f-4403-bb9a-0fcdfb868419'}
```

可以直接帶入create_agent中作为外部工具。

```
In [8]: from langchain.chat_models import init_chat_model
from langchain.agents import create_agent
from langchain_tavily import TavilySearch
from dotenv import load_dotenv
import os
from rich import print as rprint

load_dotenv(override=True)

# 1. 模型初始化
model = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL")
)

# 2. 工具实例化
web_search = TavilySearch(
    tavily_api_key=os.getenv("TAVILY_API_KEY"),
    max_results=2)

# 3. 创建Agent
agent = create_agent(
    model=model,
    tools=[web_search],
    system_prompt="你是一名多才多艺的智能助手，可以调用工具帮助用户解决问题。"
)

# 4. 运行Agent 获得结果
result = agent.invoke({
    "messages": [
        {"role": "user", "content": "请帮我查询2024年诺贝尔物理学奖得主是谁？"}
    ]
})

rprint(result)
# print(result['messages'][-1].content)
```

```

{
  'messages': [
    HumanMessage(
      content='请帮我查询2024年诺贝尔物理学奖得主是谁？',
      additional_kwargs={},
      response_metadata={},
      id='74cf79ca-ae96-4054-82b3-b14741a738d4'
    ),
    AIMessage(
      content='',
      additional_kwargs={
        'refusal': None,
        'reasoning_content': 'The user is asking about the
2024 Nobel Prize in Physics winners. Let me
search for this information.'
      },
      response_metadata={
        'token_usage': {
          'completion_tokens': 76,
          'prompt_tokens': 1918,
          'total_tokens': 1994,
          'completion_tokens_details': {
            'accepted_prediction_tokens': None,
            'audio_tokens': None,
            'reasoning_tokens': 22,
            'rejected_prediction_tokens': None
          },
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 1792},
          'prompt_cache_hit_tokens': 1792,
          'prompt_cache_miss_tokens': 126
        },
        'model_provider': 'deepseek',
        'model_name': 'deepseek-v4-flash',
        'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
        'id': 'fc31a863-1d52-40fc-9ac9-501da178cc18',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff3ed-e65f-76f0-add1-ad68b0b4f674-0',
      tool_calls=[
        {
          'name': 'tavily_search',
          'args': {'query': '2024年诺贝尔物理学奖得主是谁'},
          'id': 'call_00_qWvPd4c8IiT9XdtWJf2V6258',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 1918,
        'output_tokens': 76,
        'total_tokens': 1994,
        'input_token_details': {'cache_read': 1792},
        'output_token_details': {'reasoning': 22}
      }
    ),
    ToolMessage(
      content='{"query": "2024年诺贝尔物理学奖得主是谁",
"follow_up_questions": null, "answer": null,
"images": [], "results": [{"url":
"https://news.sciencenet.cn/htmlnews/2024/10/531146.shtm", "title":

```

"刚刚，2024年诺贝尔物理学奖揭晓！ - 新闻- 科学网", "content": "| | |\n\n| |\n| |\n| |\n"

刚刚，2024年诺贝尔物理学奖揭晓！ |\n|

|\n\n\n北京时间10月8日下午5点45分许，2024年诺贝尔物理学奖揭晓。美国普林斯顿大学科学家约翰·霍普菲尔德（John J.

Hopfield）和加拿大多伦多大学科学家杰弗里·辛顿（Geoffrey E.

Hinton）获奖，以表彰他们“基于人工神经网络实现机器学习的基础性发现和发明”。

\n\n\n他们用物理学训练人工神经网络\n\n\n

今年的两位诺贝尔物理学奖得主使用物理学的工具来开发方法，这些方法是当今强大的机器学习的基础。John J.

Hopfield创造了一种联想记忆，可以存储和重建图像和其他类型的数据模式。Geoffrey

Hinton发明了一种方法，可以自动发现数据中的属性，从而执行识别图片中的特定元素等任务。 [...] | | | | | | | |

刚刚，2024年诺贝尔物理学奖揭晓！ | | |

北京时间10月8日下午5点45分许，2024年诺贝尔物理学奖揭晓。美国普林斯顿大学科学家约翰·霍普菲尔德（John J.

Hopfield）和加拿大多伦多大学科学家杰弗里·辛顿（Geoffrey E.

Hinton）获奖，以表彰他们“基于人工神经网络实现机器学习的基础性发现和发明”。 他们用物理学训练人工神经网络

今年的两位诺贝尔物理学奖得主使用物理学的工具来开发方法，这些方法是当今强大的机器学习的基础。John J.

Hopfield创造了一种联想记忆，可以存储和重建图像和其他类型的数据模式。Geoffrey

Hinton发明了一种方法，可以自动发现数据中的属性，从而执行识别图片中的特定元素等任务。 [...] | | | |

作者：冯丽妃，梅进 来源：科学网微信公众号 发布时间：2024/10/8 17:52:45 选择

字号：小 中 大 | | | | | | | |

刚刚，2024年诺贝尔物理学奖揭晓！ | | |

北京时间10月8日下午5点45分许，2024年诺贝尔物理学奖揭晓。美国普林斯顿大学科学家约翰·霍普菲尔德（John J.

Hopfield）和加拿大多伦多大学科学家杰弗里·辛顿（Geoffrey E.

Hinton）获奖，以表彰他们“基于人工神经网络实现机器学习的基础性发现和发明”。 他们用物理学训练人工神经网络

今年的两位诺贝尔物理学奖得主使用物理学的工具来开发方法，这些方法是当今强大的机器学习的基础。John J.

Hopfield创造了一种联想记忆，可以存储和重建图像和其他类型的数据模式。Geoffrey

Hinton发明了一种方法，可以自动发现数据中的属性，从而执行识别图片中的特定元素等任务。", "score": 0.92074,

"raw_content": null, "id": "926929-00"}, {"url":

"http://www.news.cn/world/20241008/c5aff4c9f7564a4c96d80d714fba74c8/c.html", "title":

"两名科学家因机器学习方面的贡献分享2024年诺贝尔物理学奖", "content":

"瑞典皇家科学院当天发表公报说，今年的两位诺贝尔物理学奖得主使用物理学工具，为当今强大的机器学习技术奠定了基础。约翰

·霍普菲尔德创建了一种联想记忆方法，可以存储和重构图像或其他类型的数据模式。杰弗里·欣顿发明了一种可以自动发现数据中

属性的方法，可用于识别图片中的特定元素等任务。 \n\n\n\u2003\u2003诺贝尔物理学委员会主席埃伦·穆恩斯在当天的新闻发布会

上表示，两名获奖者利用统计物理的基本概念设计了人工神经网络，构建了机器学习的基础。

相关技术已被用于推动多个领域的研

究，包括粒子物理、材料科学和天体物理等，也已用于日常生活中的人脸识别和语言翻译等。

她同时警告说，机器学习的快速发展

也引发了人们对未来的担忧，人类有责任以安全且道德的方式使用这项新技术。 \n\n\n10月

8日，在瑞典斯德哥尔摩举行的2024年诺

贝尔物理学奖公布现场，屏幕显示奖项得主美国普林斯顿大学的约翰·霍普菲尔德和加拿大多伦多大学的杰弗里·欣顿。新华社记者

彭子洋

摄\\n\\n\\u2003\\u2003约翰·霍普菲尔德1933年出生于美国芝加哥，1958年获得美国康奈尔大学博士学位，现任美国普林斯顿大学教

授。", "score": 0.9161773, "raw_content": null, "id": "960ba0-01"}], "response_time": 0.0, "request_id":

"b75b7e20-193b-4efa-a3a9-77ab36b05d1a"}',

name='tavily_search',

id='020a54ab-fe40-4390-8754-2b62e5350fd6',

tool_call_id='call_00_qWvPd4c8IiT9XdtWJf2V6258'

),

AIMessage(

content='我查询到了2024年诺贝尔物理学奖的获奖信息，以下是详细内

容：\\n\\n## 🏆

2024年诺贝尔物理学奖\\n\\n**获奖者：**\\n1. **约翰·霍普菲尔德**（John J. Hopfield）— 美国普林斯顿大学教授\\n2.

杰弗里·辛顿（Geoffrey E. Hinton）— 加拿大多伦多大学教授\\n\\n**获奖理由：**\\n表彰他们

基于人工神经网络实现机器学习的基础性发现和发明。\\n\\n**主要贡献：**\\n-

约翰·霍普菲尔德：创造了一种联想记忆方法，能够存储和重构图像或其他类型的数据模式（即著名的**霍普菲尔德网络**）。\\n-

杰弗里·辛顿：发明了一种能够自动发现数据中属性的方法，可用于识别图片中的特定元素等任务（即**玻尔兹曼机**等相关技术）。\\n\\n**补充背景：**\\n-

两位获奖者利用**统计物理的基本概念**设计了人工神经网络，为当今强大的机器学习技术

奠定了基础。\\n-

他们的研究成果已广泛应用于粒子物理、材料科学、天体物理等研究领域，也渗透到日常生活中的**人脸识别**和**语言翻译**等

场景。\\n-

值得一提的是，杰弗里·辛顿还被称为"人工智能教父"，他此前也一直被认为是诺贝尔奖的热门候选人。\\n\\n有意思的是，这届物理

学奖颁发给了与计算机科学和人工智能相关的成果，这在诺贝尔物理学奖历史上较为特别。如果你对这两位科学家的研究细节或其

他年份的获奖者感兴趣，也可以继续问我哦！😊',

additional_kwargs={'refusal': None, 'reasoning_content':

''},

response_metadata={

'token_usage': {

'completion_tokens': 324,

'prompt_tokens': 3117,

'total_tokens': 3441,

'completion_tokens_details': {

'accepted_prediction_tokens': None,

'audio_tokens': None,

'reasoning_tokens': 0,

'rejected_prediction_tokens': None

},

'prompt_tokens_details': {'audio_tokens': None,

'cached_tokens': 1920},

'prompt_cache_hit_tokens': 1920,

'prompt_cache_miss_tokens': 1197

},

'model_provider': 'deepseek',

'model_name': 'deepseek-v4-flash',

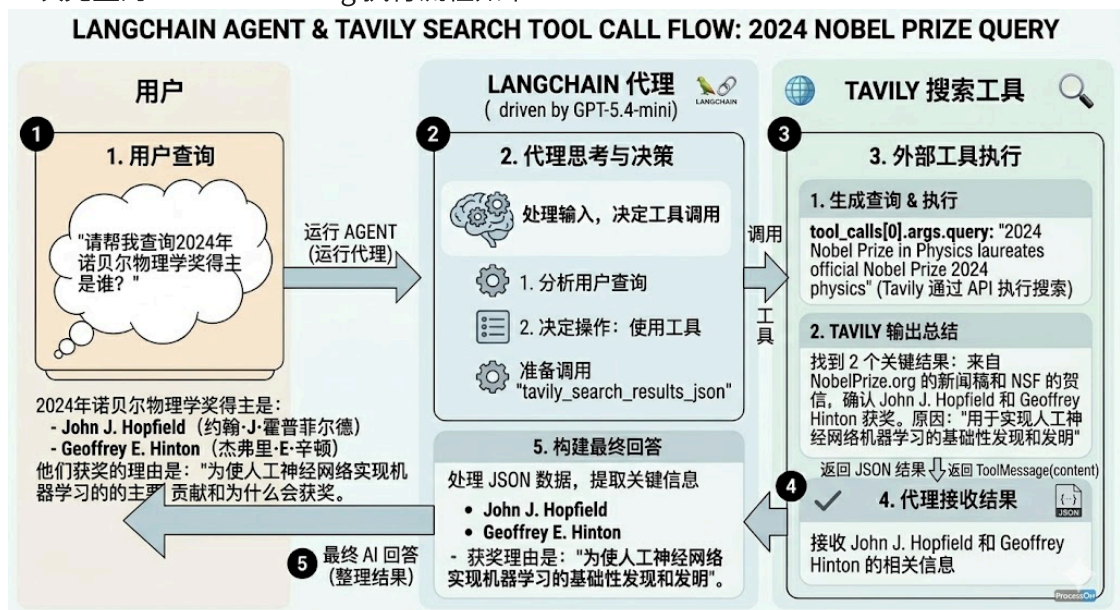
```

        'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
        'id': '73013dd3-07fd-405a-8f8a-86494cc9752d',
        'finish_reason': 'stop',
        'logprobs': None
    },
    id='lc_run--019ff3ed-eec1-7802-8302-8d8216e20eac-0',
    tool_calls=[],
    invalid_tool_calls=[],
    usage_metadata={
        'input_tokens': 3117,
        'output_tokens': 324,
        'total_tokens': 3441,
        'input_token_details': {'cache_read': 1920},
        'output_token_details': {'reasoning': 0}
    }
}
    ]
}

```

如果仔细观察本次运行过程，本次工具调用仍然是一次典型的Function calling执行流程，包括用户首次发起消息在内，总共创建了4条消息，分别是human message、ai message（涉及function call message）、tool message（涉及function response message）以及ai message（涉及final responses）。

一次完整的Function calling 执行流程如下：



举例3：绑定多个工具

```

In [10]: from langchain.agents import create_agent
from langchain.tools import tool
from dotenv import load_dotenv
from rich import print as rprint

load_dotenv()

@tool(parse_docstring=True)
def get_weather(city: str):
    """
    天气查询工具

```

```

Args:
    city: 城市名称
    """

    return f"{city}今天天气挺好"

@tool(parse_docstring=True)
def get_news():
    """
    新闻查询工具
    """

    return "近期，受全球储蓄芯片短缺等多重因素影响，多地回收商称废旧手机回收市场迎来“火热潮”，回收价格普遍上涨，旧手机成“香饽饽”。"

agent = create_agent(
    model=model,
    tools=[get_weather, get_news]
)

response = agent.invoke({
    "messages":["你好，杭州今天的天气如何？今天有哪些新闻？"]
})

rprint(response)

```

```

{
  'messages': [
    HumanMessage(
      content='你好，杭州今天的天气如何？今天有哪些新闻？',
      additional_kwargs={},
      response_metadata={},
      id='2df0024b-ebd1-4b1f-a00d-bc8bd0ed657f'
    ),
    AIMessage(
      content='',
      additional_kwargs={
        'refusal': None,
        'reasoning_content': "The user asks about weather in
Hangzhou and today's news. These are
independent calls, so I can make them in the same block."
      },
      response_metadata={
        'token_usage': {
          'completion_tokens': 88,
          'prompt_tokens': 397,
          'total_tokens': 485,
          'completion_tokens_details': {
            'accepted_prediction_tokens': None,
            'audio_tokens': None,
            'reasoning_tokens': 28,
            'rejected_prediction_tokens': None
          },
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0},
          'prompt_cache_hit_tokens': 0,
          'prompt_cache_miss_tokens': 397
        },
        'model_provider': 'deepseek',
        'model_name': 'deepseek-v4-flash',
        'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
        'id': '2086d06b-ebbb-4d0b-871b-9bed62e417ad',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff3f8-18a4-7482-bd72-4ff09fd07c63-0',
      tool_calls=[
        {
          'name': 'get_weather',
          'args': {'city': '杭州'},
          'id': 'call_00_PoLHHMcSjdI0mAXxJGzc8694',
          'type': 'tool_call'
        },
        {'name': 'get_news', 'args': {}, 'id':
'call_01_DHZw2f1wdvM6Y7SM0gvo2469', 'type': 'tool_call'}
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 397,
        'output_tokens': 88,
        'total_tokens': 485,
        'input_token_details': {'cache_read': 0},
        'output_token_details': {'reasoning': 28}
      }
    ),
    ToolMessage(
      content='杭州今天天气挺好',
      name='get_weather',

```

```

        id='abc208b7-6d90-451a-b688-bf98f5718dc7',
        tool_call_id='call_00_PoLHMcSjdI0mAXxJGzc8694'
    ),
    ToolMessage(
        content='近期，受全球储蓄芯片短缺等多重因素影响，多地回收商称废旧手机回收市场迎来“火热潮”，回收价格普遍上涨，旧手机成“香饽饽”。',
        name='get_news',
        id='4034d583-c590-471a-9d31-9393c9ab489d',
        tool_call_id='call_01_DHZw2f1wdvM6Y7SM0gvo2469'
    ),
    AIMessage(
        content='为你查询到以下信息：\n\n**🌞 杭州天气**\n今天杭州天气不错，适合出行。\n\n**📰 今日新闻**\n近期受全球储蓄芯片短缺等多重因素影响，多地回收商表示废旧手机回收市场迎来“火热潮”，回收价格普遍上涨，旧手机成了“香饽饽”。\n\n如果还有其他需要查询的，随时告诉我~',
        additional_kwargs={'refusal': None, 'reasoning_content': ''},
        response_metadata={
            'token_usage': {
                'completion_tokens': 79,
                'prompt_tokens': 549,
                'total_tokens': 628,
                'completion_tokens_details': {
                    'accepted_prediction_tokens': None,
                    'audio_tokens': None,
                    'reasoning_tokens': 0,
                    'rejected_prediction_tokens': None
                },
                'prompt_tokens_details': {'audio_tokens': None,
            },
            'cached_tokens': 384},
            'prompt_cache_hit_tokens': 384,
            'prompt_cache_miss_tokens': 165
        },
        'model_provider': 'deepseek',
        'model_name': 'deepseek-v4-flash',
        'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
        'id': 'd93fd34a-7e1d-4783-87b0-465fc1a1cb5e',
        'finish_reason': 'stop',
        'logprobs': None
    ),
    id='lc_run--019ff3f8-1cb8-7c80-bce9-bedbdf5808f5-0',
    tool_calls=[],
    invalid_tool_calls=[],
    usage_metadata={
        'input_tokens': 549,
        'output_tokens': 79,
        'total_tokens': 628,
        'input_token_details': {'cache_read': 384},
        'output_token_details': {'reasoning': 0}
    }
}
]
}

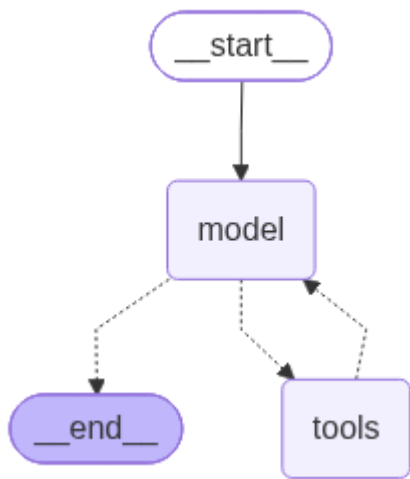
```

```

In [11]: from IPython.display import Image, display

display(Image(agent.get_graph().draw_mermaid_png()))

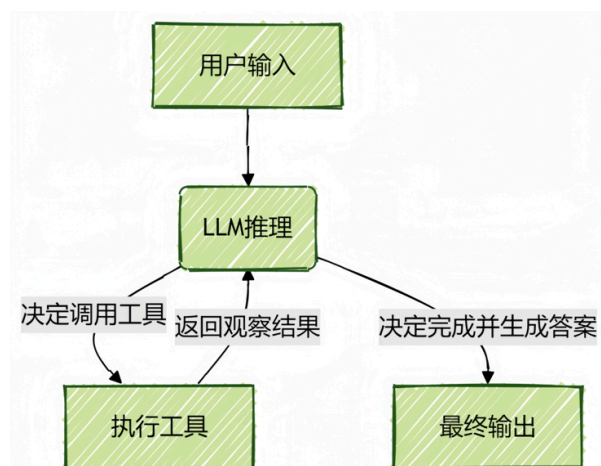
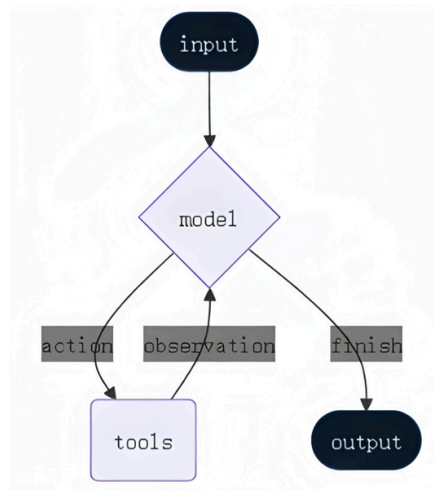
```



注意：只给 Agent 需要的工具，工具太多会混淆。一般2-5 个工具最佳

2) 工具调用流程分析

LangChain 的 Agent 会将模型与工具结合起来，在实现上由一个基于 LangGraph 的图结构来编排执行流程，如下所示。



这与前文得到的 Agent 图结构是一致的，本质上就是经典的 ReAct 结构：一个具备“思考-行动-观察”不断循环的自主工作者。

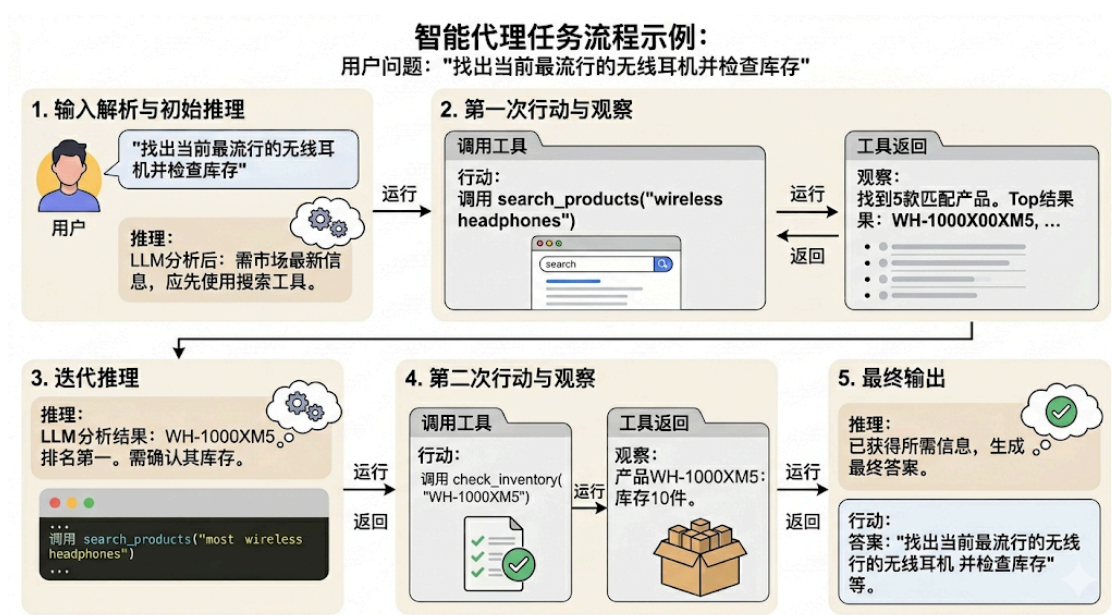
用户问题 → AI 思考 → 调用工具 → 观察结果 → 继续思考 → ... → 最终答案

当用户提出一个复杂需求时，Agent会像人类一样，先理解任务、规划步骤、使用合适的工具（如搜索网络、查询数据库、执行计算）获取信息，Agent 会在一个循环中反复调用模型和工具，直到某次模型输出中不再包含工具调用则结束，最后综合所有信息给出最终答案。

完整流程：



举例：用户问题：“找出当前最流行的无线耳机并检查库存”的任务。



1、输入解析与初始推理

- 输入：用户查询：“找出当前最流行的无线耳机并检查库存”。
- 推理：LLM分析任务后认为：“要找出‘最流行’的产品，需要最新的市场信息，我应该先使用搜索工具。”

2、第一次行动(action)与观察(observation)

- 行动：Agent调用search_products工具，参数为“wireless headphones”。
- 观察：工具返回结果：“找到5款匹配产品。Top结果：WH-1000XM5, ...”

3、迭代推理

- 推理：LLM根据搜索结果分析：“WH-1000XM5是排名第一的型号。现在需要确认其库存状态才能回答用户问题。”

4、第二次行动(action)与观察(observation)

- 行动：Agent调用check_inventory工具，参数为“WH-1000XM5”。
- 观察：工具返回：“产品WH-1000XM5：库存10件。”

5、最终输出

- 推理：LLM综合所有信息后认为：“已获得所需信息，可以生成最终答案。”
- 行动：模型生成最终答案，不再调用工具。

3) 重试机制

Agent可以在工具调用结果不满足要求时，自主重试。

```
In [12]: from langchain.agents import create_agent
from langchain.tools import tool
from langchain.messages import SystemMessage, HumanMessage
from dotenv import load_dotenv
from rich import print as rprint

load_dotenv(override=True)
flag = 0
@tool
def get_weather(city: str):
    """
    天气查询工具

    Args:
        city: 城市名称
    """
    global flag
    flag += 1

    if flag < 3:
        # raise Exception("暂时无法访问")
        return "TEMP_UNAVAILABLE: 天气服务暂时不可用，请稍后重试"

    return f"{city}今天天气挺好"

messages = [
    SystemMessage("""
    你是一个天气助手。
    当工具返回以 'TEMP_UNAVAILABLE:' 开头的结果时，
    说明是临时故障，不要立即放弃；
    你应再次调用同一个工具，最多重试 3 次。
    如果 3 次后仍失败，再向用户说明服务暂时不可用。
    """),
    HumanMessage("你好，杭州今天的天气如何？")
]

agent = create_agent(model, tools=[get_weather])
response = agent.invoke({"messages": messages})
```

```
# rprint(response)

for msg in response["messages"]:
    msg.pretty_print()
```

===== System Message =====

你是一个天气助手。
当工具返回以 'TEMP_UNAVAILABLE:' 开头的结果时，
说明是临时故障，不要立即放弃；
你应再次调用同一个工具，最多重试 3 次。
如果 3 次后仍失败，再向用户说明服务暂时不可用。

===== Human Message =====

你好，杭州今天的天气如何？

===== Ai Message =====

Tool Calls:
get_weather (call_00_5iWwAsfNtVxEM0BWULD09364)
Call ID: call_00_5iWwAsfNtVxEM0BWULD09364
Args:
city: 杭州

===== Tool Message =====

Name: get_weather

TEMP_UNAVAILABLE: 天气服务暂时不可用，请稍后重试

===== Ai Message =====

天气服务返回了临时故障提示，我马上再试一次。
Tool Calls:
get_weather (call_00_iZr1S2FLesraBCGEMuua1819)
Call ID: call_00_iZr1S2FLesraBCGEMuua1819
Args:
city: 杭州

===== Tool Message =====

Name: get_weather

TEMP_UNAVAILABLE: 天气服务暂时不可用，请稍后重试

===== Ai Message =====

仍是临时故障，我继续重试。
Tool Calls:
get_weather (call_00_ET_QdNBPf5QHAD7NkjWFZ6p1107)
Call ID: call_00_ET_QdNBPf5QHAD7NkjWFZ6p1107
Args:
city: 杭州

===== Tool Message =====

Name: get_weather

杭州今天天气挺好

===== Ai Message =====

杭州今天天气挺好的！阳光和天气状况都不错，适合外出活动。祝你有愉快的一天！☀️

模型三次调用get_weather，最终获得了满意的结果。

4) 常见问题

问题1：Agent 如何选择工具？

依据：工具的 docstring

```
In [ ]: @tool
def get_weather(city: str) -> str:
    """获取指定城市的天气信息""" # ← AI Agent 读这个！
    ...

@tool
def calculator(operation: str, a: float, b: float) -> str:
    """执行基本的数学计算""" # ← AI Agent 也读这个！
    ...
```

AI 会根据：

1. 问题内容
2. 每个工具的描述
3. 自动选择最匹配的工具

问题2：Agent 为什么没有调用工具？

原因：

- 工具的 docstring 不清晰
- 问题表述不明确
- 模型认为不需要工具

解决：

```
In [ ]: # ❌ 不好
@tool
def tool1(x: str) -> str:
    """做一些事情""" # 太模糊

# ✅ 好
@tool
def get_weather(city: str) -> str:
    """
    获取指定城市的实时天气信息

    Args:
        city: 城市名称，如"北京"、"上海"
    """
```

问题3：Agent 选错工具？

原因：

- 多个工具的功能描述相似
- 工具太多导致混淆

解决：

- 只给必要的工具
- 工具描述要有明确区分
- 在 system_prompt 中说明工具使用场景

```
In [ ]: #  好：只给需要的工具
agent = create_agent(
    model=model,
    tools=[get_weather, calculator] # 2-5 个工具最佳
)

#  不好：工具太多
agent = create_agent(
    model=model,
    tools=[tool1, tool2, ..., tool20] # 会混淆
)
```

问题4：如何知道 Agent 何时完成？

当 AIMessage 不包含 tool_calls 时：

```
In [ ]: for msg in response['messages']:
    if isinstance(msg, AIMessage):
        if hasattr(msg, 'tool_calls') and msg.tool_calls:
            print("还在调用工具...")
        else:
            print("完成！最终答案：", msg.content)
```

问题5：Agent 可以调用多少次工具？

默认没有限制，直到得到最终答案。但可能会：

- 超时
- 达到 token 限制
- 模型决定停止

问题6：如何限制工具调用次数？

LangChain 1.0 的 create_agent 默认使用 LangGraph，可以通过配置限制：

```
In [ ]: # 注意：这是高级用法，后续会详细学习
config = {
    "recursion_limit": 5 # 最多 5 步
}

response = agent.invoke(input, config=config)
```

5.Agent的高级用法1：设置Agent名称

创建Agent时，LangChain允许用户指定其名称

5.1 用法

```
In [13]: from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
import os
from rich import print as rprint

# 从 .env 文件中加载环境变量
load_dotenv(override=True)

model = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL")
)

agent = create_agent(
    model=model,
    name="chat_assistant"
)
response = agent.invoke({"messages": ["你好"]})

# rprint(response)
for msg in response["messages"]:
    msg.pretty_print()
```

===== Human Message =====

你好

===== Ai Message =====

Name: chat_assistant

你好！有什么可以帮你的吗？

输出的AI Message带有Name信息。

5.2 经典使用场景

name 在 Multi-Agent 场景中最常被提及，用于区分不同的 Agent。但它的作用并不局限于多 Agent 编排。在实际工程中，出现如下场景，通常都建议为 Agent 设置一个清晰且稳定的 name。

1. 流式输出归因

在启用流式输出时，name 可用于标识当前输出内容来自哪个 Agent。

这得多 Agent 协作、Agent 嵌套调用，或前端需要实时展示不同执行主体输出时尤其有用，便于准确区分 token 或事件的来源。

2. 消息身份标记

设置 name 后，Agent 产生的 AIMessage 会携带对应的name信息。这使得系统在保存会话记录、回放执行过程、构建审计日志或前端展示消息角色时，能够明确识别消息的生成者。

3. 调试与trace可读性

在调试、日志分析和链路追踪过程中，name 可以作为 Agent 的稳定标识，帮助开发者快速判断当前执行的是哪个 Agent。

当系统中存在多个能力相近的 Agent，或一个 Agent 被嵌套在更复杂的工作流中时，名称能够显著提升trace 的可读性和问题定位效率。

4. 组件化封装

在工程实践中，Agent 常被封装为可复用的能力模块，例如检索助手、SQL 助手、报告生成助手等。

为 Agent 设置 name，有助于在模块注册、运行监控、日志归档和能力复用时保持一致的身份标识。

如果后续需要将该 Agent 进一步作为子图节点、工具能力或子模块接入更复杂系统，也能降低维护和迁移成本。

5. 前端展示与运行态可观测性

在带有可视化界面的应用中，name 还可以直接作为运行时展示标识使用。

例如，在执行面板中显示“当前活跃 Agent”“本轮输出来源”或“调用链路中的执行节点”时，name 能帮助开发者和用户更直观地理解系统当前的执行状态。

6. 作为稳定的运行时身份标识

从更通用的角度看，name 可以理解为 Agent 在系统中的“运行时身份 ID”。

相比临时性的展示名称，一个稳定、规范的 name 更适合用于日志检索、监控统计、链路分析和跨模块协作，因此在生产环境中通常建议显式设置，而不是依赖默认行为。

6.Agent的高级用法2：系统提示词

使用 create_agent 创建 Agent 时，需传入模型和工具、可选地传入系统提示词。**提示词为 Agent提供了任务背景、行为准则和操作指南。**

系统指令，即SystemMessage，通过 system_prompt 设置，定义 Agent 行为。这个参数可以是str或者SystemMessage类型。

使用建议：

- 明确说明 Agent 的角色
- 定义输出格式
- 说明何时使用工具

比如：

```
In [ ]: agent = create_agent(  
        model=model,
```

```

tools=[get_weather],
system_prompt="""你是天气助手。
工作流程：
1. 理解用户的城市查询
2. 使用 get_weather 工具获取数据
3. 简洁清晰地回答

输出格式：
- 天气状况
- 温度
- 注意事项（如有）
"""
)

```

提示词设置有两种方式：静态设置和动态设置。动态设置需要借助中间件，后续讲解。

举例1：

```

In [14]: from langchain.chat_models import init_chat_model
from langchain.agents import create_agent
from langchain_tavily import TavilySearch
from dotenv import load_dotenv
import os
from rich import print as rprint

load_dotenv(override=True)

# 1. 模型初始化
model = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL")
)

# 2. 工具实例化
web_search = TavilySearch(
    tavily_api_key=os.getenv("TAVILY_API_KEY"),
    max_results=2)

# 3. 创建Agent
agent = create_agent(
    model=model,
    tools=[web_search],
    system_prompt="你是一名多才多艺的智能助手，可以调用工具帮助用户解决问题。"
)

# 4. 运行Agent 获得结果
result = agent.invoke({
    "messages":[
        {"role":"user","content":"请帮我查询2024年诺贝尔物理学奖得主是谁？"}
    ]
})

# rprint(result)
print(result['messages'][-1].content)

```

根据查询结果，**2024年诺贝尔物理学奖**的获奖者是：

🏆 获奖者

1. **约翰·J·霍普菲尔德** (John J. Hopfield, 美国普林斯顿大学)
2. **杰弗里·E·辛顿** (Geoffrey E. Hinton, 加拿大多伦多大学)

🎯 获奖理由

他们因**"通过人工神经网络实现机器学习的基础性发现与发明"**而获奖。

📖 贡献简介

- **霍普菲尔德**：发明了以其名字命名的"霍普菲尔德网络" (Hopfield Network)，这是一种能够存储和重建信息的联想记忆模型，其物理思想深受自旋玻璃 (spin glass) 等统计物理模型的启发。
- **辛顿**：被誉为"人工智能教父"，他基于霍普菲尔德网络发展了**玻尔兹曼机** (Boltzmann machine)，为现代深度学习的发展奠定了理论基础。

值得一提的是，这两位科学家的获奖也引发了广泛讨论——因为这是物理学奖历史上首次授予与人工智能和机器学习直接相关的研究成果，标志着物理学方法（如统计力学）与计算机科学的深度融合。

需要我进一步介绍他们具体的科学贡献，或者查询其他年份的诺贝尔奖信息吗？

举例2：

```
In [15]: from langchain.agents import create_agent
from langchain_core.messages import SystemMessage
from langchain_core.tools import tool
from rich import print as rprint

# 工具：实现两数相加
@tool
def add_numbers(a: int, b: int) -> str:
    """计算并返回两个数的和。"""
    return f"和为：{a + b}"

# 创建客服助手Agent
agent = create_agent(
    model=model,
    tools=[add_numbers], # 工具列表
    # system_prompt="你是一个数学助手，解决日常的算术问题"
    system_prompt=SystemMessage(content="你是一个数学助手，解决日常的算术问题")
)
response = agent.invoke(
    {"messages": [
        {"role": "user", "content": "10加上20再加上30是多少？"}
    ]},
)

rprint(response)
# print(response["messages"][-1].content)
```

```

{
  'messages': [
    HumanMessage(
      content='10加上20再加上30是多少？',
      additional_kwargs={},
      response_metadata={},
      id='72a032de-87a8-47e2-a580-49506b7798bf'
    ),
    AIMessage(
      content='',
      additional_kwargs={
        'refusal': None,
        'reasoning_content': 'The user asks: 10 + 20 + 30 =
?\\n\\nI can use the add_numbers tool. But it
only takes two numbers at a time. I can add 10 and 20 first, then
add 30 to the result. These are dependent, so I
must call sequentially.\\n\\nLet me first add 10 + 20.'
      },
      response_metadata={
        'token_usage': {
          'completion_tokens': 130,
          'prompt_tokens': 382,
          'total_tokens': 512,
          'completion_tokens_details': {
            'accepted_prediction_tokens': None,
            'audio_tokens': None,
            'reasoning_tokens': 70,
            'rejected_prediction_tokens': None
          },
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0},
          'prompt_cache_hit_tokens': 0,
          'prompt_cache_miss_tokens': 382
        },
        'model_provider': 'deepseek',
        'model_name': 'deepseek-v4-flash',
        'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
        'id': 'd2584c67-db97-4b9e-9187-d308de0398b3',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff45b-649c-7920-8cab-c75b87050009-0',
      tool_calls=[
        {
          'name': 'add_numbers',
          'args': {'a': 10, 'b': 20},
          'id': 'call_00_mcTMZUcbqe7HsVnUnjem0663',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 382,
        'output_tokens': 130,
        'total_tokens': 512,
        'input_token_details': {'cache_read': 0},
        'output_token_details': {'reasoning': 70}
      }
    ),
    ToolMessage(
      content='和为：30',
      name='add_numbers',
      id='dd3c3fb3-951b-4216-8109-972aed3d995a',

```

```

        tool_call_id='call_00_mcTMZUcbqe7HsVnUnjem0663'
    ),
    AIMessage(
        content='',
        additional_kwargs={'refusal': None, 'reasoning_content':
'Now add 30 to the result 30.'},
        response_metadata={
            'token_usage': {
                'completion_tokens': 70,
                'prompt_tokens': 527,
                'total_tokens': 597,
                'completion_tokens_details': {
                    'accepted_prediction_tokens': None,
                    'audio_tokens': None,
                    'reasoning_tokens': 10,
                    'rejected_prediction_tokens': None
                },
                'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 512},
                'prompt_cache_hit_tokens': 512,
                'prompt_cache_miss_tokens': 15
            },
            'model_provider': 'deepseek',
            'model_name': 'deepseek-v4-flash',
            'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
            'id': '814ab25e-cee6-4086-88a7-c0fb87351ccf',
            'finish_reason': 'tool_calls',
            'logprobs': None
        },
        id='lc_run--019ff45b-6abf-7791-9375-91c7f525f7c0-0',
        tool_calls=[
            {
                'name': 'add_numbers',
                'args': {'a': 30, 'b': 30},
                'id': 'call_00_TTSvv1kI3dICX1XLfc7k8310',
                'type': 'tool_call'
            }
        ],
        invalid_tool_calls=[],
        usage_metadata={
            'input_tokens': 527,
            'output_tokens': 70,
            'total_tokens': 597,
            'input_token_details': {'cache_read': 512},
            'output_token_details': {'reasoning': 10}
        }
    ),
    ToolMessage(
        content='和为：60',
        name='add_numbers',
        id='e8f908b9-887e-4f78-95f3-bffa62198f8e',
        tool_call_id='call_00_TTSvv1kI3dICX1XLfc7k8310'
    ),
    AIMessage(
        content='10 + 20 + 30 = **60**。 ',
        additional_kwargs={'refusal': None, 'reasoning_content':
''},
        response_metadata={
            'token_usage': {
                'completion_tokens': 13,
                'prompt_tokens': 612,
                'total_tokens': 625,
                'completion_tokens_details': {

```

```

        'accepted_prediction_tokens': None,
        'audio_tokens': None,
        'reasoning_tokens': 0,
        'rejected_prediction_tokens': None
    },
    'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 512},
    'prompt_cache_hit_tokens': 512,
    'prompt_cache_miss_tokens': 100
},
    'model_provider': 'deepseek',
    'model_name': 'deepseek-v4-flash',
    'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
    'id': '4ff494b1-7859-4607-8fe7-44f06049674a',
    'finish_reason': 'stop',
    'logprobs': None
},
    id='lc_run--019ff45b-6ead-7ac2-9b49-b6b1ba24c7cf-0',
    tool_calls=[],
    invalid_tool_calls=[],
    usage_metadata={
        'input_tokens': 612,
        'output_tokens': 13,
        'total_tokens': 625,
        'input_token_details': {'cache_read': 512},
        'output_token_details': {'reasoning': 0}
    }
}
    )
}

```

举例3：

```

In [16]: from langchain.agents import create_agent
from langchain.tools import tool
from langchain.messages import SystemMessage, HumanMessage

flag = 0

@tool
def get_weather(city: str):
    """
    天气查询工具

    Args:
        city: 城市名称
    """
    global flag
    flag += 1

    if flag < 3:
        # raise Exception("暂时无法访问")
        return "TEMP_UNAVAILABLE: 天气服务暂时不可用，请稍后重试"
    return f"{city}今天天气挺好"

messages = [
    HumanMessage("你好，杭州今天的天气如何？")
]
agent = create_agent(
    model=model,

```

```

tools=[get_weather],
system_prompt=SystemMessage(
    "你是一个天气助手。"
    "当工具返回以 'TEMP_UNAVAILABLE:' 开头的结果时，"
    "说明是临时故障，不要立即放弃；"
    "你应再次调用同一个工具，最多重试 3 次。"
    "如果 3 次后仍失败，再向用户说明服务暂时不可用。"
)
)

response = agent.invoke({"messages": messages})
# print(response)
for msg in response["messages"]:
    msg.pretty_print()

```

===== Human Message =====

你好，杭州今天的天气如何？

===== Ai Message =====

Tool Calls:

get_weather (call_00_zlrQR26x8jLm1CBxhXEL2194)

Call ID: call_00_zlrQR26x8jLm1CBxhXEL2194

Args:

city: 杭州

===== Tool Message =====

Name: get_weather

TEMP_UNAVAILABLE: 天气服务暂时不可用，请稍后重试

===== Ai Message =====

Tool Calls:

get_weather (call_00_MvUiGGZXOdHSUrZgpbRx3444)

Call ID: call_00_MvUiGGZXOdHSUrZgpbRx3444

Args:

city: 杭州

===== Tool Message =====

Name: get_weather

TEMP_UNAVAILABLE: 天气服务暂时不可用，请稍后重试

===== Ai Message =====

Tool Calls:

get_weather (call_00_QKD8ALxPTbyiRmGvRzFc0840)

Call ID: call_00_QKD8ALxPTbyiRmGvRzFc0840

Args:

city: 杭州

===== Tool Message =====

Name: get_weather

杭州今天天气挺好

===== Ai Message =====

杭州今天的天气挺好！☀️ 适合出门走走～

7.Agent的高级用法3：结构化输出

结构化输出是Agent的核心功能之一，它允许Agent以特定、可预测的格式返回数据，而不是传统的自然语言响应。通过结构化输出，开发者可以直接获得Pydantic模型、JSON对象或数据类等结构化数据，这些数据能够被应用程序直接使用，无需复杂的解析过程。

7.1 模型 vs Agent的结构化输出对比

第06章已经介绍过结构化输出，当时的重点是与模型对象的绑定，这里与Agent的结构化输出做对比：

维度	模型的结构化输出	Agent 结构化输出
操作对象	作用于 大模型对象	作用于 Agent
解析时机	每次模型调用生成 <code>AIMessage</code> 时，进行解析	仅在 Agent 决定「任务结束」并输出最终答案时解析
数据流转	模型 → 结构化对象	模型 → 工具 → 反思 → … → 结构化对象
绑定方式	使用 <code>with_structured_output</code>	使用 <code>response_format</code> 参数
适用场景	单次、确定性 的任务（如提取字段、翻译、分类）	多步、复杂推理 的任务（如查文档后汇总报表）

7.2 结构化输出的4种策略

LangChain的`create_agent()`函数自动处理结构化输出的全过程。用户只需通过“`response_format`”参数设置期望的输出模式（Schema）。当模型生成结构化数据时，系统会自动捕获、验证并将结果存储在Agent状态的`structured_response`键中。

```
In [ ]: def create_agent(  
    ...  
    response_format: Union[  
        ToolStrategy[StructuredResponseT],  
        ProviderStrategy[StructuredResponseT],  
        type[StructuredResponseT],  
        None,  
    ]
```

`create_agent`函数中的`response_format`参数支持四种不同的策略设置方式：

① ProviderStrategy

使用模型提供商的原生结构化输出功能实现结构化输出。

- 这里所说的“原生结构化输出”指的是大语言模型（LLM）提供商通过其API直接提供的、在模型响应阶段就强制保证输出格式符合预定规范的能力，这种能力能够在模型生成内容的源头确保结构化准确性。
- 适用于支持原生结构化输出的模型，比如**OpenAI、Anthropic Claude或xAI Grok**等。

注意“

1. **deepseek-v4-flash** 不支持 OpenAI 新标准 **json_schema** 模式
init_chat_model 默认 with_structured_output(method="json_schema")，会传递 response_format: {"type": "json_schema", ...}，DeepSeek API 直接拒绝。
2. 同时 deepseek-v4-flash 开启思考模式（reasoner）时，**function_calling** + **tool_choice="required"** 也会 400 报错。
3. DeepSeek 仅支持基础模式：response_format={"type": "json_object"}（只保证合法 JSON，不强制校验 Schema）。

举例：

```
In [ ]: from pydantic import BaseModel, Field
        from langchain.agents import create_agent
        from langchain.agents.structured_output import ProviderStrategy
        from langchain.messages import HumanMessage
        from rich import print as rprint
        from langchain.chat_models import init_chat_model
        from dotenv import load_dotenv
        import os

        load_dotenv(override=True)

        # 1. 模型初始化
        model = init_chat_model(
            model="deepseek-v4-flash",
            model_provider="deepseek",
            api_key=os.getenv("DEEPSEEK_API_KEY"),
            base_url=os.getenv("DEEPSEEK_BASE_URL")
        )

        # 2. Pydantic 结构化方式定义
        class ContactInfo(BaseModel):
            """用户的联系方式"""
            name: str = Field(description="用户姓名")
            email: str = Field(description="用户邮箱地址")
            phone: str = Field(description="用户的手机号")

        # 3. agent 初始化
        agent = create_agent(
            model=model,
            response_format=ProviderStrategy(ContactInfo)
        )

        # 4. 调用
        response = agent.invoke({
            "messages": [
                HumanMessage("从这段话中抽取结构化信息：小明的邮箱地址为：shkstart@atguigu.com，手机号：12345678912")
            ]
        })
        # rprint(response)

        for msg in response["messages"]:
            msg.pretty_print()
```

上面的会报错，换成gpt模型不会报错

② ToolStrategy

对于不支持原生结构化输出的模型，LangChain采用“ToolStrategy”工具调用的方式实现结构化输出。

此策略兼容绝大多数支持工具调用的现代模型，其核心原理是动态创建一个“虚拟工具”，该工具的输入参数对应着期望的数据结构。

当模型需要生成最终答案时，系统会引导模型“调用”这个虚拟工具，从而间接产生符合要求的数据。

举例：注意：deepseek-v4-flash 开启思考模式（reasoner）时，

```
** function_calling + tool_choice="required"
```

```
In [20]: from pydantic import BaseModel
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy
from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
import os

load_dotenv(override=True)

# 1. 模型初始化
model = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL"),
    extra_body={
        "thinking":{
            "type":"disabled"
        }
    }
)

# 2. Pydantic结构化方式定义
class ContactInfo(BaseModel):
    name: str = Field(description="姓名")
    email: str = Field(description="邮箱")
    phone: str = Field(description="电话")

# 3. 工具的定义（根据需要定义）
@tool
def search_tool(query: str) -> str:
    """这是一个搜索引擎。当大模型发现给定的上下文里缺少必要的联系人信息，
    需要去互联网上查询时，才会调用这个工具。
    """
    return f"搜索结果: 未找到关于 '{query}' 的更多额外信息。"

# 3. agent 初始化
agent = create_agent(
    model=model,
    tools=[search_tool],
    response_format=ToolStrategy(ContactInfo)
)

result = agent.invoke({
```

```

    "messages": [{"role": "user", "content": "联系人信息: John Doe,john@atguigu.com, (010) 56253825"}]
})

# print(result)
print(result["structured_response"])

```

name='John Doe' email='john@atguigu.com' phone='(010) 56253825'

③ type / AutoStrategy

官方没有参数列表或官方文档列出这种策略，但阅读源码可以看到。

```

In [ ]: ResponseFormat = ToolStrategy[SchemaT] | ProviderStrategy[SchemaT] |
AutoStrategy[SchemaT]
        """Union type for all supported response format strategies."""

```

当我们直接传入一个定义类型时，LangChain会自动包装为AutoStrategy，触发自动选择策略：如果模型支持原生结构化输出（如OpenAI、Anthropic Claude或xAI Grok），则优先使用ProviderStrategy；否则使用ToolStrategy。

举例1：

```

In [21]: from langchain.agents.structured_output import AutoStrategy
        from pydantic import BaseModel, Field
        from langchain.agents import create_agent

        class ContactInfo(BaseModel):
            """联系人信息"""
            name: str = Field(description="姓名")
            email: str = Field(description="邮箱")
            phone: str = Field(description="电话")

        agent = create_agent(
            model=model,
            response_format=AutoStrategy(ContactInfo)
        )

        result = agent.invoke({
            "messages": [{"role": "user", "content": "联系人信息: John Doe,john@atguigu.com, (010) 56253825"}]
        })

        # print(result)
        print(result["structured_response"])

```

name='John Doe' email='john@atguigu.com' phone='(010) 56253825'

特别注意：在LangChain 1.0及以上版本中，直接传递模式（如response_format=ContactInfo）不再支持，必须显式使用ToolStrategy或ProviderStrategy。（经过测试，目前LangChain 1.2版本还可使用）。

举例2：

```

In [22]: from pydantic import BaseModel, Field
        from langchain.agents import create_agent

```

```

class ContactInfo(BaseModel):
    """联系人信息"""
    name: str = Field(description="姓名")
    email: str = Field(description="邮箱")
    phone: str = Field(description="电话")

agent = create_agent(
    model=model,
    response_format=ContactInfo # Auto-selects ProviderStrategy
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "联系人信息: John Doe,john@atguigu.com, (010) 56253825"}]
})

# print(result)
print(result["structured_response"])

```

name='John Doe' email='john@atguigu.com' phone='(010) 56253825'

④ None

默认配置，表示不以结构化输出，以自然语言响应用户问题。

In [27]:

```

from pydantic import BaseModel, Field
from langchain.agents import create_agent

agent = create_agent(
    model=model,
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "联系人信息: John Doe,john@atguigu.com, (010) 56253825"}]
})

# print(result)
print(result)

```

```
{'messages': [HumanMessage(content='联系人信息: John Doe,john@atguigu.com, (010) 56253825', additional_kwargs={}, response_metadata={}, id='b2d2e689-bf85-43fd-8afd-3beaa22afea8'), AIMessage(content='这是一个典型的联系人信息条目。根据您的需求，我可以为您整理成以下清晰、规范的格式，并附上使用建议。\\n\\n### 1. 标准格式（适用于名片、通讯录）\\n\\n`\\`\\`\\n姓名: John Doe\\n邮箱: john@atguigu.com\\n电话: (010) 56253825\\n`\\`\\`\\n\\n### 2. 国际化/紧凑格式（适用于简历、签名档）\\n\\n`\\`\\`\\nJohn Doe\\nE-mail: john@atguigu.com\\nTel: (010) 56253825\\n`\\`\\`\\n\\n### 3. 技术/数据格式（适用于数据库存储或API）\\n\\n如果您需要将其作为数据输入，通常建议拆分字段并标准化电话格式（去掉括号，使用国际区号）:\\n\\n`\\`\\`json\\n{\\n  "name": "John Doe",\\n  "email": "john@atguigu.com",\\n  "phone": "+86-10-56253825"\\n}\\n`\\`\\`\\n\\n---\\n\\n**补充说明（针对您提供的信息）:**\\n\\n* **姓名** : John Doe 是一个英文通用名，通常用于示例或占位。\\n* **邮箱后缀** : `atguigu.com` 表明这可能是与“尚硅谷”（atguigu）相关的邮箱。\\n* **电话格式** : `(010) 56253825` 是中国北京的区号格式，但缺少国际区号 `+86`。如果是用于国际联系，建议补充为 `+86-10-56253825`。\\n\\n**请问您需要我针对这些信息做进一步的操作吗？** 例如:\\n* 检测这个邮箱或电话是否有效？\\n* 将其转换为特定软件的导入格式（如 CSV）?', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 339, 'prompt_tokens': 24, 'total_tokens': 363, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}, 'prompt_cache_hit_tokens': 0, 'prompt_cache_miss_tokens': 24}, 'model_provider': 'deepseek', 'model_name': 'deepseek-v4-flash', 'system_fingerprint': 'fp_a18b46594c_prod0820_fp8_kvcache_20260402', 'id': '5dfdcf7c-9689-4a8e-b81d-a579175d5539', 'finish_reason': 'stop', 'logprobs': None}, id='lc_run--019ff485-de23-7dc0-a426-a36e30bceab-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 24, 'output_tokens': 339, 'total_tokens': 363, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}}])}]}
```

总结：在实际大模型Agent开发场景中，如果使用到了结构化输出，推荐使用“ToolStrategy”策略，所以后续重点介绍这种策略方式结构化输出。

7.3 ToolStrategy使用详解

ToolStrategy通过工具调用（Tool Calling/Function Calling）实现结构化输出，所以LangChain会在消息列表末尾追加一条ToolMessage，让整个链路完整。但实际上没有实际的工具执行，这是一条伪消息。

ToolStrategy适用于任何支持工具调用的现代模型。

ToolStrategy的配置包含三个主要参数：

```
In [ ]: class ToolStrategy(Generic[SchemaT]):
        schema: type[SchemaT]
        tool_message_content: str | None
        handle_errors: Union[bool, str, type[Exception], tuple[type[Exception], ...], Callable[[Exception], str]]
```

- **schema**（必需参数）：与提供商策略的schema参数功能一致，支持Pydantic模型、TypedDict、JSON Schema、数据类(@dataclass)，同时还支持联合类型Union[类型1, 类型2]（允许模型根据输入内容选择最匹配的数据结构）。
- **tool_message_content**（可选参数）：用于自定义生成结构化输出时，会话历史中记录的提示信息。默认使用展示输出数据的标准响应语句。
- **handle_errors**（可选参数）：用于指定数据校验失败时的重试策略，默认值为True。

1) 结构化输出：schema参数

下面演示四种Schema进行结构化输出代码实现。

提前说明：因为涉及到不同的Schema在不同模型供应商下表现的支持力度不同

```
In [28]: from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
import os

load_dotenv(override=True)

# 1. 模型初始化
model_ds = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL"),
    extra_body={
        "thinking":{
            "type":"disabled"
        }
    }
)
```

```
In [29]: from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
import os

load_dotenv(override=True)

# 1. 模型初始化
model_bailian = init_chat_model(
    model="deepseek-v4-flash-0731",
    model_provider="openai",
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url=os.getenv("DASHSCOPE_BASE_URL"),
    extra_body={
        "thinking":{
            "type":"disabled"
        }
    }
)
```

输出模式1：Pydantic类型

Pydantic类型的Schema支持数据验证，是优先推荐使用的方式。

举例1：

```
In [30]: from pydantic import BaseModel, Field
from langchain.agents.structured_output import ToolStrategy
from langchain.agents import create_agent
from langchain.messages import HumanMessage

class ContactInfo(BaseModel):
    """用户的联系方式"""
    name: str = Field(description="用户姓名")
    email: str = Field(description="用户邮箱地址")
    phone: str = Field(description="用户的手机号")
```

```

agent = create_agent(
    model=model_ds,
    response_format=ToolStrategy(ContactInfo)
)

response = agent.invoke({
    "messages": [
        HumanMessage("从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com,
手机号：12345678912")
    ]
})
for msg in response["messages"]:
    msg.pretty_print()
# print(response["structured_response"])

```

===== Human Message =====

从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com，手机号：12345678912

===== Ai Message =====

Tool Calls:

ContactInfo (call_00_uT2UMF50MfZjmzEFWgRp1485)

Call ID: call_00_uT2UMF50MfZjmzEFWgRp1485

Args:

name: 小明

email: songhk@atguigu.com

phone: 12345678912

===== Tool Message =====

Name: ContactInfo

Returning structured response: name='小明' email='songhk@atguigu.com' phone='12345678912'

```

In [31]: from pydantic import BaseModel, Field
from langchain.agents.structured_output import ToolStrategy
from langchain.agents import create_agent
from langchain.messages import HumanMessage

class ContactInfo(BaseModel):
    """用户的联系方式"""
    name: str = Field(description="用户姓名")
    email: str = Field(description="用户邮箱地址")
    phone: str = Field(description="用户的手机号")

agent = create_agent(
    model=model_bailian,
    response_format=ToolStrategy(ContactInfo)
)

response = agent.invoke({
    "messages": [
        HumanMessage("从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com,
手机号：12345678912")
    ]
})
for msg in response["messages"]:
    msg.pretty_print()
# print(response["structured_response"])

```

===== Human Message =====

从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com，手机号：12345678912

===== Ai Message =====

Tool Calls:

ContactInfo (call_0c47eda6bd7a4ec2bb892fcd)

Call ID: call_0c47eda6bd7a4ec2bb892fcd

Args:

email: songhk@atguigu.com

name: 小明

phone: 12345678912

===== Tool Message =====

Name: ContactInfo

Returning structured response: name='小明' email='songhk@atguigu.com' phone='12345678912'

观察日志可知，这种方式将结构化信息作为伪工具传递，显然使用了Function Calling方法。

举例2：

```
In [32]: from langchain_core.messages import SystemMessage
from pydantic import BaseModel, Field
from typing import Literal
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy
from langchain.tools import tool
from rich import print as rprint

# 定义工具
@tool(parse_docstring=True)
def search_customer_database(query: str) -> str:
    """
    在客户数据库中搜索信息

    Args:
        query (str): 客户查询字符串，例如 "张三" 或 "李四"

    Returns:
        str: 客户记录字符串，包含客户姓名、等级、最近购买日期和累计消费
    """
    # 模拟数据库查询结果
    if "张三" in query.lower():
        return "客户记录：张三，VIP客户，最近购买日期：2026-01-15，累计消费：$15,000"
    elif "李四" in query.lower():
        return "客户记录：李四，普通客户，最近购买日期：2025-12-20，累计消费：$3,200"
    else:
        return f"关于客户{query}，无记录"

@tool(parse_docstring=True)
def send_email(customer: str) -> str:
    """
    发送感谢邮件

    Args:
        customer (str): 客户名称，例如 "张三" 或 "李四"

    Returns:
        str: 确认消息，包含已发送的客户名称
```

```

"""
return f"已向 {customer} 发送感谢邮件"

# 定义Pydantic Schema
class CustomerAnalysis(BaseModel):
    """客户分析报告"""
    customer_name: str = Field(None, description="客户姓名")
    customer_tier: Literal["潜在客户", "普通客户", "VIP客户", "流失风险"] = Field("潜在客户", description="客户等级,只能是潜在客户、普通客户、VIP客户或流失风险")
    recent_activity: str = Field(None, description="最近活动")
    spending_level: Literal["低", "中", "高"] = Field(None, description="消费水平")
    send_email: bool = Field(False, description="是否已发送感谢邮件")

# 创建智能体
agent = create_agent(
    model=model_ds,
    system_prompt=SystemMessage(content="""
        请分析指定客户的情况：
        1. 先搜索客户数据库了解最新情况
        2. 如果是VIP客户，则发送感谢邮件
        3. 基于搜索结果生成结构化分析报告
        4. 如果用户提问与客户记录无关或找不到客户信息，则返回空对象，不发送感谢邮件
    """),
    tools=[search_customer_database, send_email],
    response_format=ToolStrategy(CustomerAnalysis)
)

# 执行分析
result = agent.invoke({
    "messages": [{"role": "user", "content": "请分析客户张三"}]
    # "messages": [{"role": "user", "content": "请分析客户李四"}]
    # "messages": [{"role": "user", "content": "请分析客户王五"}]
    # "messages": [{"role": "user", "content": "今天天气如何"}]
})

# 处理结果
# pprint(result)

if "structured_response" in result:
    analysis = result["structured_response"]
    print(analysis)

```

customer_name='张三' customer_tier='VIP客户' recent_activity='最近购买日期：2026-01-15'
 spending_level='高' send_email=True

注意：

1. 如果是结构化输出，在系统提示词中最后提示结构化输出结果，如果提示词中先结构化输出结果（Agent已经执行完成），可能会导致一些工具不会再被调用。
2. 系统提示词中最后加入“未找到用户”时的处理提示，避免程序一直调用工具尝试查找对应用户信息。

输出模式2：TypedDict类型

TypedDict 允许为字典对象定义固定的键名和对应的值类型，是带有类型提示的字典结构。具体的：

1. TypedDict字段定义采用 Annotated[类型, 默认值, "描述"]格式
2. 可选字段使用 Optional包装，默认值在 Annotated 中指定
3. TypedDict 不支持运行时验证。

举例1：

```
In [34]: from typing import Annotated
from typing_extensions import TypedDict
from langchain.agents.structured_output import ToolStrategy
from langchain.agents import create_agent
from langchain.messages import HumanMessage

class ContactInfo(TypedDict):
    """用户的联系方式"""
    name: Annotated[str, ..., "用户姓名"]
    email: Annotated[str, ..., "用户邮箱地址"]
    phone: Annotated[str, ..., "用户的手机号"]

agent = create_agent(
    model=model_ds,
    response_format=ToolStrategy(ContactInfo)
)

response = agent.invoke({
    "messages": [
        HumanMessage("从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com，手机号：12345678912")
    ]
})

for msg in response["messages"]:
    msg.pretty_print()
# print(response["structured_response"])
```

===== Human Message =====

从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com，手机号：12345678912

===== Ai Message =====

Tool Calls:

ContactInfo (call_00_R1PORamTJzGmU7tAT6aA3564)

Call ID: call_00_R1PORamTJzGmU7tAT6aA3564

Args:

name: 小明

email: songhk@atguigu.com

phone: 12345678912

===== Tool Message =====

Name: ContactInfo

Returning structured response: {'name': '小明', 'email': 'songhk@atguigu.com', 'phone': '12345678912'}

举例2：

```
In [37]: from langchain_core.messages import SystemMessage
from typing import Literal, Optional
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy
from langchain.tools import tool
```

```

# 定义工具
@tool(parse_docstring=True)
def search_customer_database(query: str) -> str:
    """
    在客户数据库中搜索信息

    Args:
        query (str): 客户查询字符串, 例如 "张三" 或 "李四"

    Returns:
        str: 客户记录字符串, 包含客户姓名、等级、最近购买日期和累计消费
    """
    # 模拟数据库查询结果
    if "张三" in query.lower():
        return "客户记录: 张三, VIP客户, 最近购买日期: 2026-01-15, 累计消费: $15,000"
    elif "李四" in query.lower():
        return "客户记录: 李四, 普通客户, 最近购买日期: 2025-12-20, 累计消费: $3,200"
    else:
        return f"关于客户{query}, 无记录"

@tool(parse_docstring=True)
def send_email(customer: str) -> str:
    """
    发送感谢邮件

    Args:
        customer (str): 客户名称, 例如 "张三" 或 "李四"

    Returns:
        str: 确认消息, 包含已发送的客户名称
    """
    return f"已向 {customer} 发送感谢邮件"

# 使用 TypedDict 定义客户分析报告 Schema
class CustomerAnalysis(TypedDict):
    """客户分析报告"""
    customer_name: Annotated[Optional[str], None, "客户姓名"]
    customer_tier: Annotated[Literal["潜在客户", "普通客户", "VIP客户", "流失风险"], "潜在客户", "客户等级"]
    recent_activity: Annotated[Optional[str], None, "最近活动"]
    spending_level: Annotated[Optional[Literal["低", "中", "高"]], None, "消费水平"]
    send_email: Annotated[bool, False, "是否已发送感谢邮件"]

# 创建智能体
agent = create_agent(
    model=model_ds,
    system_prompt=SystemMessage(content="""
    请分析指定客户的情况:
    "1. 先搜索客户数据库了解最新情况"
    "2. 如果是VIP客户, 则发送感谢邮件"
    "3. 基于搜索结果生成结构化分析报告"
    "4. 如果用户提问与客户记录无关或找不到客户信息, 则返回空对象, 不发送感谢邮件"
    ""),
    tools=[search_customer_database, send_email],
    response_format=ToolStrategy(CustomerAnalysis)
)

# 执行分析
result = agent.invoke({

```

```
"messages": [{"role": "user", "content": "请分析客户张三"}]
# "messages": [{"role": "user", "content": "请分析客户李四"}]
# "messages": [{"role": "user", "content": "请分析客户王五"}]
# "messages": [{"role": "user", "content": "今天天气如何"}]
})

# 处理结果
rprint("result:", result)
if "structured_response" in result:
    analysis = result["structured_response"]
    print(analysis)
```

```

result:
{
  'messages': [
    HumanMessage(
      content='请分析客户张三',
      additional_kwargs={},
      response_metadata={},
      id='d1eae571-f150-4d1f-a1c5-a8898638e525'
    ),
    AIMessage(
      content='',
      additional_kwargs={'refusal': None},
      response_metadata={
        'token_usage': {
          'completion_tokens': 39,
          'prompt_tokens': 610,
          'total_tokens': 649,
          'completion_tokens_details': None,
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 512},
          'prompt_cache_hit_tokens': 512,
          'prompt_cache_miss_tokens': 98
        },
        'model_provider': 'deepseek',
        'model_name': 'deepseek-v4-flash',
        'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
        'id': 'a69ede21-d28f-49ce-91e8-10c41d512a34',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff4a8-cc05-7810-a853-2244598cedde-0',
      tool_calls=[
        {
          'name': 'search_customer_database',
          'args': {'query': '张三'},
          'id': 'call_00_ddF2VAA4u4rXg58AeiJv6899',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 610,
        'output_tokens': 39,
        'total_tokens': 649,
        'input_token_details': {'cache_read': 512},
        'output_token_details': {}
      }
    ),
    ToolMessage(
      content='客户记录：张三，VIP客户，最近购买日期：2026-01-15，累
计消费：$15,000',
      name='search_customer_database',
      id='ec426516-ed44-4b9a-a0eb-762f543f563b',
      tool_call_id='call_00_ddF2VAA4u4rXg58AeiJv6899'
    ),
    AIMessage(
      content='',
      additional_kwargs={'refusal': None},
      response_metadata={
        'token_usage': {
          'completion_tokens': 36,
          'prompt_tokens': 694,

```

```

        'total_tokens': 730,
        'completion_tokens_details': None,
        'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 640},
        'prompt_cache_hit_tokens': 640,
        'prompt_cache_miss_tokens': 54
    },
    'model_provider': 'deepseek',
    'model_name': 'deepseek-v4-flash',
    'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
    'id': '3f2bfc75-e38f-4d68-a8ec-06c184143932',
    'finish_reason': 'tool_calls',
    'logprobs': None
},
id='lc_run--019ff4a8-d0d1-7591-be77-a2b5abf66e5b-0',
tool_calls=[
    {
        'name': 'send_email',
        'args': {'customer': '张三'},
        'id': 'call_00_38yxRWwOZ4fNuTqNiwqg7733',
        'type': 'tool_call'
    }
],
invalid_tool_calls=[],
usage_metadata={
    'input_tokens': 694,
    'output_tokens': 36,
    'total_tokens': 730,
    'input_token_details': {'cache_read': 640},
    'output_token_details': {}
}
),
ToolMessage(
    content='已向 张三 发送感谢邮件',
    name='send_email',
    id='b7600850-57df-44db-abd3-3fe7c378d481',
    tool_call_id='call_00_38yxRWwOZ4fNuTqNiwqg7733'
),
AIMessage(
    content='',
    additional_kwargs={'refusal': None},
    response_metadata={
        'token_usage': {
            'completion_tokens': 119,
            'prompt_tokens': 757,
            'total_tokens': 876,
            'completion_tokens_details': None,
            'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 640},
            'prompt_cache_hit_tokens': 640,
            'prompt_cache_miss_tokens': 117
        },
        'model_provider': 'deepseek',
        'model_name': 'deepseek-v4-flash',
        'system_fingerprint':
'fp_a18b46594c_prod0820_fp8_kvcache_20260402',
        'id': 'cf04ceda-f962-42e4-a10f-fa1c912c58ee',
        'finish_reason': 'tool_calls',
        'logprobs': None
    },
    id='lc_run--019ff4a8-d51f-7c60-8ad3-c9e5654bf83f-0',
    tool_calls=[
        {

```

```

        'name': 'CustomerAnalysis',
        'args': {
            'customer_name': '张三',
            'customer_tier': 'VIP客户',
            'recent_activity': '最近购买日期：2026-01-15',
            'spending_level': '高',
            'send_email': True
        },
        'id': 'call_00_PXFX2LxbHTUIdhiOGXgb9193',
        'type': 'tool_call'
    },
    ],
    invalid_tool_calls=[],
    usage_metadata={
        'input_tokens': 757,
        'output_tokens': 119,
        'total_tokens': 876,
        'input_token_details': {'cache_read': 640},
        'output_token_details': {}
    }
),
ToolMessage(
    content="Returning structured response:
{'customer_name': '张三', 'customer_tier': 'VIP客户',
'recent_activity': '最近购买日期：2026-01-15', 'spending_level': '高',
'send_email': True}",
    name='CustomerAnalysis',
    id='0f535ec4-ae9a-44e8-886d-d9d9e0d450d8',
    tool_call_id='call_00_PXFX2LxbHTUIdhiOGXgb9193'
)
],
'structured_response': {
    'customer_name': '张三',
    'customer_tier': 'VIP客户',
    'recent_activity': '最近购买日期：2026-01-15',
    'spending_level': '高',
    'send_email': True
}
}
{'customer_name': '张三', 'customer_tier': 'VIP客户', 'recent_activity': '最近购买日期：2026-01-15',
'spending_level': '高', 'send_email': True}

```

输出模式3：JsonSchema类型

JSON Schema是提供一个标准的 JSON Schema 字典来定义结构。适合需要与多种编程语言交互或进行复杂数据约束定义的场景。

举例1：

```

In [38]: from langchain.agents import create_agent
from langchain.messages import HumanMessage

json_schema = {
    "title": "ContactInfo",
    "description": "用户的联系方式",
    "type": "object",
    "properties": {
        "name": {
            "description": "用户姓名",

```

```

        "type": "string"
    },
    "email": {
        "description": "用户邮箱地址",
        "type": "string"
    },
    "phone": {
        "description": "用户的手机号",
        "type": "string"
    }
},
"required": [
    "name",
    "email",
    "phone"
]
}
agent = create_agent(
    model=model,
    response_format=ToolStrategy(json_schema)
)

response = agent.invoke(
    {
        "messages": [
            HumanMessage("从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com，手机号：12345678912")
        ]
    }
)
for msg in response["messages"]:
    msg.pretty_print()
# print(response["structured_response"])

```

===== Human Message =====

从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com，手机号：12345678912

===== Ai Message =====

Tool Calls:

ContactInfo (call_00_6zAEJTbWpDxqJvfxalhA5920)

Call ID: call_00_6zAEJTbWpDxqJvfxalhA5920

Args:

name: 小明

email: songhk@atguigu.com

phone: 12345678912

===== Tool Message =====

Name: ContactInfo

Returning structured response: {'name': '小明', 'email': 'songhk@atguigu.com', 'phone': '12345678912'}

举例2：

```

In [39]: from langchain_core.messages import SystemMessage
        from typing import Literal, Optional
        from langchain.agents import create_agent
        from langchain.agents.structured_output import ToolStrategy
        from langchain.tools import tool

        # 定义工具

```

```

@tool(parse_docstring=True)
def search_customer_database(query: str) -> str:
    """
    在客户数据库中搜索信息

    Args:
        query (str): 客户查询字符串, 例如 "张三" 或 "李四"

    Returns:
        str: 客户记录字符串, 包含客户姓名、等级、最近购买日期和累计消费
    """
    # 模拟数据库查询结果
    if "张三" in query.lower():
        return "客户记录：张三, VIP客户, 最近购买日期：2026-01-15, 累计消费：$15,000"
    elif "李四" in query.lower():
        return "客户记录：李四, 普通客户, 最近购买日期：2025-12-20, 累计消费：$3,200"
    else:
        return f"关于客户{query}, 无记录"

@tool(parse_docstring=True)
def send_email(customer: str) -> str:
    """
    发送感谢邮件

    Args:
        customer (str): 客户名称, 例如 "张三" 或 "李四"

    Returns:
        str: 确认消息, 包含已发送的客户名称
    """
    return f"已向 {customer} 发送感谢邮件"

# 定义 JSON Schema 替代 Pydantic 模型
customer_analysis_schema = {
    "title": "CustomerAnalysis",
    "type": "object",
    "description": "客户分析报告",
    "properties": {
        "customer_name": {
            "type": "string",
            "default": "",
            "description": "客户姓名"
        },
        "customer_tier": {
            "type": "string",
            "enum": ["潜在客户", "普通客户", "VIP客户", "流失风险"],
            "default": "潜在客户",
            "description": "客户等级"
        },
        "recent_activity": {
            "type": "string",
            "default": "",
            "description": "最近活动"
        },
        "spending_level": {
            "type": "string",
            "enum": ["低", "中", "高"],
            "default": "低",
            "description": "消费水平"
        }
    }
},

```

```

    "send_email": {
        "type": "boolean",
        "default": False,
        "description": "是否已发送感谢邮件"
    }
},
# 所有字段都是必须输出的
"required": ["customer_name", "customer_tier", "recent_activity", "spending_level"]
}

# 创建智能体
agent = create_agent(
    model=model,
    system_prompt=SystemMessage(content="""
        请分析指定客户的情况：
        "1. 先搜索客户数据库了解最新情况 "
        "2. 如果是VIP客户，则发送感谢邮件 "
        "3. 基于搜索结果生成结构化分析报告 "
        "4. 如果用户提问与客户记录无关或找不到客户信息，则返回空对象，不发送感谢邮件"
    """),
    tools=[search_customer_database, send_email],
    response_format=ToolStrategy(customer_analysis_schema)
)

# 执行分析
result = agent.invoke({
    "messages": [{"role": "user", "content": "请分析客户张三"}]
    # "messages": [{"role": "user", "content": "请分析客户李四"}]
    # "messages": [{"role": "user", "content": "请分析客户王五"}]
    # "messages": [{"role": "user", "content": "今天天气如何"}]
})

# 处理结果
# print("result:", result)
if "structured_response" in result:
    analysis = result["structured_response"]
    print(analysis)

```

```
{'customer_name': '张三', 'customer_tier': 'VIP客户', 'recent_activity': '最近购买日期：2026-01-15', 'spending_level': '高', 'send_email': True}
```

注意以上代码中定义json_schema的时候指定的title, description, type, properties, required是遵循JSON Schema 规范的标准关键字，是固定写法。

输出模式4：@dataclass类型

@dataclass是Python 3.7引入的一个装饰器，用于简化数据存储类的定义。

举例1：

```

In [41]: from dataclasses import dataclass
        from langchain.agents import create_agent
        from langchain.messages import HumanMessage

        @dataclass
        class ContactInfo:
            """用户的联系方式"""

```

```

name: str = Field(description="用户姓名")
email: str = Field(description="用户邮箱地址")
phone: str = Field(description="用户的手机号")

agent = create_agent(
    model=model,
    response_format=ToolStrategy(ContactInfo)
)

response = agent.invoke(
    {
        "messages": [
            HumanMessage("从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com，手机号：12345678912")
        ]
    }
)
for msg in response["messages"]:
    msg.pretty_print()
# print(response["structured_response"])

```

===== Human Message =====

从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com，手机号：12345678912

===== Ai Message =====

Tool Calls:

ContactInfo (call_00_SO0tHhRZp6FnRrf9Ua6V1330)

Call ID: call_00_SO0tHhRZp6FnRrf9Ua6V1330

Args:

name: 小明

email: songhk@atguigu.com

phone: 12345678912

===== Tool Message =====

Name: ContactInfo

Returning structured response: ContactInfo(name='小明', email='songhk@atguigu.com', phone='12345678912')

举例2：

```

In [42]: from langchain_core.messages import SystemMessage
from pydantic import BaseModel, Field
from typing import Literal
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy
from langchain.tools import tool

# 定义工具
@tool(parse_docstring=True)
def search_customer_database(query: str) -> str:
    """
    在客户数据库中搜索信息

    Args:
        query (str): 客户查询字符串，例如 "张三" 或 "李四"

    Returns:
        str: 客户记录字符串，包含客户姓名、等级、最近购买日期和累计消费
    """

```

```

# 模拟数据库查询结果
if "张三" in query.lower():
    return "客户记录：张三，VIP客户，最近购买日期：2026-01-15，累计消费：$15,000"
elif "李四" in query.lower():
    return "客户记录：李四，普通客户，最近购买日期：2025-12-20，累计消费：$3,200"
else:
    return f"关于客户{query}，无记录"

@tool(parse_docstring=True)
def send_email(customer: str) -> str:
    """
    发送感谢邮件

    Args:
        customer (str): 客户名称，例如 "张三" 或 "李四"

    Returns:
        str: 确认消息，包含已发送的客户名称
    """
    return f"已向 {customer} 发送感谢邮件"

# 使用Dataclass定义Schema
@dataclass
class CustomerAnalysis:
    """客户分析报告"""
    customer_name: str = Field(None, description="客户姓名")
    customer_tier: Literal["潜在客户", "普通客户", "VIP客户", "流失风险"] = Field("潜在客户",
description="客户等级,只能是潜在客户、普通客户、VIP客户或流失风险")
    recent_activity: str = Field(None, description="最近活动")
    spending_level: Literal["低", "中", "高"] = Field(None, description="消费水平")
    send_email: bool = Field(False, description="是否已发送感谢邮件")

# 创建智能体
agent = create_agent(
    model=model,
    system_prompt=SystemMessage(content="""
    请分析指定客户的情况：
    "1. 先搜索客户数据库了解最新情况 "
    "2. 如果是VIP客户，则发送感谢邮件 "
    "3. 基于搜索结果生成结构化分析报告 "
    "4. 如果用户提问与客户记录无关或找不到客户信息，则返回空对象，不发送感谢邮件"
    """),
    tools=[search_customer_database, send_email],
    response_format=ToolStrategy(CustomerAnalysis)
)

# 执行分析
result = agent.invoke({
    "messages": [{"role": "user", "content": "请分析客户张三"}]
    # "messages": [{"role": "user", "content": "请分析客户李四"}]
    # "messages": [{"role": "user", "content": "请分析客户王五"}]
    # "messages": [{"role": "user", "content": "今天天气如何"}]
})

# 处理结果
# print("result:", result)
if "structured_response" in result:
    analysis = result["structured_response"]
    print(analysis)

```

CustomerAnalysis(customer_name='张三', customer_tier='VIP客户', recent_activity='最近购买日期：2026-01-15', spending_level='高', send_email=True)

多schema联合模式

ToolStrategy允许指定多个类型“ Union[类型1, 类型2] ”这种写法，LLM能够根据输入文本的内容，智能地选择最合适的一个数据模型（Schema）来生成结构化输出，但是最终会只有一种类型输出。

适用于根据不同输入内容，生成不同的结构化输出的场景，但是底层工具转换结构化输出只会转换成一种结构化类型输出。

```
In [43]: from pydantic import BaseModel, Field
from typing import Union
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy
from langchain.messages import HumanMessage

class ContactInfo(BaseModel):
    """用户的联系方式"""
    name: str = Field(description="用户姓名")
    email: str = Field(description="用户邮箱地址")
    phone: str = Field(description="用户的手机号")

class EventInfo(BaseModel):
    """事件详情"""
    event_name: str = Field(description="事件名称")
    date: str = Field(description="事件发生日期")

agent = create_agent(
    model=model,
    response_format=ToolStrategy(
        Union[ContactInfo, EventInfo]
    )
)
response = agent.invoke(
    {
        "messages": [
            HumanMessage("从这段话中抽取结构化信息：小明的邮箱地址为：
shkstart@atguigu.com，手机号：12345678912")
        ]
    }
)
for msg in response["messages"]:
    msg.pretty_print()

print(response["structured_response"])
```

===== Human Message =====

从这段话中抽取结构化信息：小明的邮箱地址为：shkstart@atguigu.com，手机号：12345678912

===== Ai Message =====

Tool Calls:

ContactInfo (call_00_hwj7uGroYvmck7wc7Vk4935)

Call ID: call_00_hwj7uGroYvmck7wc7Vk4935

Args:

name: 小明

email: shkstart@atguigu.com

phone: 12345678912

===== Tool Message =====

Name: ContactInfo

Returning structured response: name='小明' email='shkstart@atguigu.com'

phone='12345678912'

name='小明' email='shkstart@atguigu.com' phone='12345678912'

继续：

```
In [44]: response = agent.invoke(  
    {  
        "messages": [  
            HumanMessage("从这段话中抽取结构化信息：2026年高考报名人数突破1200万")  
        ]  
    }  
)  
for msg in response["messages"]:  
    msg.pretty_print()  
  
print(response["structured_response"])
```

===== Human Message =====

从这段话中抽取结构化信息：2026年高考报名人数突破1200万

===== Ai Message =====

Tool Calls:

EventInfo (call_00_qwSSmwulZIH9AdMvXhU21700)

Call ID: call_00_qwSSmwulZIH9AdMvXhU21700

Args:

event_name: 高考报名

date: 2026

===== Tool Message =====

Name: EventInfo

Returning structured response: event_name='高考报名' date='2026'

event_name='高考报名' date='2026'

2) 自定义工具消息：tool_message_content参数

如果采用ToolStrategy策略处理结构化输出时，LangChain会在消息列表末尾追加一条Tool_message，让整个链路完整。但实际上没有实际的工具执行，这是一条伪消息。

我们可以通过ToolStrategy的 tool_message_content 参数定制其消息内容，将指定的内容写入对话历史的提示信息，这样做的好处如下：

1. 在最终用户可见的对话流中，使用更自然的消息替代原始数据。
2. 用简短的确认信息替代可能很长的数据块，减少token消耗。

举例：

当不设置 `tool_message_content` 时，模型收到的 `ToolMessage` 里就包含了像 `{'name': '张三', 'email': 'zhangsan@email.com'... ...}` 这样的具体数据。当设置了 `tool_message_content` 时，模型收到的 `ToolMessage` 只是一个预定义的确认信息，如“格式化输出成功！”。这种方式节省了上下文窗口的令牌消耗，并且让对话流对最终用户更友好。

说明：无论 `tool_message_content` 如何设置，成功提取的结构化数据最终都会正确存入 `result["structured_response"]` 返回，自定义消息仅影响对话历史中的一条记录。

举例：默认情况

```
In [2]: from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
import os

load_dotenv(override=True)

# 1. 模型初始化
model_bailian = init_chat_model(
    model="deepseek-v4-flash-0731",
    model_provider="openai",
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url=os.getenv("DASHSCOPE_BASE_URL"),
    extra_body={
        "thinking":{
            "type":"disabled"
        }
    }
)

In [4]: from pydantic import BaseModel, Field
from langchain.agents.structured_output import ToolStrategy
from langchain.agents import create_agent
from langchain.messages import HumanMessage
from rich import print as rprint

class ContactInfo(BaseModel):
    """用户的联系方式"""
    name: str = Field(description="用户姓名")
    email: str = Field(description="用户邮箱地址")
    phone: str = Field(description="用户的手机号")

agent = create_agent(
    model=model_bailian,
    response_format=ToolStrategy(ContactInfo)
)
response = agent.invoke({
    "messages": [
        HumanMessage("从这段话中抽取结构化信息：小明的邮箱地址为：songhk@atguigu.com，手机号：12345678912")
    ]
})
rprint(response)
```

```

{
  'messages': [
    HumanMessage(
      content='从这段话中抽取结构化信息：小明的邮箱地址为：
songhk@atguigu.com, 手机号：12345678912',
      additional_kwargs={},
      response_metadata={},
      id='61f5e301-da38-4ab1-a442-d9e2d2daaf92'
    ),
    AIMessage(
      content='',
      additional_kwargs={'refusal': None},
      response_metadata={
        'token_usage': {
          'completion_tokens': 83,
          'prompt_tokens': 344,
          'total_tokens': 427,
          'completion_tokens_details': None,
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0}
        },
        'model_provider': 'openai',
        'model_name': 'deepseek-v4-flash-0731',
        'system_fingerprint': None,
        'id': 'chatcmpl-e97851c4-181c-9cf6-9240-
9742fc8bd0e4',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff625-161d-7412-9b84-acabfdc6bab1-0',
      tool_calls=[
        {
          'name': 'ContactInfo',
          'args': {'email': 'songhk@atguigu.com', 'name':
'小明', 'phone': '12345678912'},
          'id': 'call_128ddd40c68541bc974cda1a',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 344,
        'output_tokens': 83,
        'total_tokens': 427,
        'input_token_details': {'cache_read': 0},
        'output_token_details': {}
      }
    ),
    ToolMessage(
      content="Returning structured response: name='小明'
email='songhk@atguigu.com' phone='12345678912'",
      name='ContactInfo',
      id='0ebec4c8-3a3a-4eb0-b618-e0d409cbd332',
      tool_call_id='call_128ddd40c68541bc974cda1a'
    )
  ],
  'structured_response': ContactInfo(name='小明',
email='songhk@atguigu.com', phone='12345678912')
}

```

自定义tool_message_content

```
In [6]: from pydantic import BaseModel, Field
from langchain.agents.structured_output import ToolStrategy
from langchain.agents import create_agent
from langchain.messages import HumanMessage
from rich import print as rprint

class ContactInfo(BaseModel):
    """用户的联系方式"""
    name: str = Field(description="用户姓名")
    email: str = Field(description="用户邮箱地址")
    phone: str = Field(description="用户的手机号")

agent = create_agent(
    model=model_bailian,
    response_format=ToolStrategy(ContactInfo,tool_message_content="已成功抽取信息")
)

response = agent.invoke({
    "messages": [
        HumanMessage("从这段话中抽取结构化信息：小明的邮箱地址为：
songhk@atguigu.com，手机号：12345678912")
    ]
})
rprint(response)
```

```

{
  'messages': [
    HumanMessage(
      content='从这段话中抽取结构化信息：小明的邮箱地址为：
songhk@atguigu.com, 手机号：12345678912',
      additional_kwargs={},
      response_metadata={},
      id='44d4bb65-383f-431a-8789-6502d342ae44'
    ),
    AIMessage(
      content='',
      additional_kwargs={'refusal': None},
      response_metadata={
        'token_usage': {
          'completion_tokens': 83,
          'prompt_tokens': 344,
          'total_tokens': 427,
          'completion_tokens_details': None,
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 256}
        },
        'model_provider': 'openai',
        'model_name': 'deepseek-v4-flash-0731',
        'system_fingerprint': None,
        'id': 'chatcpl-09db813d-3813-9118-ac4d-
65bdf58dceea',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff627-829f-7650-b200-ad2b83d0ba1b-0',
      tool_calls=[
        {
          'name': 'ContactInfo',
          'args': {'email': 'songhk@atguigu.com', 'name':
'小明', 'phone': '12345678912'},
          'id': 'call_8d5dde27ed6d4d81b7782a8f',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 344,
        'output_tokens': 83,
        'total_tokens': 427,
        'input_token_details': {'cache_read': 256},
        'output_token_details': {}
      }
    ),
    ToolMessage(
      content='已成功抽取信息',
      name='ContactInfo',
      id='e5419066-0b4e-4372-9f34-d12e91e10710',
      tool_call_id='call_8d5dde27ed6d4d81b7782a8f'
    )
  ],
  'structured_response': ContactInfo(name='小明',
email='songhk@atguigu.com', phone='12345678912')
}

```

3) 错误处理：handle_errors参数

受限于模型能力，大模型输出的内容可能并不符合格式要求，ToolStrategy通过其handle_errors参数提供了结构化过程错误处理策略，以下是主要的几种方式及其用途：

- **handle_errors=True**：LangChain默认方式，捕获所有异常，并使用LangChain内置的、信息明确的错误消息模板提示模型重试，确保最终能得到符合预定格式的有效数据。适用于大多数希望自动处理错误的通用场景。
- **handle_errors=False**：关闭自动重试机制，任何异常都会直接抛出，会中断程序运行。
- **handle_errors="自定义字符串"**：捕获所有异常，但使用开发者预设的固定字符串作为错误消息。适用于需要统一、友好的用户提示，或进行特定业务引导的场景。
- **handle_errors=ExceptionType**：仅捕获指定类型(如ValueError) 或元组中的异常类型并进行重试，其他异常直接抛出。适用于需要精准控制，只对特定错误进行重试的场景。
- **handle_errors=callable**：灵活性最高的方式，使用开发者自定义的函数来处理异常，可根据不同的异常类型返回差异化的提示信息。适用于需要复杂、精细化错误处理的场景。

情况1：设置为True/False/固定字符串

设计思路：模型对于单条信息的格式化输出请求，输出了多个工具调用请求。也称为多结构化输出错误。

```
In [7]: from pydantic import BaseModel, Field
from typing import Union
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy
from rich import print as rprint

class ContactInfo(BaseModel):
    """个人联系信息"""
    name: str = Field(description="姓名")
    email: str = Field(description="电子邮箱")

class EventDetails(BaseModel):
    """活动详情"""
    event_name: str = Field(description="活动名称")
    date: str = Field(description="活动日期")

agent = create_agent(
    model=model_bailian,
    response_format=ToolStrategy(
        Union[ContactInfo, EventDetails],
        tool_message_content="提取完成！",
        handle_errors=True
        #handle_errors="请检查输入数据"
    )
)

result = agent.invoke({
    "messages": [{
        "role": "user",
        "content": f"请提取以下文本中内容：姓名：张三，电子邮箱：zhang3@atguigu.com，活动名称：公司年会，活动日期：2026-07-15"
    }]
})
```

```
})
```

```
rprint(result)
```

```

{
  'messages': [
    HumanMessage(
      content='请提取以下文本中内容：姓名：张三，电子邮箱：
zhang3@atguigu.com，活动名称：公司年会，活动日期：
2026-07-15',
      additional_kwargs={},
      response_metadata={},
      id='be4dc1c3-f151-4ca1-ba23-7004e19add12'
    ),
    AIMessage(
      content='',
      additional_kwargs={'refusal': None},
      response_metadata={
        'token_usage': {
          'completion_tokens': 118,
          'prompt_tokens': 404,
          'total_tokens': 522,
          'completion_tokens_details': None,
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0}
        },
        'model_provider': 'openai',
        'model_name': 'deepseek-v4-flash-0731',
        'system_fingerprint': None,
        'id': 'chatcpl-b69a15f5-53f5-940d-a946-
92e0bef7b71c',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff642-1fc4-7040-a704-d0148803cc5d-0',
      tool_calls=[
        {
          'name': 'ContactInfo',
          'args': {'email': 'zhang3@atguigu.com', 'name':
'张三'},
          'id': 'call_22511cd5bac5413fbbb4dd8e',
          'type': 'tool_call'
        },
        {
          'name': 'EventDetails',
          'args': {'date': '2026-07-15', 'event_name': '公
司年会'},
          'id': 'call_3e49282fa80b4242a3129f59',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 404,
        'output_tokens': 118,
        'total_tokens': 522,
        'input_token_details': {'cache_read': 0},
        'output_token_details': {}
      }
    ),
    ToolMessage(
      content='Error: Model incorrectly returned multiple
structured responses (ContactInfo, EventDetails)
when only one is expected.\n Please fix your mistakes.',
      name='ContactInfo',
      id='cbf7a7b0-9726-4f1d-9f37-273256d39144',
      tool_call_id='call_22511cd5bac5413fbbb4dd8e'
    )
  ]
}

```

```

    ),
    ToolMessage(
        content='Error: Model incorrectly returned multiple
structured responses (ContactInfo, EventDetails)
when only one is expected.\n Please fix your mistakes.',
        name='EventDetails',
        id='39dee023-bfc7-421a-b4ec-09a30369f657',
        tool_call_id='call_3e49282fa80b4242a3129f59'
    ),
    AIMessage(
        content='',
        additional_kwargs={'refusal': None},
        response_metadata={
            'token_usage': {
                'completion_tokens': 65,
                'prompt_tokens': 593,
                'total_tokens': 658,
                'completion_tokens_details': None,
                'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 256}
            },
            'model_provider': 'openai',
            'model_name': 'deepseek-v4-flash-0731',
            'system_fingerprint': None,
            'id': 'chatcpl-93756ee9-1294-98c2-9123-
fbb9fb4439ff',
            'finish_reason': 'tool_calls',
            'logprobs': None
        },
        id='lc_run--019ff642-23c6-7c91-b038-b76408dcd42-0',
        tool_calls=[
            {
                'name': 'ContactInfo',
                'args': {'email': 'zhang3@atguigu.com', 'name':
'张三'},
                'id': 'call_1bc7dc1e234e42b1a61fc1bd',
                'type': 'tool_call'
            }
        ],
        invalid_tool_calls=[],
        usage_metadata={
            'input_tokens': 593,
            'output_tokens': 65,
            'total_tokens': 658,
            'input_token_details': {'cache_read': 256},
            'output_token_details': {}
        }
    ),
    ToolMessage(
        content='提取完成!',
        name='ContactInfo',
        id='67b7cc47-3815-441a-b62f-df1bd302c58d',
        tool_call_id='call_1bc7dc1e234e42b1a61fc1bd'
    )
],
    'structured_response': ContactInfo(name='张三',
email='zhang3@atguigu.com')
}

```

1) handle_errors设置为True

运行后结果如下：

ToolMessage(content='Error: Model incorrectly returned multiple structured responses (ContactInfo, EventDetails) when only one is expected.\n Please fix your mistakes.',

模型接收到错误反馈后，再次生成新的调用请求，直至成功或达到内部的最大重试次数。ToolStrategy中自定义的tool_message_content控制的是成功后的消息内容，不影响错误消息内容。

```
In [ ]: from pydantic import BaseModel, Field
        from typing import Union
        from langchain.agents import create_agent
        from langchain.agents.structured_output import ToolStrategy
        from rich import print as rprint

        class ContactInfo(BaseModel):
            """个人联系信息"""
            name: str = Field(description="姓名")
            email: str = Field(description="电子邮箱")

        class EventDetails(BaseModel):
            """活动详情"""
            event_name: str = Field(description="活动名称")
            date: str = Field(description="活动日期")

        agent = create_agent(
            model=model_bailian,
            response_format=ToolStrategy(
                Union[ContactInfo, EventDetails],
                tool_message_content="提取完成！",
                handle_errors=False
                #handle_errors="请检查输入数据"
            )
        )

        result = agent.invoke({
            "messages":[{
                "role":"user",
                "content":"请提取以下文本中内容：姓名：张三，电子邮箱：zhang3@atguigu.com，活动名称：公司年会，活动日期：2026-07-15"
            }]
        })

        rprint(result)
```

2) handle_errors设置为False

运行后结果如下：MultipleStructuredOutputsError: Model incorrectly returned multiple structured responses (ContactInfo, EventDetails) when only one is expected. During task with name 'model' and id '91685c0f-9b1f-9d38-545f-839462aca4e9'

3) handle_errors设置为“请检查输入数据”

运行后结果如下：

```
In [10]: from pydantic import BaseModel, Field
         from typing import Union
```

```

from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy
from rich import print as rprint

class ContactInfo(BaseModel):
    """个人联系信息"""
    name: str = Field(description="姓名")
    email: str = Field(description="电子邮箱")

class EventDetails(BaseModel):
    """活动详情"""
    event_name: str = Field(description="活动名称")
    date: str = Field(description="活动日期")

agent = create_agent(
    model=model_bailian,
    response_format=ToolStrategy(
        Union[ContactInfo, EventDetails],
        tool_message_content="提取完成！",
        handle_errors="请检查输入数据"
    )
)

result = agent.invoke({
    "messages": [{
        "role": "user",
        "content": f"请提取以下文本中内容：姓名：张三，电子邮箱：zhang3@atguigu.com，活动名称：公司年会，活动日期：2026-07-15"
    }]
})

rprint(result)

```

```

{
  'messages': [
    HumanMessage(
      content='请提取以下文本中内容：姓名：张三，电子邮箱：
zhang3@atguigu.com，活动名称：公司年会，活动日期：
2026-07-15',
      additional_kwargs={},
      response_metadata={},
      id='671250a4-1662-4f78-945a-1ea423af7bf7'
    ),
    AIMessage(
      content='',
      additional_kwargs={'refusal': None},
      response_metadata={
        'token_usage': {
          'completion_tokens': 118,
          'prompt_tokens': 404,
          'total_tokens': 522,
          'completion_tokens_details': None,
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 256}
        },
        'model_provider': 'openai',
        'model_name': 'deepseek-v4-flash-0731',
        'system_fingerprint': None,
        'id': 'chatcmpl-bddefd48-968c-975a-a4e3-
c1a5bbd8be34',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff646-021b-72c0-9ca2-23968458471c-0',
      tool_calls=[
        {
          'name': 'ContactInfo',
          'args': {'email': 'zhang3@atguigu.com', 'name':
'张三'},
          'id': 'call_c4c0c568ff514df58078a41b',
          'type': 'tool_call'
        },
        {
          'name': 'EventDetails',
          'args': {'date': '2026-07-15', 'event_name': '公
司年会'},
          'id': 'call_0ba1d4cbd2b0462592e06fd6',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 404,
        'output_tokens': 118,
        'total_tokens': 522,
        'input_token_details': {'cache_read': 256},
        'output_token_details': {}
      }
    ),
    ToolMessage(
      content='请检查输入数据',
      name='ContactInfo',
      id='aa9bbb37-b9e7-4a4b-9c35-571c8e7ef0c7',
      tool_call_id='call_c4c0c568ff514df58078a41b'
    ),
    ToolMessage(

```

```

        content='请检查输入数据',
        name='EventDetails',
        id='bbf93ddb-7a2f-46f3-8872-2f6915269d58',
        tool_call_id='call_0ba1d4cbd2b0462592e06fd6'
    ),
    AIMessage(
        content='',
        additional_kwargs={'refusal': None},
        response_metadata={
            'token_usage': {
                'completion_tokens': 118,
                'prompt_tokens': 551,
                'total_tokens': 669,
                'completion_tokens_details': None,
                'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 256}
            },
            'model_provider': 'openai',
            'model_name': 'deepseek-v4-flash-0731',
            'system_fingerprint': None,
            'id': 'chatcpl-56095e45-d854-936c-bf5e-fbcdca36d789',
            'finish_reason': 'tool_calls',
            'logprobs': None
        },
        id='lc_run--019ff646-05bd-7ef1-a4b8-98c3cf21af5f-0',
        tool_calls=[
            {
                'name': 'ContactInfo',
                'args': {'email': 'zhang3@atguigu.com', 'name':
'张三'},
                'id': 'call_bb5f0c1e366349ffa69ed982',
                'type': 'tool_call'
            },
            {
                'name': 'EventDetails',
                'args': {'date': '2026-07-15', 'event_name': '公
司年会'},
                'id': 'call_b54e1faf2c3043248a0fdc76',
                'type': 'tool_call'
            }
        ],
        invalid_tool_calls=[],
        usage_metadata={
            'input_tokens': 551,
            'output_tokens': 118,
            'total_tokens': 669,
            'input_token_details': {'cache_read': 256},
            'output_token_details': {}
        }
    ),
    ToolMessage(
        content='请检查输入数据',
        name='ContactInfo',
        id='d0d8a1b9-a0d0-4cb5-bcce-14145743d709',
        tool_call_id='call_bb5f0c1e366349ffa69ed982'
    ),
    ToolMessage(
        content='请检查输入数据',
        name='EventDetails',
        id='0c8aa974-9a3b-49d8-a8d2-48a02d954275',
        tool_call_id='call_b54e1faf2c3043248a0fdc76'
    ),

```

```

AIMessage(
    content='',
    additional_kwargs={'refusal': None},
    response_metadata={
        'token_usage': {
            'completion_tokens': 118,
            'prompt_tokens': 698,
            'total_tokens': 816,
            'completion_tokens_details': None,
            'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 512}
        },
        'model_provider': 'openai',
        'model_name': 'deepseek-v4-flash-0731',
        'system_fingerprint': None,
        'id': 'chatcmpl-6b8fbbbd-ed94-95eb-a419-2f799d7ae64b',
        'finish_reason': 'tool_calls',
        'logprobs': None
    },
    id='lc_run--019ff646-090f-77f1-8846-4d0a97188f2d-0',
    tool_calls=[
        {
            'name': 'ContactInfo',
            'args': {'email': 'zhang3@atguigu.com', 'name': '张三'},
            'id': 'call_b8e3e42bcc704af18b09903e',
            'type': 'tool_call'
        },
        {
            'name': 'EventDetails',
            'args': {'date': '2026-07-15', 'event_name': '公司年会'},
            'id': 'call_04c2267127fc45248e7a59d1',
            'type': 'tool_call'
        }
    ],
    invalid_tool_calls=[],
    usage_metadata={
        'input_tokens': 698,
        'output_tokens': 118,
        'total_tokens': 816,
        'input_token_details': {'cache_read': 512},
        'output_token_details': {}
    }
),
ToolMessage(
    content='请检查输入数据',
    name='ContactInfo',
    id='5e584269-8f12-480d-a3d8-35acc9b8c91b',
    tool_call_id='call_b8e3e42bcc704af18b09903e'
),
ToolMessage(
    content='请检查输入数据',
    name='EventDetails',
    id='44d62d17-c78b-4fdf-8975-a1666306fa03',
    tool_call_id='call_04c2267127fc45248e7a59d1'
),
AIMessage(
    content='',
    additional_kwargs={'refusal': None},
    response_metadata={
        'token_usage': {

```

```

        'completion_tokens': 118,
        'prompt_tokens': 845,
        'total_tokens': 963,
        'completion_tokens_details': None,
        'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 512}
    },
    'model_provider': 'openai',
    'model_name': 'deepseek-v4-flash-0731',
    'system_fingerprint': None,
    'id': 'chatcmpl-99cb3d82-5d83-966a-98c1-
1908752b5186',
    'finish_reason': 'tool_calls',
    'logprobs': None
},
id='lc_run--019ff646-0c94-76d2-8a9e-7f4ba37b48cc-0',
tool_calls=[
    {
        'name': 'EventDetails',
        'args': {'date': '2026-07-15', 'event_name': '公
司年会'},
        'id': 'call_66a7299eb42043848e4acfa0',
        'type': 'tool_call'
    },
    {
        'name': 'ContactInfo',
        'args': {'email': 'zhang3@atguigu.com', 'name':
'张三'},
        'id': 'call_27e12910614f4a678389cf95',
        'type': 'tool_call'
    }
],
invalid_tool_calls=[],
usage_metadata={
    'input_tokens': 845,
    'output_tokens': 118,
    'total_tokens': 963,
    'input_token_details': {'cache_read': 512},
    'output_token_details': {}
}
),
ToolMessage(
    content='请检查输入数据',
    name='EventDetails',
    id='6d9e6ec0-d62b-4ecf-adae-2d173efe2817',
    tool_call_id='call_66a7299eb42043848e4acfa0'
),
ToolMessage(
    content='请检查输入数据',
    name='ContactInfo',
    id='e5a8e6f6-04bd-4590-b78d-eb66fe50be29',
    tool_call_id='call_27e12910614f4a678389cf95'
),
AIMessage(
    content='',
    additional_kwargs={'refusal': None},
    response_metadata={
        'token_usage': {
            'completion_tokens': 118,
            'prompt_tokens': 992,
            'total_tokens': 1110,
            'completion_tokens_details': None,
            'prompt_tokens_details': {'audio_tokens': None,

```

```

'cached_tokens': 768}
    },
    'model_provider': 'openai',
    'model_name': 'deepseek-v4-flash-0731',
    'system_fingerprint': None,
    'id': 'chatcmpl-a59cd824-b38f-9280-bf7b-f174934dc281',
    'finish_reason': 'tool_calls',
    'logprobs': None
  },
  id='lc_run--019ff646-0fc6-7381-bf94-5d24ce3dd612-0',
  tool_calls=[
    {
      'name': 'EventDetails',
      'args': {'date': '2026-07-15', 'event_name': '公
司年会'},
      'id': 'call_34fab19cc5994aebbddf0d8f',
      'type': 'tool_call'
    },
    {
      'name': 'ContactInfo',
      'args': {'email': 'zhang3@atguigu.com', 'name':
'张三'},
      'id': 'call_1e4c607d79f24ca99fb7f727',
      'type': 'tool_call'
    }
  ],
  invalid_tool_calls=[],
  usage_metadata={
    'input_tokens': 992,
    'output_tokens': 118,
    'total_tokens': 1110,
    'input_token_details': {'cache_read': 768},
    'output_token_details': {}
  }
),
ToolMessage(
  content='请检查输入数据',
  name='EventDetails',
  id='dcbfea5e-4dc4-4310-91ae-f1c35b4b47cb',
  tool_call_id='call_34fab19cc5994aebbddf0d8f'
),
ToolMessage(
  content='请检查输入数据',
  name='ContactInfo',
  id='e276c498-7c11-4eb5-9a1a-08c27984bf75',
  tool_call_id='call_1e4c607d79f24ca99fb7f727'
),
AIMessage(
  content='',
  additional_kwargs={'refusal': None},
  response_metadata={
    'token_usage': {
      'completion_tokens': 65,
      'prompt_tokens': 1139,
      'total_tokens': 1204,
      'completion_tokens_details': None,
      'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 512}
    },
    'model_provider': 'openai',
    'model_name': 'deepseek-v4-flash-0731',
    'system_fingerprint': None,

```

```

        'id': 'chatcmpl-81c83be6-8feb-90b1-a24a-
94e812e41b5f',
        'finish_reason': 'tool_calls',
        'logprobs': None
    },
    id='lc_run--019ff646-135f-7c13-b6f6-0d5aa2a8df54-0',
    tool_calls=[
        {
            'name': 'ContactInfo',
            'args': {'email': 'zhang3@atguigu.com', 'name':
'张三'},
            'id': 'call_52e97821019c496b9e74ffc8',
            'type': 'tool_call'
        }
    ],
    invalid_tool_calls=[],
    usage_metadata={
        'input_tokens': 1139,
        'output_tokens': 65,
        'total_tokens': 1204,
        'input_token_details': {'cache_read': 512},
        'output_token_details': {}
    }
),
ToolMessage(
    content='提取完成!',
    name='ContactInfo',
    id='be18fc6b-907e-4ff8-be23-f3fbc7f430d8',
    tool_call_id='call_52e97821019c496b9e74ffc8'
),
    ],
    'structured_response': ContactInfo(name='张三',
email='zhang3@atguigu.com')
}

```

以上代码中注意如下几点：

1. ToolStrategy允许指定多个类型“Union[ContactInfo, EventDetails]”这种写法，但是最终只会转换成一种结构化类型输出。
2. 当ToolStrategy通过“Union[ContactInfo, EventDetails]”指定多个类型时，在内部调用生成结构化类型工具会报错。此时：
 - handle_errors=True（默认值）开始发挥作用，系统会生成一个ToolMessage，明确告诉LLM“Error: Model incorrectly returned multiple structured responses (ContactInfo,EventDetails) when only one is expected.”，大模型收到这个精准的反馈后，会重新进行推理，最终选择并输出一个最符合要求的Schema。
 - 如果handle_errors 设置为False，执行代码过程直接报错。
3. 当格式化输出有错误时，Agent内部会进行工具调用重试，直到符合要求格式化输出前，可能会进行多次重试。

情况2：设置为指定异常类型

默认情况下，LangChain会处理结构化输出处理时抛出的两类异常：

1. MultipleStructuredOutputsError

多结构化输出错误，当返回的工具调用请求数量大于1时，抛出该异常，默认情况下

LangChain会拦截异常并提醒模型重试。

2. StructuredOutputValidationError

输出结构化验证错误，当输出格式不符合结构化要求时，抛出上述异常。

默认情况下LangChain会拦截该异常并自动重试。

```
In [12]: from pydantic import BaseModel, Field
from typing import Union
from langchain.agents import create_agent
from langchain.agents.structured_output import
ToolStrategy, StructuredOutputValidationError, MultipleStructuredOutputsError
from rich import print as rprint

class ContactInfo(BaseModel):
    """个人联系信息"""
    name: str = Field(description="姓名")
    email: str = Field(description="电子邮箱")

class EventDetails(BaseModel):
    """活动详情"""
    event_name: str = Field(description="活动名称")
    date: str = Field(description="活动日期")

agent = create_agent(
    model=model_bailian,
    response_format=ToolStrategy(
        Union[ContactInfo, EventDetails],
        tool_message_content="提取完成！",
        handle_errors=(MultipleStructuredOutputsError, StructuredOutputValidationError)
        # handle_errors=(StructuredOutputValidationError)
    )
)

result = agent.invoke({
    "messages": [{
        "role": "user",
        "content": f"请提取以下文本中内容：姓名：张三，电子邮箱：zhang3@atguigu.com，活动名
        称：公司年会，活动日期：2026-07-15"
    }]
})

rprint(result)
```

```

{
  'messages': [
    HumanMessage(
      content='请提取以下文本中内容：姓名：张三，电子邮箱：
zhang3@atguigu.com，活动名称：公司年会，活动日期：
2026-07-15',
      additional_kwargs={},
      response_metadata={},
      id='19867dd3-9416-417a-8d50-34efeba1c5f9'
    ),
    AIMessage(
      content='',
      additional_kwargs={'refusal': None},
      response_metadata={
        'token_usage': {
          'completion_tokens': 118,
          'prompt_tokens': 404,
          'total_tokens': 522,
          'completion_tokens_details': None,
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 256}
        },
        'model_provider': 'openai',
        'model_name': 'deepseek-v4-flash-0731',
        'system_fingerprint': None,
        'id': 'chatcpl-c77025e5-ee17-9fad-a0b0-
c527c2820b12',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff64a-6a63-7f03-bed4-872e086b09cc-0',
      tool_calls=[
        {
          'name': 'ContactInfo',
          'args': {'email': 'zhang3@atguigu.com', 'name':
'张三'},
          'id': 'call_d1befaa0b21c4bf2948834e6',
          'type': 'tool_call'
        },
        {
          'name': 'EventDetails',
          'args': {'date': '2026-07-15', 'event_name': '公
司年会'},
          'id': 'call_0282ceb1c4714da1947b3830',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 404,
        'output_tokens': 118,
        'total_tokens': 522,
        'input_token_details': {'cache_read': 256},
        'output_token_details': {}
      }
    ),
    ToolMessage(
      content='Error: Model incorrectly returned multiple
structured responses (ContactInfo, EventDetails)
when only one is expected.\n Please fix your mistakes.',
      name='ContactInfo',
      id='e2c8f8b6-47a4-46c0-9210-4c24093c993f',
      tool_call_id='call_d1befaa0b21c4bf2948834e6'
    )
  ]
}

```

```

    ),
    ToolMessage(
        content='Error: Model incorrectly returned multiple
structured responses (ContactInfo, EventDetails)
when only one is expected.\n Please fix your mistakes.',
        name='EventDetails',
        id='22749292-494b-465b-aa89-1fffbde310134',
        tool_call_id='call_0282ceb1c4714da1947b3830'
    ),
    AIMessage(
        content='',
        additional_kwargs={'refusal': None},
        response_metadata={
            'token_usage': {
                'completion_tokens': 65,
                'prompt_tokens': 593,
                'total_tokens': 658,
                'completion_tokens_details': None,
                'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 512}
            },
            'model_provider': 'openai',
            'model_name': 'deepseek-v4-flash-0731',
            'system_fingerprint': None,
            'id': 'chatcmpl-01c85472-06a2-98b3-8196-
07992ba26c54',
            'finish_reason': 'tool_calls',
            'logprobs': None
        },
        id='lc_run--019ff64a-6e95-7e33-a49a-7e721f789417-0',
        tool_calls=[
            {
                'name': 'ContactInfo',
                'args': {'email': 'zhang3@atguigu.com', 'name':
'张三'},
                'id': 'call_f21f33d31bc54a4fbcfaf8f1',
                'type': 'tool_call'
            }
        ],
        invalid_tool_calls=[],
        usage_metadata={
            'input_tokens': 593,
            'output_tokens': 65,
            'total_tokens': 658,
            'input_token_details': {'cache_read': 512},
            'output_token_details': {}
        }
    ),
    ToolMessage(
        content='提取完成!',
        name='ContactInfo',
        id='e4196950-fe4c-4c62-b187-9e52b74c3302',
        tool_call_id='call_f21f33d31bc54a4fbcfaf8f1'
    )
],
    'structured_response': ContactInfo(name='张三',
email='zhang3@atguigu.com')
}

```

情况3：设置为自定义错误处理函数

指定异常处理函数并返回字符串时，LangChain会在遇到异常时自动重试并将异常处理函数的返回值作为ToolMessage的内容。

我们也可以选择在异常处理函数中抛出异常。

```
In [13]: from pydantic import BaseModel, Field
from typing import Union
from langchain.agents import create_agent
from langchain.agents.structured_output import
ToolStrategy, StructuredOutputValidationError, MultipleStructuredOutputsError
from rich import print as rprint

# 自定义错误处理函数
def custom_error_handler(error: Exception) -> str:
    """自定义错误处理器"""
    error_str = str(error)

    print(f"捕获到错误类型：{type(error).__name__}")
    print(f"错误详情：{error_str}")

    if isinstance(error, StructuredOutputValidationError):
        return "数据格式有误，请检查字段是否符合要求。"
    elif isinstance(error, MultipleStructuredOutputsError):
        return "检测到多个响应，请选择最相关的一个进行返回。"
    else:
        return f"Error: {error_str}"

class ContactInfo(BaseModel):
    """个人联系信息"""
    name: str = Field(description="姓名")
    email: str = Field(description="电子邮箱")

class EventDetails(BaseModel):
    """活动详情"""
    event_name: str = Field(description="活动名称")
    date: str = Field(description="活动日期")

agent = create_agent(
    model=model_bailian,
    response_format=ToolStrategy(
        Union[ContactInfo, EventDetails],
        tool_message_content="提取完成！",
        handle_errors=custom_error_handler
    )
)
result = agent.invoke({
    "messages": [{
        "role": "user",
        "content": f"请提取以下文本中内容：姓名：张三，电子邮箱：zhang3@atguigu.com，活动名称：公司年会，活动日期：2026-07-15"
    }]
})

rprint(result)
```

捕获到错误类型：MultipleStructuredOutputsError
错误详情：Model incorrectly returned multiple structured responses (ContactInfo, EventDetails) when only one is expected.

```

{
  'messages': [
    HumanMessage(
      content='请提取以下文本中内容：姓名：张三，电子邮箱：
zhang3@atguigu.com，活动名称：公司年会，活动日期：
2026-07-15',
      additional_kwargs={},
      response_metadata={},
      id='d5807607-776b-4ec3-ba5c-2c9330b73707'
    ),
    AIMessage(
      content='',
      additional_kwargs={'refusal': None},
      response_metadata={
        'token_usage': {
          'completion_tokens': 118,
          'prompt_tokens': 404,
          'total_tokens': 522,
          'completion_tokens_details': None,
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 256}
        },
        'model_provider': 'openai',
        'model_name': 'deepseek-v4-flash-0731',
        'system_fingerprint': None,
        'id': 'chatcpl-d5e9f3bd-8790-931c-b44a-
814c6387ef68',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff64d-4a0c-76c2-adfa-a6d623070f14-0',
      tool_calls=[
        {
          'name': 'ContactInfo',
          'args': {'email': 'zhang3@atguigu.com', 'name':
'张三'},
          'id': 'call_9df1d40e28e24b4c8ccb7e8c',
          'type': 'tool_call'
        },
        {
          'name': 'EventDetails',
          'args': {'date': '2026-07-15', 'event_name': '公
司年会'},
          'id': 'call_8d309e7d64234b62b253d087',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 404,
        'output_tokens': 118,
        'total_tokens': 522,
        'input_token_details': {'cache_read': 256},
        'output_token_details': {}
      }
    ),
    ToolMessage(
      content='检测到多个响应，请选择最相关的一个进行返回。',
      name='ContactInfo',
      id='a0cf02da-bfea-4f1e-acb3-253cd1cc66cc',
      tool_call_id='call_9df1d40e28e24b4c8ccb7e8c'
    ),
    ToolMessage(

```

```

        content='检测到多个响应，请选择最相关的一个进行返回。',
        name='EventDetails',
        id='dfa78d9a-691c-41be-b5c6-56d6b28ed72b',
        tool_call_id='call_8d309e7d64234b62b253d087'
    ),
    AIMessage(
        content='',
        additional_kwargs={'refusal': None},
        response_metadata={
            'token_usage': {
                'completion_tokens': 65,
                'prompt_tokens': 567,
                'total_tokens': 632,
                'completion_tokens_details': None,
                'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 512}
            },
            'model_provider': 'openai',
            'model_name': 'deepseek-v4-flash-0731',
            'system_fingerprint': None,
            'id': 'chatcmpl-8abb13c5-f617-91b8-80fd-d03fbd9e0461',
            'finish_reason': 'tool_calls',
            'logprobs': None
        },
        id='lc_run--019ff64d-4de1-71f3-9eea-a6970a19e0f6-0',
        tool_calls=[
            {
                'name': 'ContactInfo',
                'args': {'email': 'zhang3@atguigu.com', 'name':
'张三'},
                'id': 'call_73fcb112568e4dabb47b9de0',
                'type': 'tool_call'
            }
        ],
        invalid_tool_calls=[],
        usage_metadata={
            'input_tokens': 567,
            'output_tokens': 65,
            'total_tokens': 632,
            'input_token_details': {'cache_read': 512},
            'output_token_details': {}
        }
    ),
    ToolMessage(
        content='提取完成!',
        name='ContactInfo',
        id='3a12bd45-38cb-472b-ae1-1e7bd55e8fa8',
        tool_call_id='call_73fcb112568e4dabb47b9de0'
    )
],
    'structured_response': ContactInfo(name='张三',
email='zhang3@atguigu.com')
}

```

8.Agent的高级用法4：流式输出及模式

8.1 流式输出的说明

通过invoke 调用Agent时，内部可能经历多次调用，长时间看不到调用情况，用户体验不好，可以通过流式调用（渐进式显示输出）优化用户体验，实时显示 Agent 运行过程中的更新。特别是在处理LLM 延迟时尤其有效。

流式输出好处：

- 大型语言模型生成完整响应通常需要几秒钟时间，对于长输出可能达到10-20 秒，用户期望即时反馈，流式传输让等待过程更加可控。
- 相比非流式传输需要用户长时间等待完整响应，流式传输可以立即显示文字逐渐出现的效果，大幅降低用户的等待焦虑。

设置方式：

通过“agent.stream(stream_mode=指定模式)”来指定。具体模式有：values、updates（默认）、messages、custom、checkpoints、tasks、debug。

8.2 具体的输出模式

1) values输出模式

当stream_mode 设置为values模式时，每个步骤执行后，都会输出完整的状态信息，适用于每一步都要获取完整状态、状态持久化场景。

举例：

```
In [15]: from langchain.chat_models import init_chat_model
from dotenv import load_dotenv
import os

load_dotenv(override=True)

# 1. 模型初始化
model = init_chat_model(
    model="deepseek-v4-flash-0731",
    model_provider="openai",
    api_key=os.getenv("DASHSCOPE_API_KEY"),
    base_url=os.getenv("DASHSCOPE_BASE_URL"),
    extra_body={
        "thinking":{
            "type":"disabled"
        }
    }
)

In [16]: from langchain.agents import create_agent
from langchain.tools import tool
from typing import Dict, Any
from rich import print as rprint
@tool
def query_customer_data(customer_id: str) -> Dict[str, Any]:
    """
    查询客户基本信息

    Args:
        customer_id: 客户ID，用于唯一标识客户

    Returns:
```

```

        包含客户基本信息的字典，如姓名、等级、加入日期等
        """
        # 模拟数据库查询
        return {"name": "张三", "level": "VIP", "join_date": "2023-01-15"}

@tool
def check_order_history(customer_id: str) -> Dict[str, Any]:
    """
    查询客户订单历史

    Args:
        customer_id: 客户ID，用于唯一标识客户

    Returns:
        包含客户订单历史的字典，如总订单数、总花费等
        """
    return {"total_orders": 15, "total_spent": 25800.00}

@tool
def get_current_promotions() -> Dict[str, Any]:
    """
    获取当前可用促销活动

    Returns:
        包含当前可用促销活动的字典，如活动名称、有效日期等
        """
    return {
        "promotions": ["老用户优惠", "会员专属折扣"],
        "valid_until": "2027-01-31"
    }

# 创建客户服务Agent
customer_service_agent = create_agent(
    model=model,
    tools=[query_customer_data, check_order_history, get_current_promotions]
)

for chunk in customer_service_agent.stream(
    {"messages": [{"role": "user", "content": "查询客户ID为 CUST123456 的个人信息、历史订单和可用优惠"}]},
    stream_mode="values"
):
    rprint(chunk)
    print("-" * 50)

{
    'messages': [
        HumanMessage(
            content='查询客户ID为 CUST123456 的个人信息、历史订单和可用优惠',
            additional_kwargs={},
            response_metadata={},
            id='de32cad4-ac44-4dbf-8e15-57e80e158835'
        )
    ]
}

```

```

{
  'messages': [
    HumanMessage(
      content='查询客户ID为 CUST123456 的个人信息、历史订单和可用优
惠',
      additional_kwargs={},
      response_metadata={},
      id='de32cad4-ac44-4dbf-8e15-57e80e158835'
    ),
    AIMessage(
      content='我来为您查询客户 CUST123456 的个人信息、历史订单和可用
优惠信息。\\n\\n',
      additional_kwargs={'refusal': None},
      response_metadata={
        'token_usage': {
          'completion_tokens': 120,
          'prompt_tokens': 467,
          'total_tokens': 587,
          'completion_tokens_details': None,
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0}
        },
        'model_provider': 'openai',
        'model_name': 'deepseek-v4-flash-0731',
        'system_fingerprint': None,
        'id': 'chatcpl-7142666a-bdbe-9637-9478-
c71497d9ae29',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff653-b620-7a60-bfb6-0b8b9e9b668c-0',
      tool_calls=[
        {
          'name': 'query_customer_data',
          'args': {'customer_id': 'CUST123456'},
          'id': 'call_beaf30b80b904596949fcbde',
          'type': 'tool_call'
        },
        {
          'name': 'check_order_history',
          'args': {'customer_id': 'CUST123456'},
          'id': 'call_9a9b6e6a35ed459c99b0c13c',
          'type': 'tool_call'
        },
        {
          'name': 'get_current_promotions',
          'args': {},
          'id': 'call_d03302c7cbd04aaba3404e43',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 467,
        'output_tokens': 120,
        'total_tokens': 587,
        'input_token_details': {'cache_read': 0},
        'output_token_details': {}
      }
    )
  ]
}

```

```

{
  'messages': [
    HumanMessage(
      content='查询客户ID为 CUST123456 的个人信息、历史订单和可用优
惠',
      additional_kwargs={},
      response_metadata={},
      id='de32cad4-ac44-4dbf-8e15-57e80e158835'
    ),
    AIMessage(
      content='我来为您查询客户 CUST123456 的个人信息、历史订单和可用
优惠信息。\\n\\n',
      additional_kwargs={'refusal': None},
      response_metadata={
        'token_usage': {
          'completion_tokens': 120,
          'prompt_tokens': 467,
          'total_tokens': 587,
          'completion_tokens_details': None,
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0}
        },
        'model_provider': 'openai',
        'model_name': 'deepseek-v4-flash-0731',
        'system_fingerprint': None,
        'id': 'chatcpl-7142666a-bdbe-9637-9478-
c71497d9ae29',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff653-b620-7a60-bfb6-0b8b9e9b668c-0',
      tool_calls=[
        {
          'name': 'query_customer_data',
          'args': {'customer_id': 'CUST123456'},
          'id': 'call_beaf30b80b904596949fcbde',
          'type': 'tool_call'
        },
        {
          'name': 'check_order_history',
          'args': {'customer_id': 'CUST123456'},
          'id': 'call_9a9b6e6a35ed459c99b0c13c',
          'type': 'tool_call'
        },
        {
          'name': 'get_current_promotions',
          'args': {},
          'id': 'call_d03302c7cbd04aaba3404e43',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 467,
        'output_tokens': 120,
        'total_tokens': 587,
        'input_token_details': {'cache_read': 0},
        'output_token_details': {}
      }
    ),
    ToolMessage(
      content='{"name": "张三", "level": "VIP", "join_date":
"2023-01-15"}',

```

```

        name='query_customer_data',
        id='d01fd8-9385-4b71-b969-9437497d22d6',
        tool_call_id='call_beaf30b80b904596949fcbde'
    ),
    ToolMessage(
        content='{"total_orders": 15, "total_spent": 25800.0}',
        name='check_order_history',
        id='15301d67-7413-46cf-8560-f7e3e9c6fb06',
        tool_call_id='call_9a9b6e6a35ed459c99b0c13c'
    ),
    ToolMessage(
        content='{"promotions": ["老用户优惠", "会员专属折扣"],
"valid_until": "2027-01-31"}',
        name='get_current_promotions',
        id='c73dbc4e-3561-49aa-93cd-19074df7c225',
        tool_call_id='call_d03302c7cbd04aaba3404e43'
    )
]
}

```

```

{
  'messages': [
    HumanMessage(
      content='查询客户ID为 CUST123456 的个人信息、历史订单和可用优惠',
      additional_kwargs={},
      response_metadata={},
      id='de32cad4-ac44-4dbf-8e15-57e80e158835'
    ),
    AIMessage(
      content='我来为您查询客户 CUST123456 的个人信息、历史订单和可用优惠信息。\\n\\n',
      additional_kwargs={'refusal': None},
      response_metadata={
        'token_usage': {
          'completion_tokens': 120,
          'prompt_tokens': 467,
          'total_tokens': 587,
          'completion_tokens_details': None,
          'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0}
        },
        'model_provider': 'openai',
        'model_name': 'deepseek-v4-flash-0731',
        'system_fingerprint': None,
        'id': 'chatcmpl-7142666a-bdbe-9637-9478-c71497d9ae29',
        'finish_reason': 'tool_calls',
        'logprobs': None
      },
      id='lc_run--019ff653-b620-7a60-bfb6-0b8b9e9b668c-0',
      tool_calls=[
        {
          'name': 'query_customer_data',
          'args': {'customer_id': 'CUST123456'},
          'id': 'call_beaf30b80b904596949fcbde',
          'type': 'tool_call'
        },
        {
          'name': 'check_order_history',
          'args': {'customer_id': 'CUST123456'},
          'id': 'call_9a9b6e6a35ed459c99b0c13c',
          'type': 'tool_call'
        },
        {
          'name': 'get_current_promotions',
          'args': {},
          'id': 'call_d03302c7cbd04aaba3404e43',
          'type': 'tool_call'
        }
      ],
      invalid_tool_calls=[],
      usage_metadata={
        'input_tokens': 467,
        'output_tokens': 120,
        'total_tokens': 587,
        'input_token_details': {'cache_read': 0},
        'output_token_details': {}
      }
    ),
    ToolMessage(
      content='{"name": "张三", "level": "VIP", "join_date": "2023-01-15"}',

```

```

        name='query_customer_data',
        id='d01fdfd8-9385-4b71-b969-9437497d22d6',
        tool_call_id='call_beaf30b80b904596949fcbde'
    ),
    ToolMessage(
        content='{"total_orders": 15, "total_spent": 25800.0}',
        name='check_order_history',
        id='15301d67-7413-46cf-8560-f7e3e9c6fb06',
        tool_call_id='call_9a9b6e6a35ed459c99b0c13c'
    ),
    ToolMessage(
        content='{"promotions": ["老用户优惠", "会员专属折扣"],
"valid_until": "2027-01-31"}',
        name='get_current_promotions',
        id='c73dbc4e-3561-49aa-93cd-19074df7c225',
        tool_call_id='call_d03302c7cbd04aaba3404e43'
    ),
    AIMessage(
        content='我已经为您查询到了客户 CUST123456 的全部信息，以下是详细汇总：\n\n## 📄 客户个人信息\n| 项目 | 详情 |\n|-----|-----|\n| **姓名** | 张三 |\n| **客户等级** | VIP |\n| **加入日期** | 2023-01-15 |\n\n## 🛒 订单历史\n| 项目 | 详情 |\n|-----|-----|\n| **总订单数** | 15 笔 |\n| **总消费金额** | ¥25,800.00 |\n\n## 🎁 可用优惠\n| 项目 | 详情 |\n|-----|-----|\n| **可用优惠活动** | 老用户优惠、会员专属折扣 |\n| **有效期至** | 2027-01-31 |\n\n---\n\n## **总结：**\n张三先生是VIP等级客户，自2023年1月加入以来共消费15笔、总计¥25,800，目前可享受"老用户优惠"和"会员专属折扣"两项优惠活动，有效期至2027年1月31日。如需进一步了解某个特定优惠的具体内容，或查询其他信息，请随时告诉我！',
        additional_kwargs={'refusal': None},
        response_metadata={
            'token_usage': {
                'completion_tokens': 240,
                'prompt_tokens': 684,
                'total_tokens': 924,
                'completion_tokens_details': None,
                'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0}
            },
            'model_provider': 'openai',
            'model_name': 'deepseek-v4-flash-0731',
            'system_fingerprint': None,
            'id': 'chatcmpl-2b23d623-a582-9c8f-9ed7-d733380aa3b5',
            'finish_reason': 'stop',
            'logprobs': None
        },
        id='lc_run--019ff653-bc55-7150-a204-2a6e77862535-0',
        tool_calls=[],
        invalid_tool_calls=[],
        usage_metadata={
            'input_tokens': 684,
            'output_tokens': 240,
            'total_tokens': 924,
            'input_token_details': {'cache_read': 0},
            'output_token_details': {}
        }
    )

```

```
} ]
```

代码运行后示意结果如下：

```
{'messages': [HumanMessage(...)]}
{'messages': [HumanMessage(...), AIMessage(...)]}
{'messages': [HumanMessage(...), AIMessage(...), ToolMessage(...)]}
{'messages': [HumanMessage(...), AIMessage(...), ToolMessage(...), AIMessage(...)]}
{'messages': [HumanMessage(...), AIMessage(...), ToolMessage(...), AIMessage(...), ToolMessage(...)]}
{'messages': [HumanMessage(...), AIMessage(...), ToolMessage(...), AIMessage(...), ToolMessage(...), AIMessage(...)]}
{'messages': [HumanMessage(...), AIMessage(...), ToolMessage(...), AIMessage(...), ToolMessage(...), AIMessage(...), ToolMessage(...)]}
{'messages': [HumanMessage(...), AIMessage(...), ToolMessage(...), AIMessage(...), ToolMessage(...), AIMessage(...), ToolMessage(...), AIMessage(...)]}
```

3) updates输出模式

这种模式就是默认模式。该模式中，每个步骤执行后，只增量更新状态中发生变化的内容，用于监控Agent 执行进度，例如观察Agent 决定调用工具、工具执行结果等步骤。

代码如下：

```
In [17]: from langchain.agents import create_agent
from langchain.tools import tool
from typing import Dict, Any
from rich import print as rprint
@tool
def query_customer_data(customer_id: str) -> Dict[str, Any]:
    """
    查询客户基本信息

    Args:
        customer_id: 客户ID，用于唯一标识客户

    Returns:
        包含客户基本信息的字典，如姓名、等级、加入日期等
    """
    # 模拟数据库查询
    return {"name": "张三", "level": "VIP", "join_date": "2023-01-15"}

@tool
def check_order_history(customer_id: str) -> Dict[str, Any]:
    """
    查询客户订单历史

    Args:
        customer_id: 客户ID，用于唯一标识客户

    Returns:
        包含客户订单历史的字典，如总订单数、总花费等
    """
    return {"total_orders": 15, "total_spent": 25800.00}

@tool
def get_current_promotions() -> Dict[str, Any]:
    """
    获取当前可用促销活动

    Returns:
        包含当前可用促销活动的字典，如活动名称、有效日期等
    """
```

```

"""
return {
    "promotions": ["老用户优惠", "会员专属折扣"],
    "valid_until": "2027-01-31"
}

# 创建客户服务Agent
customer_service_agent = create_agent(
    model=model,
    tools=[query_customer_data, check_order_history, get_current_promotions]
)

for chunk in customer_service_agent.stream(
    {"messages": [{"role": "user", "content": "查询客户ID为 CUST123456 的个人信息、历史订单和可用优惠"}]},
    stream_mode="updates"
):
    rprint(chunk)
    print("-" * 50)

```

```

{
  'model': {
    'messages': [
      AIMessage(
        content='我来为您查询客户 CUST123456
的相关信息。我将同时查询客户基本信息、订单历史和当前可用优惠。\\n\\n',
        additional_kwargs={'refusal': None},
        response_metadata={
          'token_usage': {
            'completion_tokens': 125,
            'prompt_tokens': 467,
            'total_tokens': 592,
            'completion_tokens_details': None,
            'prompt_tokens_details': {'audio_tokens':
None, 'cached_tokens': 0}
          },
          'model_provider': 'openai',
          'model_name': 'deepseek-v4-flash-0731',
          'system_fingerprint': None,
          'id': 'chatcmpl-dd0f8f7c-4520-9504-b46b-
9ac39ab94eff',
          'finish_reason': 'tool_calls',
          'logprobs': None
        },
        id='lc_run--019ff655-eb63-70f1-ba2d-47bd95962cdb-0',
        tool_calls=[
          {
            'name': 'query_customer_data',
            'args': {'customer_id': 'CUST123456'},
            'id': 'call_c3610bfa09f94dda961752db',
            'type': 'tool_call'
          },
          {
            'name': 'check_order_history',
            'args': {'customer_id': 'CUST123456'},
            'id': 'call_8a391da0f8744ef3a2e28984',
            'type': 'tool_call'
          },
          {
            'name': 'get_current_promotions',
            'args': {},
            'id': 'call_75db0b3b4b9440d1af7ace1f',
            'type': 'tool_call'
          }
        ],
        invalid_tool_calls=[],
        usage_metadata={
          'input_tokens': 467,
          'output_tokens': 125,
          'total_tokens': 592,
          'input_token_details': {'cache_read': 0},
          'output_token_details': {}
        }
      )
    ]
  }
}

```

```

{
  'tools': {
    'messages': [
      ToolMessage(
        content='{"name": "张三", "level": "VIP",
"join_date": "2023-01-15"}',
        name='query_customer_data',
        id='73cb7caf-fbe4-4fb5-a465-32e016cc69f8',
        tool_call_id='call_c3610bfa09f94dda961752db'
      )
    ]
  }
}
-----
{
  'tools': {
    'messages': [
      ToolMessage(
        content='{"total_orders": 15, "total_spent":
25800.0}',
        name='check_order_history',
        id='59726f5c-6f0e-477b-85a2-28982347738b',
        tool_call_id='call_8a391da0f8744ef3a2e28984'
      )
    ]
  }
}
-----
{
  'tools': {
    'messages': [
      ToolMessage(
        content='{"promotions": ["老用户优惠", "会员专属折扣"],
"valid_until": "2027-01-31"}',
        name='get_current_promotions',
        id='6463e3cc-c8a9-4abf-95df-cde711b9e8b2',
        tool_call_id='call_75db0b3b4b9440d1af7ace1f'
      )
    ]
  }
}
-----

```

```

{
  'model': {
    'messages': [
      AIMessage(
        content='我已成功查询到客户 CUST123456 的相关信息，以下是
详细汇总：\n\n---\n\n### 👤
客户个人信息\n| 项目 | 信息 |\n|-----|-----|\n| **姓名** | 张三 |\n|
**会员等级** | VIP |\n| **加入日期** |
2023-01-15 |\n\n---\n\n### 📦 订单历史\n| 项目 | 信息 |\n|-----|-----
-|\n| **总订单数** | 15 笔 |\n|
**累计消费金额** | ¥25,800.00 |\n\n---\n\n### 🎁 当前可用优惠\n| 优惠活动
| 有效期 |\n|-----|-----|\n|
老用户优惠 | 至 2027-01-31 |\n| 会员专属折扣 | 至 2027-01-31 |\n\n---
\n\n**总结：** 客户张三是一位 **VIP 会员**，自
2023 年 1 月加入以来累计下单 15 笔，总消费
¥25,800。目前可享受两项优惠活动（老用户优惠和会员专属折扣），两项优惠均有效至
2027 年 1 月 31
日。 \n\n如需更多信息或进行其他操作，请随时告知！',
        additional_kwargs={'refusal': None},
        response_metadata={
          'token_usage': {
            'completion_tokens': 255,
            'prompt_tokens': 689,
            'total_tokens': 944,
            'completion_tokens_details': None,
            'prompt_tokens_details': {'audio_tokens':
None, 'cached_tokens': 0}
          },
          'model_provider': 'openai',
          'model_name': 'deepseek-v4-flash-0731',
          'system_fingerprint': None,
          'id': 'chatcmpl-6d70e3ad-b6bb-93d4-9624-
13ca18b3219a',
          'finish_reason': 'stop',
          'logprobs': None
        },
        id='lc_run--019ff655-f2ce-7f61-bb84-1fcee0f68625-0',
        tool_calls=[],
        invalid_tool_calls=[],
        usage_metadata={
          'input_tokens': 689,
          'output_tokens': 255,
          'total_tokens': 944,
          'input_token_details': {'cache_read': 0},
          'output_token_details': {}
        }
      )
    ]
  }
}

```

运行结果如下：

```
{
  "model": {
    "messages": [
      {
        "content": "我来帮您查询客户CUST123456的完整信息和当前可用优惠。让我先获取客户的基本信息。",
        "additional_kwargs": {
          "refusal": None
        },
        "response": {
          "tools": {
            "messages": [
              {
                "content": {
                  "name": "张三",
                  "level": "VIP",
                  "join_date": "2023-01-15"
                },
                "name": "query_customer_data",
                "id": "61238935-f850-4838"
              }
            ]
          }
        }
      },
      {
        "content": "现在让我查询该客户的订单历史。",
        "additional_kwargs": {
          "refusal": None
        },
        "response_metadata": {
          "token_usage": {
            "completion_tokens": 15,
            "prompt_tokens": 25800,
            "total_tokens": 25815
          }
        }
      }
    ]
  },
  "tools": {
    "messages": [
      {
        "content": {
          "total_orders": 15,
          "total_spent": 25800.0
        },
        "name": "check_order_history",
        "id": "41aca09f-d73c-4d71-8f9f-8e9ce28b2"
      }
    ]
  }
}

{
  "model": {
    "messages": [
      {
        "content": "接下来让我获取当前可用的促销活动。",
        "additional_kwargs": {
          "refusal": None
        },
        "response_metadata": {
          "token_usage": {
            "completion_tokens": 12,
            "prompt_tokens": 124,
            "total_tokens": 136
          }
        }
      },
      {
        "content": {
          "promotions": [
            "老客户优惠",
            "会员专属折扣"
          ],
          "valid_until": "2027-01-31"
        },
        "name": "get_current_promotions",
        "id": "124"
      }
    ]
  },
  "tools": {
    "messages": [
      {
        "content": "根据查询结果，以下是客户CUST123456的完整信息和可用优惠：\n\n## 客户基本信息\n- **客户ID**：CUST123456\n- **姓名**：张三\n- **公",
        "additional_kwargs": {
          "refusal": None
        },
        "response": {
          "tools": {
            "messages": [
              {
                "content": {
                  "name": "张三",
                  "level": "VIP",
                  "join_date": "2023-01-15"
                },
                "name": "query_customer_data",
                "id": "61238935-f850-4838"
              }
            ]
          }
        }
      },
      {
        "content": "现在让我查询该客户的订单历史。",
        "additional_kwargs": {
          "refusal": None
        },
        "response_metadata": {
          "token_usage": {
            "completion_tokens": 15,
            "prompt_tokens": 25800,
            "total_tokens": 25815
          }
        }
      }
    ]
  },
  "tools": {
    "messages": [
      {
        "content": {
          "total_orders": 15,
          "total_spent": 25800.0
        },
        "name": "check_order_history",
        "id": "41aca09f-d73c-4d71-8f9f-8e9ce28b2"
      }
    ]
  }
}
```

3) messages输出模式

该模式中会输出流式返回的Token以及相关的元数据（如：来自哪个节点），可以用在实现类似ChatGPT 的打字机效果场景，为聊天机器人等交互式应用提供最佳的实时体验。

代码如下：

```
In [19]: from langchain.agents import create_agent
from langchain.tools import tool
from typing import Dict, Any
from rich import print as rprint
@tool
def query_customer_data(customer_id: str) -> Dict[str, Any]:
    """
    查询客户基本信息

    Args:
        customer_id: 客户ID，用于唯一标识客户

    Returns:
        包含客户基本信息的字典，如姓名、等级、加入日期等
    """
    # 模拟数据库查询
    return {"name": "张三", "level": "VIP", "join_date": "2023-01-15"}

@tool
def check_order_history(customer_id: str) -> Dict[str, Any]:
    """
    查询客户订单历史

    Args:
        customer_id: 客户ID，用于唯一标识客户

    Returns:
        包含客户订单历史的字典，如总订单数、总花费等
    """
    return {"total_orders": 15, "total_spent": 25800.00}

@tool
def get_current_promotions() -> Dict[str, Any]:
    """
    获取当前可用促销活动

    Returns:
        包含当前可用促销活动的字典，如活动名称、有效日期等
    """
    return {
        "promotions": ["老客户优惠", "会员专属折扣"],
```

```

    "valid_until": "2027-01-31"
}

# 创建客户服务Agent
customer_service_agent = create_agent(
    model=model,
    tools=[query_customer_data, check_order_history, get_current_promotions]
)

for chunk in customer_service_agent.stream(
    {"messages": [{"role": "user", "content": "查询客户ID为 CUST123456 的个人信息、历史订单和可用优惠"}]},
    stream_mode="messages"
):
    # rprint(chunk)
    # print("-" * 50)
    print(chunk[0].content, end="", flush=True)

```

我为您查询客户 CUST123456 的个人信息、历史订单和可用优惠信息。让我同时发起这些查询。

```

{"name": "张三", "level": "VIP", "join_date": "2023-01-15"}{"total_orders": 15, "total_spent": 25800.0}{"promotions": ["老用户优惠", "会员专属折扣"], "valid_until": "2027-01-31"}

```

我已成功查询到客户 CUST123456 的相关信息，以下是详细结果：

📄 客户个人信息

项目	详情
姓名	张三
会员等级	VIP
加入日期	2023年1月15日

🛒 历史订单信息

项目	详情
总订单数	15 笔
累计消费	¥25,800.00

🎁 可用优惠活动

项目	详情
可用优惠	老用户优惠、会员专属折扣
有效期至	2027年1月31日

总结： 客户张三是VIP会员，自2023年加入以来共完成15笔订单，累计消费25,800元。目前可享受"老用户优惠"和"会员专属折扣"两项优惠，有效期至2027年1月底。

如需进一步查询或操作，请随时告诉我！

4) tasks输出模式

该模式会输出当前task任务开始和结束的时间，包含任务的结果和错误信息，该模式用于监控任务的生命周期。

代码如下：

```

In [20]: from langchain.agents import create_agent
        from langchain.tools import tool

```

```

from typing import Dict, Any
from rich import print as rprint
@tool
def query_customer_data(customer_id: str) -> Dict[str, Any]:
    """
    查询客户基本信息

    Args:
        customer_id: 客户ID，用于唯一标识客户

    Returns:
        包含客户基本信息的字典，如姓名、等级、加入日期等
    """
    # 模拟数据库查询
    return {"name": "张三", "level": "VIP", "join_date": "2023-01-15"}

@tool
def check_order_history(customer_id: str) -> Dict[str, Any]:
    """
    查询客户订单历史

    Args:
        customer_id: 客户ID，用于唯一标识客户

    Returns:
        包含客户订单历史的字典，如总订单数、总花费等
    """
    return {"total_orders": 15, "total_spent": 25800.00}

@tool
def get_current_promotions() -> Dict[str, Any]:
    """
    获取当前可用促销活动

    Returns:
        包含当前可用促销活动的字典，如活动名称、有效日期等
    """
    return {
        "promotions": ["老用户优惠", "会员专属折扣"],
        "valid_until": "2027-01-31"
    }

# 创建客户服务Agent
customer_service_agent = create_agent(
    model=model,
    tools=[query_customer_data, check_order_history, get_current_promotions]
)

for chunk in customer_service_agent.stream(
    {"messages": [{"role": "user", "content": "查询客户ID为 CUST123456 的个人信息、历史订单和可用优惠"}]},
    stream_mode="tasks"
):
    rprint(chunk)
    print("-" * 50)

```

```

{
  'id': '5abf172f-d530-f4cc-6ae3-5b9c51788073',
  'name': 'model',
  'input': {
    'messages': [
      HumanMessage(
        content='查询客户ID为 CUST123456 的个人信息、历史订单和可用优惠',
        additional_kwargs={},
        response_metadata={},
        id='cc503bf0-acae-41ef-bf20-cc152b9296fd'
      )
    ],
    'triggers': ('branch:to:model',)
  }
}

```

```

{
  'id': '5abf172f-d530-f4cc-6ae3-5b9c51788073',
  'name': 'model',
  'error': None,
  'result': {
    'messages': [
      AIMessage(
        content='我来帮您查询客户 CUST123456
的个人信息、历史订单和可用优惠。这三个查询是独立的，我将同时进行查询',
        additional_kwargs={'refusal': None},
        response_metadata={
          'token_usage': {
            'completion_tokens': 130,
            'prompt_tokens': 467,
            'total_tokens': 597,
            'completion_tokens_details': None,
            'prompt_tokens_details': {'audio_tokens':
None, 'cached_tokens': 0}
          },
          'model_provider': 'openai',
          'model_name': 'deepseek-v4-flash-0731',
          'system_fingerprint': None,
          'id': 'chatcmpl-050cdd80-6a88-90d7-a003-
e9bbef0dd06c',
          'finish_reason': 'tool_calls',
          'logprobs': None
        },
        id='lc_run--019ff659-3d22-75c3-a031-b10e0265a688-0',
        tool_calls=[
          {
            'name': 'query_customer_data',
            'args': {'customer_id': 'CUST123456'},
            'id': 'call_4ba175f4d1be470099640739',
            'type': 'tool_call'
          },
          {
            'name': 'check_order_history',
            'args': {'customer_id': 'CUST123456'},
            'id': 'call_0fc49e45b1fd4432b268d94a',
            'type': 'tool_call'
          },
          {
            'name': 'get_current_promotions',
            'args': {},
            'id': 'call_77a08215fce9428d8bf96a5f',
            'type': 'tool_call'
          }
        ],
        invalid_tool_calls=[],
        usage_metadata={
          'input_tokens': 467,
          'output_tokens': 130,
          'total_tokens': 597,
          'input_token_details': {'cache_read': 0},
          'output_token_details': {}
        }
      )
    ],
    'interrupts': []
  }
}

```

```

{
  'id': 'bf5bf44a-fd3a-5e93-e6a3-39071c79927c',
  'name': 'tools',
  'input': {
    '__type': 'tool_call_with_context',
    'tool_call': {
      'name': 'query_customer_data',
      'args': {'customer_id': 'CUST123456'},
      'id': 'call_4ba175f4d1be470099640739',
      'type': 'tool_call'
    },
    'state': {
      'messages': [
        HumanMessage(
          content='查询客户ID为 CUST123456 的个人信息、历史订单
和可用优惠',
          additional_kwargs={},
          response_metadata={},
          id='cc503bf0-acae-41ef-bf20-cc152b9296fd'
        ),
        AIMessage(
          content='我来帮您查询客户 CUST123456
的个人信息、历史订单和可用优惠。这三个查询是独立的，我将同时进行查询',
          additional_kwargs={'refusal': None},
          response_metadata={
            'token_usage': {
              'completion_tokens': 130,
              'prompt_tokens': 467,
              'total_tokens': 597,
              'completion_tokens_details': None,
              'prompt_tokens_details':
{'audio_tokens': None, 'cached_tokens': 0}
            },
            'model_provider': 'openai',
            'model_name': 'deepseek-v4-flash-0731',
            'system_fingerprint': None,
            'id': 'chatcpl-050cdd80-6a88-90d7-a003-
e9bbef0dd06c',
            'finish_reason': 'tool_calls',
            'logprobs': None
          },
          id='lc_run--019ff659-3d22-75c3-a031-
b10e0265a688-0',
          tool_calls=[
            {
              'name': 'query_customer_data',
              'args': {'customer_id': 'CUST123456'},
              'id': 'call_4ba175f4d1be470099640739',
              'type': 'tool_call'
            },
            {
              'name': 'check_order_history',
              'args': {'customer_id': 'CUST123456'},
              'id': 'call_0fc49e45b1fd4432b268d94a',
              'type': 'tool_call'
            },
            {
              'name': 'get_current_promotions',
              'args': {},
              'id': 'call_77a08215fce9428d8bf96a5f',
              'type': 'tool_call'
            }
          ]
        )
      ]
    }
  }
}

```

```
invalid_tool_calls=[],
usage_metadata={
  'input_tokens': 467,
  'output_tokens': 130,
  'total_tokens': 597,
  'input_token_details': {'cache_read': 0},
  'output_token_details': {}
}
),
]
},
},
'triggers': ('__pregel_push',)
}
```

```

{
  'id': '95521535-b010-7ef2-2f70-f8337c5f470e',
  'name': 'tools',
  'input': {
    '__type': 'tool_call_with_context',
    'tool_call': {
      'name': 'check_order_history',
      'args': {'customer_id': 'CUST123456'},
      'id': 'call_0fc49e45b1fd4432b268d94a',
      'type': 'tool_call'
    },
  },
  'state': {
    'messages': [
      HumanMessage(
        content='查询客户ID为 CUST123456 的个人信息、历史订单
和可用优惠',
        additional_kwargs={},
        response_metadata={},
        id='cc503bf0-acae-41ef-bf20-cc152b9296fd'
      ),
      AIMessage(
        content='我来帮您查询客户 CUST123456
的个人信息、历史订单和可用优惠。这三个查询是独立的，我将同时进行查询',
        additional_kwargs={'refusal': None},
        response_metadata={
          'token_usage': {
            'completion_tokens': 130,
            'prompt_tokens': 467,
            'total_tokens': 597,
            'completion_tokens_details': None,
            'prompt_tokens_details':
{'audio_tokens': None, 'cached_tokens': 0}
          },
          'model_provider': 'openai',
          'model_name': 'deepseek-v4-flash-0731',
          'system_fingerprint': None,
          'id': 'chatcmpl-050cdd80-6a88-90d7-a003-
e9bbef0dd06c',
          'finish_reason': 'tool_calls',
          'logprobs': None
        },
        id='lc_run--019ff659-3d22-75c3-a031-
b10e0265a688-0',
        tool_calls=[
          {
            'name': 'query_customer_data',
            'args': {'customer_id': 'CUST123456'},
            'id': 'call_4ba175f4d1be470099640739',
            'type': 'tool_call'
          },
          {
            'name': 'check_order_history',
            'args': {'customer_id': 'CUST123456'},
            'id': 'call_0fc49e45b1fd4432b268d94a',
            'type': 'tool_call'
          },
          {
            'name': 'get_current_promotions',
            'args': {},
            'id': 'call_77a08215fce9428d8bf96a5f',
            'type': 'tool_call'
          }
        ]
      )
    ]
  }
}

```



```

{
  'id': '7ef8e151-3b6c-46b3-d2d6-cb41200b82a8',
  'name': 'tools',
  'input': {
    '__type': 'tool_call_with_context',
    'tool_call': {
      'name': 'get_current_promotions',
      'args': {},
      'id': 'call_77a08215fce9428d8bf96a5f',
      'type': 'tool_call'
    },
  },
  'state': {
    'messages': [
      HumanMessage(
        content='查询客户ID为 CUST123456 的个人信息、历史订单
和可用优惠',
        additional_kwargs={},
        response_metadata={},
        id='cc503bf0-acae-41ef-bf20-cc152b9296fd'
      ),
      AIMessage(
        content='我来帮您查询客户 CUST123456
的个人信息、历史订单和可用优惠。这三个查询是独立的，我将同时进行查询',
        additional_kwargs={'refusal': None},
        response_metadata={
          'token_usage': {
            'completion_tokens': 130,
            'prompt_tokens': 467,
            'total_tokens': 597,
            'completion_tokens_details': None,
            'prompt_tokens_details':
{'audio_tokens': None, 'cached_tokens': 0}
          },
          'model_provider': 'openai',
          'model_name': 'deepseek-v4-flash-0731',
          'system_fingerprint': None,
          'id': 'chatcmpl-050cdd80-6a88-90d7-a003-
e9bbef0dd06c',
          'finish_reason': 'tool_calls',
          'logprobs': None
        },
        id='lc_run--019ff659-3d22-75c3-a031-
b10e0265a688-0',
        tool_calls=[
          {
            'name': 'query_customer_data',
            'args': {'customer_id': 'CUST123456'},
            'id': 'call_4ba175f4d1be470099640739',
            'type': 'tool_call'
          },
          {
            'name': 'check_order_history',
            'args': {'customer_id': 'CUST123456'},
            'id': 'call_0fc49e45b1fd4432b268d94a',
            'type': 'tool_call'
          },
          {
            'name': 'get_current_promotions',
            'args': {},
            'id': 'call_77a08215fce9428d8bf96a5f',
            'type': 'tool_call'
          }
        ]
      )
    ]
  }
}

```

```

        invalid_tool_calls=[],
        usage_metadata={
            'input_tokens': 467,
            'output_tokens': 130,
            'total_tokens': 597,
            'input_token_details': {'cache_read': 0},
            'output_token_details': {}
        }
    )
]
},
'triggers': ('__pregel_push',)
}

-----
{
    'id': 'bf5bf44a-fd3a-5e93-e6a3-39071c79927c',
    'name': 'tools',
    'error': None,
    'result': {
        'messages': [
            ToolMessage(
                content='{"name": "张三", "level": "VIP",
"join_date": "2023-01-15"}',
                name='query_customer_data',
                id='ff09b75f-4cda-4657-b417-0385d6d7de24',
                tool_call_id='call_4ba175f4d1be470099640739'
            )
        ],
        'interrupts': []
    }
}

-----
{
    'id': '95521535-b010-7ef2-2f70-f8337c5f470e',
    'name': 'tools',
    'error': None,
    'result': {
        'messages': [
            ToolMessage(
                content='{"total_orders": 15, "total_spent":
25800.0}',
                name='check_order_history',
                id='7c434a02-ec08-41f5-8236-73db63b809bb',
                tool_call_id='call_0fc49e45b1fd4432b268d94a'
            )
        ],
        'interrupts': []
    }
}

-----

```

```

{
  'id': '7ef8e151-3b6c-46b3-d2d6-cb41200b82a8',
  'name': 'tools',
  'error': None,
  'result': {
    'messages': [
      ToolMessage(
        content='{"promotions": ["老用户优惠", "会员专属折扣"],
"valid_until": "2027-01-31"}',
        name='get_current_promotions',
        id='85272dcb-2b57-4706-91c9-4286a2d4d446',
        tool_call_id='call_77a08215fce9428d8bf96a5f'
      )
    ],
    'interrupts': []
  }
}

```

```

{
  'id': 'f13f0bcf-d104-1d5a-4470-bf2faf4c220d',
  'name': 'model',
  'input': {
    'messages': [
      HumanMessage(
        content='查询客户ID为 CUST123456 的个人信息、历史订单和可
用优惠',
        additional_kwargs={},
        response_metadata={},
        id='cc503bf0-acae-41ef-bf20-cc152b9296fd'
      ),
      AIMessage(
        content='我来帮您查询客户 CUST123456
的个人信息、历史订单和可用优惠。这三个查询是独立的，我将同时进行查询',
        additional_kwargs={'refusal': None},
        response_metadata={
          'token_usage': {
            'completion_tokens': 130,
            'prompt_tokens': 467,
            'total_tokens': 597,
            'completion_tokens_details': None,
            'prompt_tokens_details': {'audio_tokens':
None, 'cached_tokens': 0}
          },
          'model_provider': 'openai',
          'model_name': 'deepseek-v4-flash-0731',
          'system_fingerprint': None,
          'id': 'chatcmpl-050cdd80-6a88-90d7-a003-
e9bbef0dd06c',
          'finish_reason': 'tool_calls',
          'logprobs': None
        },
        id='lc_run--019ff659-3d22-75c3-a031-b10e0265a688-0',
        tool_calls=[
          {
            'name': 'query_customer_data',
            'args': {'customer_id': 'CUST123456'},
            'id': 'call_4ba175f4d1be470099640739',
            'type': 'tool_call'
          },
          {
            'name': 'check_order_history',
            'args': {'customer_id': 'CUST123456'},
            'id': 'call_0fc49e45b1fd4432b268d94a',
            'type': 'tool_call'
          },
          {
            'name': 'get_current_promotions',
            'args': {},
            'id': 'call_77a08215fce9428d8bf96a5f',
            'type': 'tool_call'
          }
        ],
        invalid_tool_calls=[],
        usage_metadata={
          'input_tokens': 467,
          'output_tokens': 130,
          'total_tokens': 597,
          'input_token_details': {'cache_read': 0},
          'output_token_details': {}
        }
      ),
    ]
  }
}

```

```

        ToolMessage(
            content='{"name": "张三", "level": "VIP",
"join_date": "2023-01-15"}',
            name='query_customer_data',
            id='ff09b75f-4cda-4657-b417-0385d6d7de24',
            tool_call_id='call_4ba175f4d1be470099640739'
        ),
        ToolMessage(
            content='{"total_orders": 15, "total_spent":
25800.0}',
            name='check_order_history',
            id='7c434a02-ec08-41f5-8236-73db63b809bb',
            tool_call_id='call_0fc49e45b1fd4432b268d94a'
        ),
        ToolMessage(
            content='{"promotions": ["老用户优惠", "会员专属折扣"],
"valid_until": "2027-01-31"}',
            name='get_current_promotions',
            id='85272dcb-2b57-4706-91c9-4286a2d4d446',
            tool_call_id='call_77a08215fce9428d8bf96a5f'
        )
    ],
    'triggers': ('branch:to:model',)
}

```

```

{
  'id': 'f13f0bcf-d104-1d5a-4470-bf2faf4c220d',
  'name': 'model',
  'error': None,
  'result': {
    'messages': [
      AIMessage(
        content='我已经成功查询到了客户 CUST123456 的相关信息，以
下是汇总结果：\n\n## 📋 客户个人信息\n|
项目 | 详情 | \n|-----|-----| \n| **姓名** | 张三 | \n| **会员等级** |
VIP | \n| **加入日期** | 2023-01-15 | \n\n## 🛒
订单历史\n| 项目 | 详情 | \n|-----|-----| \n| **总订单数** | 15 单 | \n|
**总消费金额** | ¥25,800.00 | \n\n## 🎁
可用优惠\n| 项目 | 详情 | \n|-----|-----| \n| **可用促销活动** | ① 老用户
优惠 ② 会员专属折扣 | \n| **有效期至** |
2027-01-31 | \n\n---\n\n**总结：** 客户张三是我司的 VIP 会员，自 2023 年加
入以来共下单 15 次，累计消费
¥25,800。目前可享受"老用户优惠"和"会员专属折扣"两项促销活动，有效期至 2027 年
1
月底。 \n\n如需进一步分析或查询其他信息，请随时告知！',
        additional_kwargs={'refusal': None},
        response_metadata={
          'token_usage': {
            'completion_tokens': 245,
            'prompt_tokens': 693,
            'total_tokens': 938,
            'completion_tokens_details': None,
            'prompt_tokens_details': {'audio_tokens':
None, 'cached_tokens': 0}
          },
          'model_provider': 'openai',
          'model_name': 'deepseek-v4-flash-0731',
          'system_fingerprint': None,
          'id': 'chatcmpl-0eef277b-0c17-98c6-9f01-
da2653d48727',
          'finish_reason': 'stop',
          'logprobs': None
        },
        id='lc_run--019ff659-45fa-7d43-b705-a41151332dd5-0',
        tool_calls=[],
        invalid_tool_calls=[],
        usage_metadata={
          'input_tokens': 693,
          'output_tokens': 245,
          'total_tokens': 938,
          'input_token_details': {'cache_read': 0},
          'output_token_details': {}
        }
      )
    ],
    'interrupts': []
  }
}

```

5) debug输出模式

该模式与tasks模式类似，比task模式多输出任务步骤、时间戳、task类型 (task/task_result)，该模式用于调试、监控task任务的生命周期。

代码如下：

```
In [21]: from langchain.agents import create_agent
from langchain.tools import tool
from typing import Dict, Any
from rich import print as rprint
@tool
def query_customer_data(customer_id: str) -> Dict[str, Any]:
    """
    查询客户基本信息

    Args:
        customer_id: 客户ID，用于唯一标识客户

    Returns:
        包含客户基本信息的字典，如姓名、等级、加入日期等
    """
    # 模拟数据库查询
    return {"name": "张三", "level": "VIP", "join_date": "2023-01-15"}

@tool
def check_order_history(customer_id: str) -> Dict[str, Any]:
    """
    查询客户订单历史

    Args:
        customer_id: 客户ID，用于唯一标识客户

    Returns:
        包含客户订单历史的字典，如总订单数、总花费等
    """
    return {"total_orders": 15, "total_spent": 25800.00}

@tool
def get_current_promotions() -> Dict[str, Any]:
    """
    获取当前可用促销活动

    Returns:
        包含当前可用促销活动的字典，如活动名称、有效日期等
    """
    return {
        "promotions": ["老用户优惠", "会员专属折扣"],
        "valid_until": "2027-01-31"
    }

# 创建客户服务Agent
customer_service_agent = create_agent(
    model=model,
    tools=[query_customer_data, check_order_history, get_current_promotions]
)

for chunk in customer_service_agent.stream(
    {"messages": [{"role": "user", "content": "查询客户ID为 CUST123456 的个人信息、历史订单和可用优惠"}]},
    stream_mode="debug"
):
    rprint(chunk)
    print("-" * 50)
```

```

{
  'step': 1,
  'timestamp': '2026-08-12T14:22:50.948154+00:00',
  'type': 'task',
  'payload': {
    'id': 'dff063c6-e6a2-a302-26d6-5586b5ce697f',
    'name': 'model',
    'input': {
      'messages': [
        HumanMessage(
          content='查询客户ID为 CUST123456 的个人信息、历史订单
和可用优惠',
          additional_kwargs={},
          response_metadata={},
          id='4de03288-105d-4755-a9a9-45a306f47959'
        )
      ],
      'triggers': ('branch:to:model',)
    }
  }
}

```

```

{
  'step': 1,
  'timestamp': '2026-08-12T14:22:52.884443+00:00',
  'type': 'task_result',
  'payload': {
    'id': 'dff063c6-e6a2-a302-26d6-5586b5ce697f',
    'name': 'model',
    'error': None,
    'result': {
      'messages': [
        AIMessage(
          content='我来为您查询客户 CUST123456 的个人信息、历史
订单和可用优惠信息',
          additional_kwargs={'refusal': None},
          response_metadata={
            'token_usage': {
              'completion_tokens': 120,
              'prompt_tokens': 467,
              'total_tokens': 587,
              'completion_tokens_details': None,
              'prompt_tokens_details':
{'audio_tokens': None, 'cached_tokens': 0}
            },
            'model_provider': 'openai',
            'model_name': 'deepseek-v4-flash-0731',
            'system_fingerprint': None,
            'id': 'chatcmpl-f533b6e8-f1e3-9c36-ae1b-
ec825d046a24',
            'finish_reason': 'tool_calls',
            'logprobs': None
          },
          id='lc_run--019ff65a-c254-78e2-8a78-
ce7f2a8c1d8b-0',
          tool_calls=[
            {
              'name': 'query_customer_data',
              'args': {'customer_id': 'CUST123456'},
              'id': 'call_6c38ec4791f14761b0a29123',
              'type': 'tool_call'
            },
            {
              'name': 'check_order_history',
              'args': {'customer_id': 'CUST123456'},
              'id': 'call_3fa0afdf50a94dcb9a1bce71',
              'type': 'tool_call'
            },
            {
              'name': 'get_current_promotions',
              'args': {},
              'id': 'call_d1f587d2598b4ee0ab68e91c',
              'type': 'tool_call'
            }
          ],
          invalid_tool_calls=[],
          usage_metadata={
            'input_tokens': 467,
            'output_tokens': 120,
            'total_tokens': 587,
            'input_token_details': {'cache_read': 0},
            'output_token_details': {}
          }
        )
      ]
    }
  },
  1
},

```

```
    'interrupts': []  
  }  
}
```

```

{
  'step': 2,
  'timestamp': '2026-08-12T14:22:52.884443+00:00',
  'type': 'task',
  'payload': {
    'id': 'c51b9484-725d-3388-7ed9-6e53766a93db',
    'name': 'tools',
    'input': {
      '__type': 'tool_call_with_context',
      'tool_call': {
        'name': 'query_customer_data',
        'args': {'customer_id': 'CUST123456'},
        'id': 'call_6c38ec4791f14761b0a29123',
        'type': 'tool_call'
      }
    },
    'state': {
      'messages': [
        HumanMessage(
          content='查询客户ID为 CUST123456 的个人信息、历史订单和可用优惠',
          additional_kwargs={},
          response_metadata={},
          id='4de03288-105d-4755-a9a9-45a306f47959'
        ),
        AIMessage(
          content='我来为您查询客户 CUST123456 的个人信息、历史订单和可用优惠信息',
          additional_kwargs={'refusal': None},
          response_metadata={
            'token_usage': {
              'completion_tokens': 120,
              'prompt_tokens': 467,
              'total_tokens': 587,
              'completion_tokens_details': None,
              'prompt_tokens_details':
{'audio_tokens': None, 'cached_tokens': 0}
            },
            'model_provider': 'openai',
            'model_name': 'deepseek-v4-flash-0731',
            'system_fingerprint': None,
            'id': 'chatcmpl-f533b6e8-f1e3-9c36-ae1b-ec825d046a24',
            'finish_reason': 'tool_calls',
            'logprobs': None
          },
          id='lc_run--019ff65a-c254-78e2-8a78-ce7f2a8c1d8b-0',
          tool_calls=[
            {
              'name': 'query_customer_data',
              'args': {'customer_id': 'CUST123456'},
              'id': 'call_6c38ec4791f14761b0a29123',
              'type': 'tool_call'
            },
            {
              'name': 'check_order_history',
              'args': {'customer_id': 'CUST123456'},
              'id': 'call_3fa0afdf50a94dcb9a1bce71',
              'type': 'tool_call'
            }
          ]
        )
      ]
    }
  }
}

```



```

{
  'step': 2,
  'timestamp': '2026-08-12T14:22:52.884443+00:00',
  'type': 'task',
  'payload': {
    'id': 'ba81389f-06a5-1096-cb13-e178c1bcf8eb',
    'name': 'tools',
    'input': {
      '__type': 'tool_call_with_context',
      'tool_call': {
        'name': 'check_order_history',
        'args': {'customer_id': 'CUST123456'},
        'id': 'call_3fa0afdf50a94dcb9a1bce71',
        'type': 'tool_call'
      }
    },
    'state': {
      'messages': [
        HumanMessage(
          content='查询客户ID为 CUST123456 的个人信息、历史订单和可用优惠',
          additional_kwargs={},
          response_metadata={},
          id='4de03288-105d-4755-a9a9-45a306f47959'
        ),
        AIMessage(
          content='我来为您查询客户 CUST123456 的个人信息、历史订单和可用优惠信息',
          additional_kwargs={'refusal': None},
          response_metadata={
            'token_usage': {
              'completion_tokens': 120,
              'prompt_tokens': 467,
              'total_tokens': 587,
              'completion_tokens_details': None,
              'prompt_tokens_details':
{'audio_tokens': None, 'cached_tokens': 0}
            },
            'model_provider': 'openai',
            'model_name': 'deepseek-v4-flash-0731',
            'system_fingerprint': None,
            'id': 'chatcmpl-f533b6e8-f1e3-9c36-ae1b-ec825d046a24',
            'finish_reason': 'tool_calls',
            'logprobs': None
          },
          id='lc_run--019ff65a-c254-78e2-8a78-ce7f2a8c1d8b-0',
          tool_calls=[
            {
              'name': 'query_customer_data',
              'args': {'customer_id': 'CUST123456'},
              'id': 'call_6c38ec4791f14761b0a29123',
              'type': 'tool_call'
            },
            {
              'name': 'check_order_history',
              'args': {'customer_id': 'CUST123456'},
              'id': 'call_3fa0afdf50a94dcb9a1bce71',
              'type': 'tool_call'
            }
          ]
        )
      ]
    }
  }
}

```



```

{
  'step': 2,
  'timestamp': '2026-08-12T14:22:52.884443+00:00',
  'type': 'task',
  'payload': {
    'id': '45192082-a250-2db2-86a7-230ba2b95df1',
    'name': 'tools',
    'input': {
      '__type': 'tool_call_with_context',
      'tool_call': {
        'name': 'get_current_promotions',
        'args': {},
        'id': 'call_d1f587d2598b4ee0ab68e91c',
        'type': 'tool_call'
      },
      'state': {
        'messages': [
          HumanMessage(
            content='查询客户ID为 CUST123456 的个人信息、历史订单和可用优惠',
            additional_kwargs={},
            response_metadata={},
            id='4de03288-105d-4755-a9a9-45a306f47959'
          ),
          AIMessage(
            content='我来为您查询客户 CUST123456 的个人信息、历史订单和可用优惠信息',
            additional_kwargs={'refusal': None},
            response_metadata={
              'token_usage': {
                'completion_tokens': 120,
                'prompt_tokens': 467,
                'total_tokens': 587,
                'completion_tokens_details': None,
                'prompt_tokens_details':
{'audio_tokens': None, 'cached_tokens': 0}
              },
              'model_provider': 'openai',
              'model_name': 'deepseek-v4-flash-0731',
              'system_fingerprint': None,
              'id': 'chatcmpl-f533b6e8-f1e3-9c36-ae1b-ec825d046a24',
              'finish_reason': 'tool_calls',
              'logprobs': None
            },
            id='lc_run--019ff65a-c254-78e2-8a78-ce7f2a8c1d8b-0',
            tool_calls=[
              {
                'name': 'query_customer_data',
                'args': {'customer_id': 'CUST123456'},
                'id': 'call_6c38ec4791f14761b0a29123',
                'type': 'tool_call'
              },
              {
                'name': 'check_order_history',
                'args': {'customer_id': 'CUST123456'},
                'id': 'call_3fa0afdf50a94dcb9a1bce71',
                'type': 'tool_call'
              }
            ]
          )
        ]
      }
    }
  }
}

```



```

{
  'step': 2,
  'timestamp': '2026-08-12T14:22:52.991052+00:00',
  'type': 'task_result',
  'payload': {
    'id': 'ba81389f-06a5-1096-cb13-e178c1bcf8eb',
    'name': 'tools',
    'error': None,
    'result': {
      'messages': [
        ToolMessage(
          content='{ "total_orders": 15, "total_spent":
25800.0}',
          name='check_order_history',
          id='ec6404f1-83ad-4bbf-8fd3-df26ff8ad28f',
          tool_call_id='call_3fa0afdf50a94dcb9a1bce71'
        )
      ],
      'interrupts': []
    }
  }
}
-----
{
  'step': 2,
  'timestamp': '2026-08-12T14:22:53.003041+00:00',
  'type': 'task_result',
  'payload': {
    'id': '45192082-a250-2db2-86a7-230ba2b95df1',
    'name': 'tools',
    'error': None,
    'result': {
      'messages': [
        ToolMessage(
          content='{ "promotions": ["老用户优惠", "会员专属折
扣"], "valid_until": "2027-01-31"}',
          name='get_current_promotions',
          id='5abae585-6958-4155-8d82-b5cb770525bf',
          tool_call_id='call_d1f587d2598b4ee0ab68e91c'
        )
      ],
      'interrupts': []
    }
  }
}
-----

```

```

{
  'step': 3,
  'timestamp': '2026-08-12T14:22:53.012040+00:00',
  'type': 'task',
  'payload': {
    'id': 'e099e702-0a69-6dc6-6e75-db2cbb810733',
    'name': 'model',
    'input': {
      'messages': [
        HumanMessage(
          content='查询客户ID为 CUST123456 的个人信息、历史订单
和可用优惠',

          additional_kwargs={},
          response_metadata={},
          id='4de03288-105d-4755-a9a9-45a306f47959'
        ),
        AIMessage(
          content='我来为您查询客户 CUST123456 的个人信息、历史
订单和可用优惠信息',

          additional_kwargs={'refusal': None},
          response_metadata={
            'token_usage': {
              'completion_tokens': 120,
              'prompt_tokens': 467,
              'total_tokens': 587,
              'completion_tokens_details': None,
              'prompt_tokens_details':
{'audio_tokens': None, 'cached_tokens': 0}
            },
            'model_provider': 'openai',
            'model_name': 'deepseek-v4-flash-0731',
            'system_fingerprint': None,
            'id': 'chatcmpl-f533b6e8-f1e3-9c36-ae1b-
ec825d046a24',

            'finish_reason': 'tool_calls',
            'logprobs': None
          },
          id='lc_run--019ff65a-c254-78e2-8a78-
ce7f2a8c1d8b-0',

          tool_calls=[
            {
              'name': 'query_customer_data',
              'args': {'customer_id': 'CUST123456'},
              'id': 'call_6c38ec4791f14761b0a29123',
              'type': 'tool_call'
            },
            {
              'name': 'check_order_history',
              'args': {'customer_id': 'CUST123456'},
              'id': 'call_3fa0afdf50a94dcb9a1bce71',
              'type': 'tool_call'
            },
            {
              'name': 'get_current_promotions',
              'args': {},
              'id': 'call_d1f587d2598b4ee0ab68e91c',
              'type': 'tool_call'
            }
          ],
          invalid_tool_calls=[],
          usage_metadata={
            'input_tokens': 467,
            'output_tokens': 120,

```

```

        'total_tokens': 587,
        'input_token_details': {'cache_read': 0},
        'output_token_details': {}
    }
),
ToolMessage(
    content='{"name": "张三", "level": "VIP",
"join_date": "2023-01-15"}',
    name='query_customer_data',
    id='0dfe608c-0715-48b4-a42c-8a75483d2cd2',
    tool_call_id='call_6c38ec4791f14761b0a29123'
),
ToolMessage(
    content='{"total_orders": 15, "total_spent":
25800.0}',
    name='check_order_history',
    id='ec6404f1-83ad-4bbf-8fd3-df26ff8ad28f',
    tool_call_id='call_3fa0afdf50a94dcb9a1bce71'
),
ToolMessage(
    content='{"promotions": ["老用户优惠", "会员专属折
扣"], "valid_until": "2027-01-31"}',
    name='get_current_promotions',
    id='5abae585-6958-4155-8d82-b5cb770525bf',
    tool_call_id='call_d1f587d2598b4ee0ab68e91c'
)
    ]
},
    'triggers': ('branch:to:model',)
}
}
}

```

```

{
  'step': 3,
  'timestamp': '2026-08-12T14:22:56.260428+00:00',
  'type': 'task_result',
  'payload': {
    'id': 'e099e702-0a69-6dc6-6e75-db2cbb810733',
    'name': 'model',
    'error': None,
    'result': {
      'messages': [
        AIMessage(
          content='我已经成功查询到客户 CUST123456 的相关信息，
          以下是详细结果：\n\n---\n\n## 📋
          客户个人信息\n| 项目 | 内容 |\n|-----|-----|\n| **客户ID** |
          CUST123456 |\n| **姓名** | 张三 |\n| **会员等级** |
          VIP |\n| **加入日期** | 2023-01-15 |\n\n---\n\n## 🛒 历史订单信息\n| 项目 | 内容 |\n|-----|-----|\n| **总订单数**
          | 15 笔 |\n| **累计消费金额** | ¥25,800.00 |\n\n---\n\n## 🎁 当前可用优惠\n| 项目 | 内容 |\n|-----|-----|\n
          **可用促销活动** | ① 老用户优惠<br>② 会员专属折扣 |\n| **优惠有效期至** |
          2027-01-31 |\n\n---\n\n### 📌 小结\n客户
          **张三** 是 **VIP 等级** 会员，自 2023 年加入以来共完成 **15 笔订单**，累计
          消费 **¥25,800**。目前可享受
          **老用户优惠** 和 **会员专属折扣** 两项促销活动，有效期至 2027 年 1 月 31
          日。\n\n如需进一步了解订单详情或优惠具体内容，请随时告诉我！',
          additional_kwargs={'refusal': None},
          response_metadata={
            'token_usage': {
              'completion_tokens': 281,
              'prompt_tokens': 683,
              'total_tokens': 964,
              'completion_tokens_details': None,
              'prompt_tokens_details':
                {'audio_tokens': None, 'cached_tokens': 0}
            },
            'model_provider': 'openai',
            'model_name': 'deepseek-v4-flash-0731',
            'system_fingerprint': None,
            'id': 'chatcmpl-8c805add-40c3-94e8-9f27-5c1d82374aee',
            'finish_reason': 'stop',
            'logprobs': None
          },
          id='lc_run--019ff65a-ca6a-77e1-8e01-218848286855-0',
          tool_calls=[],
          invalid_tool_calls=[],
          usage_metadata={
            'input_tokens': 683,
            'output_tokens': 281,
            'total_tokens': 964,
            'input_token_details': {'cache_read': 0},
            'output_token_details': {}
          }
        )
      ]
    },
    'interrupts': []
  }
}

```

6) checkpoints输出模式

该模式中，每当检查点（checkpoint）被创建时会触发输出，输出包含检查点中的状态，用于需要状态持久化、 workflow 恢复 或 分布式执行跟踪的高级场景。

代码案例如下：

```
In [22]: from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents import create_agent
from langchain.tools import tool
from typing import Dict, Any
from rich import print as rprint

@tool
def query_customer_data(customer_id: str) -> Dict[str, Any]:
    """
    查询客户基本信息

    Args:
        customer_id: 客户ID，用于唯一标识客户

    Returns:
        包含客户基本信息的字典，如姓名、等级、加入日期等
    """
    # 模拟数据库查询
    return {"name": "张三", "level": "VIP", "join_date": "2023-01-15"}

@tool
def check_order_history(customer_id: str) -> Dict[str, Any]:
    """
    查询客户订单历史

    Args:
        customer_id: 客户ID，用于唯一标识客户

    Returns:
        包含客户订单历史的字典，如总订单数、总花费等
    """
    return {"total_orders": 15, "total_spent": 25800.00}

@tool
def get_current_promotions() -> Dict[str, Any]:
    """
    获取当前可用促销活动

    Returns:
        包含当前可用促销活动的字典，如活动名称、有效日期等
    """
    return {
        "promotions": ["老用户优惠", "会员专属折扣"],
        "valid_until": "2027-01-31"
    }

# 1. 创建内存检查点存储
checkpointer = InMemorySaver()
# 2. 创建Agent
```

```

customer_service_agent = create_agent(
    model=model,
    tools=[query_customer_data, check_order_history, get_current_promotions],
    checkpoint=checkpoint # 启用检查点
)

# 3. 创建唯一的会话ID
config = {"configurable": {"thread_id": "session01"}}

# 4. 调用Agent
checkpoint_count = 0
# 使用checkpoints模式进行流式监控
for chunk in customer_service_agent.stream(
    {"messages": [{"role": "user", "content": "查询客户ID为 CUST123456 的完整信息和可用优惠"}]},
    config=config,
    stream_mode="checkpoints"
):
    checkpoint_count += 1
    print(f"检查点 #{checkpoint_count}")
    print(chunk)
    print("-" * 50)

```

检查点 #1

```
{'config': {'configurable': {'checkpoint_ns': '', 'thread_id': 'session01', 'checkpoint_id': '1f19659b-b6cc-61de-bfff-a9cfe91edb35'}}, 'parent_config': None, 'values': {'messages': [], 'metadata': {'source': 'input', 'step': -1, 'parents': {}}, 'next': ['__start__'], 'tasks': [{'id': 'cab75a35-2576-0882-3b2e-74d6eb350861', 'name': '__start__', 'interrupts': (), 'state': None}]}}
```

检查点 #2

```
{'config': {'configurable': {'checkpoint_ns': '', 'thread_id': 'session01', 'checkpoint_id': '1f19659b-b6dd-62c9-8000-a1f00421f55f'}}, 'parent_config': {'configurable': {'checkpoint_ns': '', 'thread_id': 'session01', 'checkpoint_id': '1f19659b-b6cc-61de-bfff-a9cfe91edb35'}}, 'values': {'messages': [HumanMessage(content='查询客户ID为 CUST123456 的完整信息和可用优惠', additional_kwargs={}, response_metadata={}, id='0d7fb01e-269d-4c2c-9d1b-bd0431944f2e')], 'metadata': {'source': 'loop', 'step': 0, 'parents': {}}, 'next': ['model'], 'tasks': [{'id': 'f018b6f3-0956-94a3-b1b9-958626897951', 'name': 'model', 'interrupts': (), 'state': None}]}}
```

检查点 #3

```
{'config': {'configurable': {'checkpoint_ns': '', 'thread_id': 'session01', 'checkpoint_id': '1f19659b-c956-6fb1-8001-cc0e6d5f6fc8'}}, 'parent_config': {'configurable': {'checkpoint_ns': '', 'thread_id': 'session01', 'checkpoint_id': '1f19659b-b6dd-62c9-8000-a1f00421f55f'}}, 'values': {'messages': [HumanMessage(content='查询客户ID为 CUST123456 的完整信息和可用优惠', additional_kwargs={}, response_metadata={}, id='0d7fb01e-269d-4c2c-9d1b-bd0431944f2e'), AIMessage(content='我来帮你查询客户 CUST123456 的完整信息和可用优惠。我需要同步查询客户基本信息、订单历史和当前促销活动。\\n\\n', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 128, 'prompt_tokens': 464, 'total_tokens': 592, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'deepseek-v4-flash-0731', 'system_fingerprint': None, 'id': 'chatcmpl-d4300683-53ee-970a-981f-3d4cd5116a1e', 'finish_reason': 'tool_calls', 'logprobs': None, id='lc_run--019ff65d-9785-7163-9da3-60505df87ce8-0', tool_calls=[{'name': 'query_customer_data', 'args': {'customer_id': 'CUST123456'}, 'id': 'call_5d15dc198dd5472889fa43a0', 'type': 'tool_call'}, {'name': 'check_order_history', 'args': {'customer_id': 'CUST123456'}, 'id': 'call_454b948b69a54dbfb0e21436', 'type': 'tool_call'}, {'name': 'get_current_promotions', 'args': {}, 'id': 'call_e8b0aa2d1c6d4245bcf5834d', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 464, 'output_tokens': 128, 'total_tokens': 592, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}})], 'metadata': {'source': 'loop', 'step': 1, 'parents': {}}, 'next': ['tools', 'tools', 'tools'], 'tasks': [{'id': '7786a039-77b8-f35a-5ba7-69e075b130b7', 'name': 'tools', 'interrupts': (), 'state': None}, {'id': '5e669a7d-648c-64b9-a14e-dc9f91eec8ba', 'name': 'tools', 'interrupts': (), 'state': None}, {'id': 'b6a1d9fe-bc69-967c-b6ce-598add494653', 'name': 'tools', 'interrupts': (), 'state': None}]}}
```

检查点 #4

```
{'config': {'configurable': {'checkpoint_ns': '', 'thread_id': 'session01', 'checkpoint_id': '1f19659b-c97a-64af-8002-2f3dbb9face1'}}, 'parent_config': {'configurable': {'checkpoint_ns': '', 'thread_id': 'session01', 'checkpoint_id': '1f19659b-c956-6fb1-8001-cc0e6d5f6fc8'}}, 'values': {'messages': [HumanMessage(content='查询客户ID为 CUST123456 的完整信息和可用优惠', additional_kwargs={}, response_metadata={}, id='0d7fb01e-269d-4c2c-9d1b-bd0431944f2e'), AIMessage(content='我来帮你查询客户 CUST123456 的完整信息和可用优惠。我需要同步查询客户基本信息、订单历史和当前促销活动。\\n\\n', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 128, 'prompt_tokens': 464, 'total_tokens': 592, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'deepseek-v4-flash-0731', 'system_fingerprint': None, 'id': 'chatcmpl-d4300683-53ee-970a-981f-3d4cd5116a1e', 'finish_reason': 'tool_calls', 'logprobs': None, id='lc_run--019ff65d-9785-7163-9da3-60505df87ce8-0', tool_calls=[{'name': 'query_customer_data', 'args': {'customer_id': 'CUST123456'}, 'id': 'call_5d15dc198dd5472889fa43a0', 'type': 'tool_call'}, {'name': 'check_order_history', 'args': {'customer_id': 'CUST123456'}, 'id': 'call_454b948b69a54dbfb0e21436', 'type': 'tool_call'}, {'name': 'get_current_promotions', 'args': {}, 'id': 'call_e8b0aa2d1c6d4245bcf5834d', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 464, 'output_tokens': 128, 'total_tokens': 592, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}})], 'metadata': {'source': 'loop', 'step': 1, 'parents': {}}, 'next': ['tools', 'tools', 'tools'], 'tasks': [{'id': '7786a039-77b8-f35a-5ba7-69e075b130b7', 'name': 'tools', 'interrupts': (), 'state': None}, {'id': '5e669a7d-648c-64b9-a14e-dc9f91eec8ba', 'name': 'tools', 'interrupts': (), 'state': None}, {'id': 'b6a1d9fe-bc69-967c-b6ce-598add494653', 'name': 'tools', 'interrupts': (), 'state': None}]}}
```

```
ToolMessage(content={'name': "张三", "level": "VIP", "join_date": "2023-01-15"},
name='query_customer_data', id='996ca940-4d8f-448c-b927-e2db9e90f0a8',
tool_call_id='call_5d15dc198dd5472889fa43a0'), ToolMessage(content={'total_orders': 15,
"total_spent": 25800.0}, name='check_order_history', id='07f77f43-9c8d-4e1c-9271-
454035dbb836', tool_call_id='call_454b948b69a54dbfb0e21436'),
ToolMessage(content={'promotions': ["老用户优惠", "会员专属折扣"], "valid_until": "2027-01-
31"}, name='get_current_promotions', id='d2c21fbd-0c13-4486-ad21-960dd67604e3',
tool_call_id='call_e8b0aa2d1c6d4245bcf5834d')), 'metadata': {'source': 'loop', 'step': 2, 'parents':
{}}, 'next': ['model'], 'tasks': [{'id': 'a80c364a-fa1d-5c4e-efa5-67d6ee08264d', 'name': 'model',
'interrupts': (), 'state': None}]}
```

检查点 #5

```
{'config': {'configurable': {'checkpoint_ns': '', 'thread_id': 'session01', 'checkpoint_id': '1f19659b-
e894-67c1-8003-cd4e201e3129'}}, 'parent_config': {'configurable': {'checkpoint_ns': '', 'thread_id':
'session01', 'checkpoint_id': '1f19659b-c97a-64af-8002-2f3dbb9face1'}}, 'values': {'messages':
[HumanMessage(content='查询客户ID为 CUST123456 的完整信息和可用优惠', additional_kwargs=
{}, response_metadata={}, id='0d7fb01e-269d-4c2c-9d1b-bd0431944f2e'), AIMessage(content='我
来帮你查询客户 CUST123456 的完整信息和可用优惠。我需要同步查询客户基本信息、订单历史和当
前促销活动。\\n\\n', additional_kwargs={'refusal': None}, response_metadata={'token_usage':
{'completion_tokens': 128, 'prompt_tokens': 464, 'total_tokens': 592,
'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None,
'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'deepseek-v4-flash-0731',
'system_fingerprint': None, 'id': 'chatcmpl-d4300683-53ee-970a-981f-3d4cd5116a1e',
'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run--019ff65d-9785-7163-9da3-
60505df87ce8-0', tool_calls=[{'name': 'query_customer_data', 'args': {'customer_id':
'CUST123456'}, 'id': 'call_5d15dc198dd5472889fa43a0', 'type': 'tool_call'}, {'name':
'check_order_history', 'args': {'customer_id': 'CUST123456'}, 'id':
'call_454b948b69a54dbfb0e21436', 'type': 'tool_call'}, {'name': 'get_current_promotions', 'args':
{}, 'id': 'call_e8b0aa2d1c6d4245bcf5834d', 'type': 'tool_call'}], invalid_tool_calls=[],
usage_metadata={'input_tokens': 464, 'output_tokens': 128, 'total_tokens': 592,
'input_token_details': {'cache_read': 0}, 'output_token_details': {}},
ToolMessage(content={'name': "张三", "level": "VIP", "join_date": "2023-01-15"},
name='query_customer_data', id='996ca940-4d8f-448c-b927-e2db9e90f0a8',
tool_call_id='call_5d15dc198dd5472889fa43a0'), ToolMessage(content={'total_orders': 15,
"total_spent": 25800.0}, name='check_order_history', id='07f77f43-9c8d-4e1c-9271-
454035dbb836', tool_call_id='call_454b948b69a54dbfb0e21436'),
ToolMessage(content={'promotions': ["老用户优惠", "会员专属折扣"], "valid_until": "2027-01-
31"}, name='get_current_promotions', id='d2c21fbd-0c13-4486-ad21-960dd67604e3',
tool_call_id='call_e8b0aa2d1c6d4245bcf5834d'), AIMessage(content='已成功查询到客户
CUST123456 的完整信息，以下是汇总结果：\\n\\n## 📄 客户基本信息\\n| 项目 | 内容 |\\n|-----|-----|\\n|
**客户ID** | CUST123456 |\\n| **姓名** | 张三 |\\n| **会员等级** | VIP |\\n| **加入日期** | 2023-01-15
|\\n\\n## 📋 订单历史\\n| 项目 | 内容 |\\n|-----|-----|\\n| **总订单数** | 15 笔 |\\n| **总消费金额** |
¥25,800.00 |\\n\\n## 🎁 可用优惠活动\\n| 项目 | 内容 |\\n|-----|-----|\\n| **可用促销** | ① 老用户优惠
<br>② 会员专属折扣 |\\n| **有效期至** | 2027-01-31 |\\n\\n---\\n\\n### 💡 综合建议\\n作为 **VIP 会员**
（加入时间较长，消费能力较强），您目前可以享受以下优惠：\\n- **老用户优惠**：符合您较长的会员
历史\\n- **会员专属折扣**：VIP 等级专属权益\\n\\n两项优惠均有效至 2027 年 1 月 31 日，建议您在有
效期内及时使用。如需进一步了解某个优惠的具体内容或办理相关业务，请随时告诉我！',
additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens':
278, 'prompt_tokens': 689, 'total_tokens': 967, 'completion_tokens_details': None,
'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}}, 'model_provider': 'openai',
'model_name': 'deepseek-v4-flash-0731', 'system_fingerprint': None, 'id': 'chatcmpl-c19bd572-
b049-9193-825e-076bf459c058', 'finish_reason': 'stop', 'logprobs': None}, id='lc_run--019ff65d-
9f18-7ae3-9ccd-21443cf0fd57-0', tool_calls=[], invalid_tool_calls=[], usage_metadata=
{'input_tokens': 689, 'output_tokens': 278, 'total_tokens': 967, 'input_token_details':
{'cache_read': 0}, 'output_token_details': {}}]), 'metadata': {'source': 'loop', 'step': 3, 'parents': {}},
'next': [], 'tasks': []}
```

7) custom输出模式

开发者通过`get_stream_writer` 在工具或节点内部自定义发送的数据，用于输出业务逻辑相关的进度信息（如“已处理10/100条记录”）、自定义日志或指标。

举例：生成销售报告和库存报告Agent

```
In [23]: from langchain.agents import create_agent
from langgraph.config import get_stream_writer
from langchain.tools import tool
import time

@tool
def generate_sales_report() -> str:
    """生成销售报告"""
    writer = get_stream_writer()
    writer({"type": "生成销售报告", "message": "开始生成销售报告"})
    # 模拟数据处理
    for i in range(1, 4):
        time.sleep(0.5)
        writer({"type": "生成销售报告", "message": f"生成销售报告进度百分比：{i * 25}%"})

    writer({"type": "生成销售报告", "message": "报告生成完成"})

    return f"销售报告：总收入150万元，同比增长12%"

@tool
def generate_inventory_report() -> str:
    """生成库存报告"""
    writer = get_stream_writer()
    writer("开始库存分析...")
    time.sleep(0.5)
    writer("检查当前库存量...")
    time.sleep(0.5)
    writer("生成库存报告...")

    return "当前库存量为10000件，库存充足，无异常"

# 创建报告生成agent
reporting_agent = create_agent(
    model=model,
    tools=[generate_sales_report, generate_inventory_report]
)
for chunk in reporting_agent.stream(
    {"messages": [{"role": "user", "content": "生成销售报告和库存报告"}]},
    stream_mode="custom"
):
    print(chunk)
    print("-" * 50)
```

```
{'type': '生成销售报告', 'message': '开始生成销售报告'}
-----
开始库存分析...
-----
{'type': '生成销售报告', 'message': '生成销售报告进度百分比：25%'}
-----
检查当前库存量...
-----
{'type': '生成销售报告', 'message': '生成销售报告进度百分比：50%'}
-----
生成库存报告...
-----
{'type': '生成销售报告', 'message': '生成销售报告进度百分比：75%'}
-----
{'type': '生成销售报告', 'message': '报告生成完成'}
```

8.3 流式输出模式总结

如下是LangChain Agent输出模式对比：

模式	输出内容	使用场景
values	每个步骤执行后，都会输出完整的状态信息	适用于每一步都要获取完整状态、状态持久化场景
updates（默认）	每个步骤执行后，只增量更新状态中发生变化的内容	用于监控Agent执行进度，例如观察Agent决定调用工具、工具执行结果等步骤
messages	输出流式返回的Token以及相关的元数据（如：来自哪个节点model/tool）	实现类似ChatGPT的打字机效果，为聊天机器人等交互式应用提供最佳的实时体验
tasks	输出当前task任务开始和结束的时间，包含任务的结果和错误信息	该模式用于监控任务的生命周期
debug	与tasks模式类似，比task模式多输出任务步骤、时间戳、task类型（task/task_result）	该模式用于调试、监控task任务的生命周期
checkpoints	当检查点（checkpoint）被创建时会触发输出，输出包含检查点中的状态	用于需要状态持久化、工作流恢复或分布式执行跟踪的高级场景
custom	通过get_stream_writer在工具或节点内部自定义发送的数据	用于输出业务逻辑相关的进度信息（如“已处理10/100条记录”）、自定义日志或指标

我们可以根据不同的目标来选择不同的输出模式。例如：

- 实现实时对话交互，优先选择messages模式；
- 观察Agent的思考与执行步骤，优先选择updates模式；
- 需要查看每一步状态优先选择values/tasks/debug模式；
- 在工具执行时输出自定义业务日志优先选择custom模式。

此外，以上这些模式还可以组合使用，例如，可以同时指定`stream_mode=`
[“tasks”，“updates”]，这样在同一个循环里既能查看Agent task任务执行内容，又能
显示Agent每步的更新。

举例：

```
In [25]: from langchain.agents import create_agent
from langchain.tools import tool
from typing import Dict, Any
from rich import print as rprint
@tool
def query_customer_data(customer_id: str) -> Dict[str, Any]:
    """
    查询客户基本信息

    Args:
        customer_id: 客户ID，用于唯一标识客户

    Returns:
        包含客户基本信息的字典，如姓名、等级、加入日期等
    """
    # 模拟数据库查询
    return {"name": "张三", "level": "VIP", "join_date": "2023-01-15"}

@tool
def check_order_history(customer_id: str) -> Dict[str, Any]:
    """
    查询客户订单历史

    Args:
        customer_id: 客户ID，用于唯一标识客户

    Returns:
        包含客户订单历史的字典，如总订单数、总花费等
    """
    return {"total_orders": 15, "total_spent": 25800.00}

@tool
def get_current_promotions() -> Dict[str, Any]:
    """
    获取当前可用促销活动

    Returns:
        包含当前可用促销活动的字典，如活动名称、有效日期等
    """
    return {
        "promotions": ["老用户优惠", "会员专属折扣"],
        "valid_until": "2027-01-31"
    }

# 创建客户服务Agent
customer_service_agent = create_agent(
    model=model,
    tools=[query_customer_data, check_order_history, get_current_promotions]
)

for stream_mode, chunk in customer_service_agent.stream(
    {"messages": [{"role": "user", "content": "查询客户ID为 CUST123456 的个人信息、历史订单和可用优惠"}]},
```

```

stream_mode=["tasks","updates"]
):
rprint(f"当前流模式：{stream_mode},当前数据：{chunk}")
print("-" * 50)

```

当前流模式：tasks, 当前数据：{'id': 'acf00515-7129-6ac6-3dbb-5aa16861c25e', 'name': 'model', 'input': {'messages': [HumanMessage(content='查询客户ID为 CUST123456 的个人信息、历史订单和可用优惠', additional_kwargs={}, response_metadata={}, id='8ae558fc-cd77-4cc6-935a-1f85d151c49f')]}}, 'triggers': ('branch:to:model',)}

当前流模式：updates, 当前数据：{'model': {'messages': [AIMessage(content='我来为您查询客户 CUST123456 的个人信息、历史订单和可用优惠信息。\\n\\n', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 120, 'prompt_tokens': 467, 'total_tokens': 587, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'deepseek-v4-flash-0731', 'system_fingerprint': None, 'id': 'chatcpl-bd803a1c-b5db-94f8-a59f-a084b4e1fa3a', 'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run--019ff667-1611-7641-986b-20c0a9b808c2-0', tool_calls=[{'name': 'query_customer_data', 'args': {'customer_id': 'CUST123456'}, 'id': 'call_ff04aaac3dff41f9929f1f38', 'type': 'tool_call'}, {'name': 'check_order_history', 'args': {'customer_id': 'CUST123456'}, 'id': 'call_cf75141970ef4837a874a036', 'type': 'tool_call'}, {'name': 'get_current_promotions', 'args': {}, 'id': 'call_8b499bdd0ae145d488515c8d', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 467, 'output_tokens': 120, 'total_tokens': 587, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}})]}}

当前流模式：tasks, 当前数据：{'id': 'acf00515-7129-6ac6-3dbb-5aa16861c25e', 'name': 'model', 'error': None, 'result': {'messages': [AIMessage(content='我来为您查询客户 CUST123456 的个人信息、历史订单和可用优惠信息。\\n\\n', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 120, 'prompt_tokens': 467, 'total_tokens': 587, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'deepseek-v4-flash-0731', 'system_fingerprint': None, 'id': 'chatcpl-bd803a1c-b5db-94f8-a59f-a084b4e1fa3a', 'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run--019ff667-1611-7641-986b-20c0a9b808c2-0', tool_calls=[{'name': 'query_customer_data', 'args': {'customer_id': 'CUST123456'}, 'id': 'call_ff04aaac3dff41f9929f1f38', 'type': 'tool_call'}, {'name': 'check_order_history', 'args': {'customer_id': 'CUST123456'}, 'id': 'call_cf75141970ef4837a874a036', 'type': 'tool_call'}, {'name': 'get_current_promotions', 'args': {}, 'id': 'call_8b499bdd0ae145d488515c8d', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 467, 'output_tokens': 120, 'total_tokens': 587, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}})]}, 'interrupts': []}

```

-----
当前流模式：tasks, 当前数据：{'id': '7e3cd596-721b-cfc7-222d-
e1f75c5b9a4b', 'name': 'tools', 'input': {'__type':
'tool_call_with_context', 'tool_call': {'name':
'query_customer_data', 'args': {'customer_id': 'CUST123456'}, 'id':
'call_ff04aaac3dff41f9929f1f38', 'type': 'tool_call'}, 'state':
{'messages': [HumanMessage(content='查询客户ID为
CUST123456 的个人信息、历史订单和可用优惠', additional_kwargs={},
response_metadata={},
id='8ae558fc-cd77-4cc6-935a-1f85d151c49f'), AIMessage(content='我来为
您查询客户 CUST123456
的个人信息、历史订单和可用优惠信息。\\n\\n', additional_kwargs={'refusal':
None}, response_metadata={'token_usage':
{'completion_tokens': 120, 'prompt_tokens': 467, 'total_tokens':
587, 'completion_tokens_details': None,
'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens':
0}}, 'model_provider': 'openai', 'model_name':
'deepseek-v4-flash-0731', 'system_fingerprint': None, 'id':
'chatcmpl-bd803a1c-b5db-94f8-a59f-a084b4e1fa3a',
'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run-
-019ff667-1611-7641-986b-20c0a9b808c2-0',
tool_calls=[{'name': 'query_customer_data', 'args': {'customer_id':
'CUST123456'}, 'id':
'call_ff04aaac3dff41f9929f1f38', 'type': 'tool_call'}, {'name':
'check_order_history', 'args': {'customer_id':
'CUST123456'}, 'id': 'call_cf75141970ef4837a874a036', 'type':
'tool_call'}, {'name': 'get_current_promotions',
'args': {}, 'id': 'call_8b499bdd0ae145d488515c8d', 'type':
'tool_call'}], invalid_tool_calls=[],
usage_metadata={'input_tokens': 467, 'output_tokens': 120,
'total_tokens': 587, 'input_token_details':
{'cache_read': 0}, 'output_token_details': {}}]}], 'triggers':
('__pregel_push',)}}
-----

```

```

当前流模式：tasks, 当前数据：{'id': '44b8785f-f292-6b9c-e73c-
e1e343c32ad3', 'name': 'tools', 'input': {'__type':
'tool_call_with_context', 'tool_call': {'name':
'check_order_history', 'args': {'customer_id': 'CUST123456'}, 'id':
'call_cf75141970ef4837a874a036', 'type': 'tool_call'}, 'state':
{'messages': [HumanMessage(content='查询客户ID为
CUST123456 的个人信息、历史订单和可用优惠', additional_kwargs={},
response_metadata={},
id='8ae558fc-cd77-4cc6-935a-1f85d151c49f'), AIMessage(content='我来为
您查询客户 CUST123456
的个人信息、历史订单和可用优惠信息。\\n\\n', additional_kwargs={'refusal':
None}, response_metadata={'token_usage':
{'completion_tokens': 120, 'prompt_tokens': 467, 'total_tokens':
587, 'completion_tokens_details': None,
'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens':
0}}, 'model_provider': 'openai', 'model_name':
'deepseek-v4-flash-0731', 'system_fingerprint': None, 'id':
'chatcmpl-bd803a1c-b5db-94f8-a59f-a084b4e1fa3a',
'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run-
-019ff667-1611-7641-986b-20c0a9b808c2-0',
tool_calls=[{'name': 'query_customer_data', 'args': {'customer_id':
'CUST123456'}, 'id':
'call_ff04aaac3dff41f9929f1f38', 'type': 'tool_call'}, {'name':
'check_order_history', 'args': {'customer_id':
'CUST123456'}, 'id': 'call_cf75141970ef4837a874a036', 'type':
'tool_call'}, {'name': 'get_current_promotions',
'args': {}, 'id': 'call_8b499bdd0ae145d488515c8d', 'type':
'tool_call'}], invalid_tool_calls=[],
usage_metadata={'input_tokens': 467, 'output_tokens': 120,
'total_tokens': 587, 'input_token_details':
{'cache_read': 0}, 'output_token_details': {}})]}, 'triggers':
('__pregel_push',)}

```

```

当前流模式：tasks, 当前数据：{'id': '9d617496-ce1f-3112-7617-489d3b6e75a9', 'name': 'tools', 'input': {'__type': 'tool_call_with_context', 'tool_call': {'name': 'get_current_promotions', 'args': {}, 'id': 'call_8b499bdd0ae145d488515c8d', 'type': 'tool_call'}, 'state': {'messages': [HumanMessage(content='查询客户ID为CUST123456 的个人信息、历史订单和可用优惠', additional_kwargs={}, response_metadata={}, id='8ae558fc-cd77-4cc6-935a-1f85d151c49f'), AIMessage(content='我来为您查询客户 CUST123456 的个人信息、历史订单和可用优惠信息。\\n\\n', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 120, 'prompt_tokens': 467, 'total_tokens': 587, 'completion_tokens_details': None, 'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'deepseek-v4-flash-0731', 'system_fingerprint': None, 'id': 'chatcmpl-bd803a1c-b5db-94f8-a59f-a084b4e1fa3a', 'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run-019ff667-1611-7641-986b-20c0a9b808c2-0', tool_calls=[{'name': 'query_customer_data', 'args': {'customer_id': 'CUST123456'}, 'id': 'call_ff04aaac3dff41f9929f1f38', 'type': 'tool_call'}, {'name': 'check_order_history', 'args': {'customer_id': 'CUST123456'}, 'id': 'call_cf75141970ef4837a874a036', 'type': 'tool_call'}, {'name': 'get_current_promotions', 'args': {}, 'id': 'call_8b499bdd0ae145d488515c8d', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 467, 'output_tokens': 120, 'total_tokens': 587, 'input_token_details': {'cache_read': 0}, 'output_token_details': {}})]}, 'triggers': ('__pregel_push',))}

```

```

-----
当前流模式：updates, 当前数据：{'tools': {'messages': [ToolMessage(content='{"name": "张三", "level": "VIP", "join_date": "2023-01-15"}', name='query_customer_data', id='b7b88b34-e545-4a38-855d-a826bcc1775e', tool_call_id='call_ff04aaac3dff41f9929f1f38')]]}

```

```

-----
当前流模式：tasks, 当前数据：{'id': '7e3cd596-721b-cfc7-222d-e1f75c5b9a4b', 'name': 'tools', 'error': None, 'result': {'messages': [ToolMessage(content='{"name": "张三", "level": "VIP", "join_date": "2023-01-15"}', name='query_customer_data', id='b7b88b34-e545-4a38-855d-a826bcc1775e', tool_call_id='call_ff04aaac3dff41f9929f1f38')]]}, 'interrupts': []}

```

```

-----
当前流模式：updates, 当前数据：{'tools': {'messages': [ToolMessage(content='{"total_orders": 15, "total_spent": 25800.0}', name='check_order_history', id='4c4cf726-af78-4f78-8093-4f1eef089dc3', tool_call_id='call_cf75141970ef4837a874a036')]]}

```

当前流模式：tasks, 当前数据：{'id': '44b8785f-f292-6b9c-e73c-e1e343c32ad3', 'name': 'tools', 'error': None, 'result': {'messages': [ToolMessage(content='{"total_orders": 15, "total_spent": 25800.0}', name='check_order_history', id='4c4cf726-af78-4f78-8093-4f1eef089dc3', tool_call_id='call_cf75141970ef4837a874a036')]}}, 'interrupts': []}

当前流模式：updates, 当前数据：{'tools': {'messages': [ToolMessage(content='{"promotions": ["老用户优惠", "会员专属折扣"], "valid_until": "2027-01-31"}', name='get_current_promotions', id='57fd1d0b-89d6-498c-a47f-9620dddb8932', tool_call_id='call_8b499bdd0ae145d488515c8d')]}]}

当前流模式：tasks, 当前数据：{'id': '9d617496-ce1f-3112-7617-489d3b6e75a9', 'name': 'tools', 'error': None, 'result': {'messages': [ToolMessage(content='{"promotions": ["老用户优惠", "会员专属折扣"], "valid_until": "2027-01-31"}', name='get_current_promotions', id='57fd1d0b-89d6-498c-a47f-9620dddb8932', tool_call_id='call_8b499bdd0ae145d488515c8d')]}}, 'interrupts': []}

```

当前流模式：tasks, 当前数据：{'id': '5e911902-64b2-1498-3036-a9293de59bb6', 'name': 'model', 'input': {'messages':
[HumanMessage(content='查询客户ID为 CUST123456 的个人信息、历史订单和可用
优惠', additional_kwargs={},
response_metadata={}, id='8ae558fc-cd77-4cc6-935a-1f85d151c49f'),
AIMessage(content='我来为您查询客户 CUST123456
的个人信息、历史订单和可用优惠信息。\\n\\n', additional_kwargs={'refusal':
None}, response_metadata={'token_usage':
{'completion_tokens': 120, 'prompt_tokens': 467, 'total_tokens':
587, 'completion_tokens_details': None,
'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens':
0}}, 'model_provider': 'openai', 'model_name':
'deepseek-v4-flash-0731', 'system_fingerprint': None, 'id':
'chatcmpl-bd803a1c-b5db-94f8-a59f-a084b4e1fa3a',
'finish_reason': 'tool_calls', 'logprobs': None}, id='lc-run-
-019ff667-1611-7641-986b-20c0a9b808c2-0',
tool_calls=[{'name': 'query_customer_data', 'args': {'customer_id':
'CUST123456'}, 'id':
'call_ff04aaac3dffa41f9929f1f38', 'type': 'tool_call'}, {'name':
'check_order_history', 'args': {'customer_id':
'CUST123456'}, 'id': 'call_cf75141970ef4837a874a036', 'type':
'tool_call'}, {'name': 'get_current_promotions',
'args': {}, 'id': 'call_8b499bdd0ae145d488515c8d', 'type':
'tool_call'}], invalid_tool_calls=[],
usage_metadata={'input_tokens': 467, 'output_tokens': 120,
'total_tokens': 587, 'input_token_details':
{'cache_read': 0}, 'output_token_details': {}}),
ToolMessage(content='{"name": "张三", "level": "VIP", "join_date":
"2023-01-15"}', name='query_customer_data', id='b7b88b34-e545-4a38-
855d-a826bcc1775e',
tool_call_id='call_ff04aaac3dffa41f9929f1f38'),
ToolMessage(content='{"total_orders": 15, "total_spent": 25800.0}',
name='check_order_history', id='4c4cf726-af78-4f78-8093-
4f1eef089dc3',
tool_call_id='call_cf75141970ef4837a874a036'),
ToolMessage(content='{"promotions": ["老用户优惠", "会员专属折扣"],
"valid_until": "2027-01-31"}', name='get_current_promotions',
id='57fd1d0b-89d6-498c-a47f-9620dddb8932',
tool_call_id='call_8b499bdd0ae145d488515c8d')]]}, 'triggers':
('branch:to:model',))}

```

当前流模式：updates, 当前数据：{'model': {'messages':
[AIMessage(content='我已经成功查询到客户 CUST123456
的相关信息，以下是详细汇总：\n\n\n## 📄 客户个人信息\n- **姓名**：张三\n- **会员等级**：VIP\n- **加入日期**：2023年1月15日\n\n\n## 🛒 历史订单信息\n- **总订单数**：15 单\n- **累计消费金额**：¥25,800.00\n\n\n## 🎁
当前可用优惠\n- **可用活动**：\n 1. 老用户优惠\n 2. 会员专属折扣\n- **优惠有效期**：至
2027年1月31日\n\n\n---\n\n\n**总结**：客户张三是一位 VIP
级别的老客户，自2023年1月加入以来共下单15次，累计消费25,800元。目前有两项优惠活动（老用户优惠和会员专属折扣）可供使
用，有效期至2027年1月31日。作为VIP客户，建议重点关注会员专属折扣，通常能为高消费客户带来更多优惠。如需进一步了解某
个优惠的具体内容，请随时告知！', additional_kwargs={'refusal': None},
response_metadata={'token_usage':
{'completion_tokens': 226, 'prompt_tokens': 684, 'total_tokens':
910, 'completion_tokens_details': None,
'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens':
0}}, 'model_provider': 'openai', 'model_name':
'deepseek-v4-flash-0731', 'system_fingerprint': None, 'id':
'chatcmpl-f0de3639-b10e-9de4-8fc5-84f3a3b76884',
'finish_reason': 'stop', 'logprobs': None}, id='lc_run--019ff667-
1f2e-7471-b86b-6836e42e8c29-0', tool_calls=[],
invalid_tool_calls=[], usage_metadata={'input_tokens': 684,
'output_tokens': 226, 'total_tokens': 910,
'input_token_details': {'cache_read': 0}, 'output_token_details':
{}})]]}]}

当前流模式：tasks, 当前数据：{'id': '5e911902-64b2-1498-3036-a9293de59bb6', 'name': 'model', 'error': None,
'result': {'messages': [AIMessage(content='我已经成功查询到客户
CUST123456 的相关信息，以下是详细汇总：\n\n\n## 📄
客户个人信息\n- **姓名**：张三\n- **会员等级**：VIP\n- **加入日期**：2023年1月15日\n\n\n## 🛒 历史订单信息\n- **总订单数**：15 单\n- **累计消费金额**：¥25,800.00\n\n\n## 🎁 当前可用优惠\n- **可用活动**：\n 1. 老用户优惠\n 2.
会员专属折扣\n- **优惠有效期**：至 2027年1月31日\n\n\n---\n\n\n**总结**：客户张三是一位 VIP
级别的老客户，自2023年1月加入以来共下单15次，累计消费25,800元。目前有两项优惠活动（老用户优惠和会员专属折扣）可供使
用，有效期至2027年1月31日。作为VIP客户，建议重点关注会员专属折扣，通常能为高消费客户带来更多优惠。如需进一步了解某
个优惠的具体内容，请随时告知！', additional_kwargs={'refusal': None},
response_metadata={'token_usage':
{'completion_tokens': 226, 'prompt_tokens': 684, 'total_tokens':
910, 'completion_tokens_details': None,
'prompt_tokens_details': {'audio_tokens': None, 'cached_tokens':
0}}, 'model_provider': 'openai', 'model_name':
'deepseek-v4-flash-0731', 'system_fingerprint': None, 'id':
'chatcmpl-f0de3639-b10e-9de4-8fc5-84f3a3b76884',
'finish_reason': 'stop', 'logprobs': None}, id='lc_run--019ff667-
1f2e-7471-b86b-6836e42e8c29-0', tool_calls=[],
invalid_tool_calls=[], usage_metadata={'input_tokens': 684,
'output_tokens': 226, 'total_tokens': 910,
'input_token_details': {'cache_read': 0}, 'output_token_details':
{}})]]}, 'interrupts': []]}

当指定多模式后，可以通过“for stream_mode, chunk in customer_service_agent.stream...”来遍历dict，dict的key(stream_mode)是执行模式，value(chunk)是该模式输出的结果。

9. 实战：多功能智能助手

项目需求：开发一个多功能智能助手，支持：

1. 天气查询：查询城市天气
2. 数学计算：复杂数学运算
3. 时间查询：获取当前时间、日期计算
4. 货币转换：多种货币之间转换
5. 信息搜索：搜索产品、新闻等信息

9.1 模型的初始化

```
In [30]: # 1、模型的初始化
import os
from dotenv import load_dotenv
from langchain.chat_models import init_chat_model

# 从.env文件中加载环境变量
load_dotenv(override=True)

# 模型的初始化
model = init_chat_model(
    model="deepseek-v4-flash",
    model_provider="deepseek",
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL")
)
```

9.2 工具的定义

```
In [31]: # ===== 工具定义 =====
from langchain_core.tools import tool
import math
from datetime import datetime, timedelta

@tool
def get_weather(city: str) -> str:
    """获取指定城市的实时天气信息
    支持中国主要城市的天气查询

    Args:
        city: 城市名称，如"北京"、"上海"、"深圳"等

    Returns:
        包含温度、天气状况、空气质量的详细信息
```

Examples:

```
get_weather("北京") 返回 "多云, 15-22°C, 空气质量良"
```

```
weather_db = {
```

```
    "北京": "多云, 15-22°C, 空气质量良, 湿度 45%",
    "上海": "晴天, 18-25°C, 空气质量优, 湿度 60%",
    "深圳": "小雨, 22-28°C, 空气质量优, 湿度 75%",
    "成都": "阴天, 16-23°C, 空气质量良, 湿度 70%",
    "杭州": "晴天, 17-24°C, 空气质量优, 湿度 55%",
    "广州": "多云, 21-29°C, 空气质量良, 湿度 72%"
}
```

```
result = weather_db.get(city)
```

```
if result:
```

```
    return f"{city}: {result}"
```

```
else:
```

```
    return f"抱歉, 暂不支持查询{city}的天气信息。当前支持: 北京、上海、深圳、成都、杭州、广州"
```

@tool

```
def calculator(expression: str) -> str:
```

```
    """执行数学计算
```

```
    支持基本运算符 (+、-、*、/、**) 和常用数学函数
```

Args:

expression: 数学表达式, 可以包含:

- 基本运算: $2 + 3$, $10 * 5$, $100 / 4$
- 幂运算: $2 ** 10$
- 函数: $\text{sqrt}(16)$, $\text{abs}(-5)$, $\text{pow}(2, 3)$

Returns:

计算结果或错误信息

Examples:

```
calculator("2 + 3 * 4") 返回 "14"
```

```
calculator("sqrt(16)") 返回 "4.0"
```

```
"""
```

```
try:
```

```
    # 安全的数学运算环境
```

```
    safe_functions = {
```

```
        "sqrt": math.sqrt,
```

```
        "pow": pow,
```

```
        "abs": abs,
```

```
        "round": round,
```

```
        "sin": math.sin,
```

```
        "cos": math.cos,
```

```
        "tan": math.tan,
```

```
        "log": math.log,
```

```
        "pi": math.pi,
```

```
        "e": math.e
```

```
    }
```

```
    result = eval(expression, {"__builtins__": {}}, safe_functions)
```

```
    return f"{expression} = {result}"
```

```
except Exception as e:
```

```
    return f"计算出错: {str(e)}\n提示: 请检查表达式格式, 支持的函数有 sqrt, abs, pow, sin, cos, tan, log"
```

@tool

```
def get_time_info(query_type: str = "current") -> str:
    """获取时间相关信息

    Args:
        query_type: 查询类型
            - "current": 当前时间
            - "date": 今天日期
            - "tomorrow": 明天日期
            - "yesterday": 昨天日期
            - "weekday": 星期几

    Returns:
        时间信息字符串

    Examples:
        get_time_info("current") 返回 "2025年1月25日 14:30:25"
        get_time_info("weekday") 返回 "星期六"
    """
    now = datetime.now()

    if query_type == "current":
        return now.strftime("当前时间：%Y年%m月%d日 %H:%M:%S")
    elif query_type == "date":
        return now.strftime("今天是：%Y年%m月%d日")
    elif query_type == "tomorrow":
        tomorrow = now + timedelta(days=1)
        return tomorrow.strftime("明天是：%Y年%m月%d日")
    elif query_type == "yesterday":
        yesterday = now - timedelta(days=1)
        return yesterday.strftime("昨天是：%Y年%m月%d日")
    elif query_type == "weekday":
        weekdays = ["星期一", "星期二", "星期三", "星期四", "星期五", "星期六", "星期日"]
        return f"今天是{weekdays[now.weekday()]}"
    else:
        return f"不支持的查询类型：{query_type}。支持：current, date, tomorrow, yesterday, weekday"
```

@tool

```
def convert_currency(amount: float, from_curr: str, to_curr: str) -> str:
    """货币转换工具
    支持主要货币之间的实时汇率转换
```

Args:

amount: 金额数值
 from_curr: 源货币代码 (CNY/USD/EUR/GBP/JPY/HKD)
 to_curr: 目标货币代码 (CNY/USD/EUR/GBP/JPY/HKD)

Returns:

转换结果

Examples:

convert_currency(100, "CNY", "USD") 返回 "100 CNY = 14.00 USD"

汇率表 (相对于 CNY)

```
exchange_rates = {
    "CNY": 1.0, # 人民币
    "USD": 0.14, # 美元
    "EUR": 0.13, # 欧元
    "GBP": 0.11, # 英镑
```

```

    "JPY": 20.8, # 日元
    "HKD": 1.09 # 港币
}

# 货币名称
currency_names = {
    "CNY": "人民币", "USD": "美元", "EUR": "欧元",
    "GBP": "英镑", "JPY": "日元", "HKD": "港币"
}

from_curr = from_curr.upper()
to_curr = to_curr.upper()

if from_curr not in exchange_rates:
    return f"不支持的源货币：{from_curr}。支持的货币：CNY, USD, EUR, GBP, JPY, HKD"
if to_curr not in exchange_rates:
    return f"不支持的目标货币：{to_curr}。支持的货币：CNY, USD, EUR, GBP, JPY, HKD"

# 转换逻辑：先转为 CNY，再转为目标货币
cny_amount = amount / exchange_rates[from_curr]
result_amount = cnx_amount * exchange_rates[to_curr]

from_name = currency_names[from_curr]
to_name = currency_names[to_curr]

return f"{amount} {from_name} ({from_curr}) = {result_amount:.2f} {to_name} ({to_curr}) "

@tool
def search_info(keyword: str, category: str = "all") -> str:
    """搜索各类信息
    Args:
    keyword: 搜索关键词
    category: 搜索分类
    - "product": 搜索产品
    - "news": 搜索新闻
    - "all": 搜索所有
    Returns:
    搜索结果
    """
    # 模拟数据库
    products = {
        "手机": "iPhone 15 (¥5999), 小米14 (¥3999), 华为Mate60 (¥6999)",
        "笔记本": "MacBook Pro (¥12999), ThinkPad X1 (¥9999), 华为MateBook(¥7999)",
        "耳机": "AirPods Pro (¥1999), Sony WH-1000XM5 (¥2499)"
    }

    news = {
        "AI": "1. GPT-5 即将发布 2. AI 芯片市场增长 30% 3. 新AI法规出台",
        "科技": "1. 量子计算新突破 2. 6G 技术测试 3. 新能源汽车销量创新高"
    }

    results = []

    if category in ["product", "all"]:
        for key, value in products.items():
            if keyword in key:
                results.append(f"【产品】 {key} : {value}")

```

```

if category in ["news", "all"]:
    for key, value in news.items():
        if keyword in key or keyword in value:
            results.append(f"【新闻】 {key} 相关 : {value}")
if results:
    return "\n".join(results)
else:
    return f"未找到关于 '{keyword}' 的{category}信息"

```

9.3 agent的创建

In [32]: **from** langchain.agents **import** create_agent

```

class SmartAssistant:
    """多功能智能助手"""
    def __init__(self):
        # 初始化模型
        self.model = model

        # 工具列表
        self.tools = [
            get_weather,
            calculator,
            get_time_info,
            convert_currency,
            search_info
        ]

        # 系统提示词
        system_prompt = """你是一个多功能智能助手，可以帮助用户：
        🌤️ 查询天气：使用 get_weather 工具
        🧮 数学计算：使用 calculator 工具
        🕒 时间查询：使用 get_time_info 工具
        💱 货币转换：使用 convert_currency 工具
        🔍 信息搜索：使用 search_info 工具

        重要提示：
        1. 仔细阅读用户问题，确定需要使用哪个工具
        2. 如果需要多个工具，按顺序调用
        3. 总是用友好、专业的语气回答
        4. 如果工具返回了数据，要用通俗易懂的语言解释给用户
        5. 如果无法完成任务，诚实地告诉用户原因

        请始终使用中文回答。"""

        # ✅ 创建 agent
        self.agent = create_agent(
            model=self.model,
            tools=self.tools,
            system_prompt=system_prompt
        )

        # 对话历史
        self.messages = []

    def chat(self, user_input: str) -> str:
        """对话接口"""

```

```

# 添加用户消息
self.messages.append({"role": "user", "content": user_input})

# 调用 agent
result = self.agent.invoke({"messages": self.messages})

# 更新消息历史
self.messages = result["messages"]

# 返回最后一条 AI 消息
for msg in reversed(self.messages):
    if msg.type == "ai" and msg.content:
        return msg.content

return "抱歉，我无法处理这个请求。"

def reset(self):
    """重置对话历史"""
    self.messages = []

```

9.4 主程序

```

In [33]: # ===== 主程序 =====
def main():
    assistant = SmartAssistant()
    print("=" * 40)
    print("🤖 多功能智能助手 (LangChain 1.2) ")
    print("=" * 40)
    print("\n我可以帮你：")
    print("☀️ 查询天气")
    print("🧮 数学计算")
    print("🕒 时间查询")
    print("💵 货币转换")
    print("🔍 信息搜索")
    print("\n输入 'quit' 退出，输入 'reset' 重置对话\n")

    demos = [
        "北京今天天气怎么样？",
        "帮我算一下 (25 + 17) * 3",
        "现在几点了？",
        "100 美元等于多少人民币？"
    ]

    for demo in demos:
        print(f"👤 {demo}")
        response = assistant.chat(demo)
        print(f"🤖 {response}\n")

    # 重置对话
    assistant.reset()

    # 交互模式
    print("=" * 40)
    print("💬 进入交互模式")
    print("=" * 40)

    while True:
        user_input = input("\n👤 你: ")

```

```
if user_input.lower() == 'quit':  
    print("再见！👋")  
    break  
  
if user_input.lower() == 'reset':  
    assistant.reset()  
    print("✅ 对话已重置")  
    continue  
  
if not user_input.strip():  
    continue  
# 调用助手  
response = assistant.chat(user_input)  
print(f"🤖 助手: {response}")  
  
if __name__ == "__main__":  
    main()
```

=====

 多功能智能助手 (LangChain 1.2)

=====

我可以帮你：

-  查询天气
-  数学计算
-  时间查询
-  货币转换
-  信息搜索

输入 'quit' 退出，输入 'reset' 重置对话

 北京今天天气怎么样？

 好的！我帮您查询了北京的天气情况，以下是详细信息：

 **北京今日天气**

- **天气状况**：多云
- **温度范围**：15°C ~ 22°C
- **空气质量**：良 
- **湿度**：45%

温馨提示：

- 今天温度比较舒适，适合外出活动，但早晚温差较大（15°C vs 22°C），建议带一件薄外套 
- 多云天气，紫外线强度适中，如果长时间在户外还是可以适当防晒
- 空气质量良好，适合开窗通风和户外运动

请问还有其他需要帮助的吗？

 帮我算一下 $(25 + 17) \times 3$

 我帮您算好了！

$(25 + 17) \times 3 = 126$

计算过程如下：

1. 先算括号内的加法： $25 + 17 = 42$
2. 再用结果乘以 3： $42 \times 3 = 126$

这个计算遵循了数学中的运算顺序规则——**先算括号，再做乘法**，最终结果是 **126**。

还有其他需要计算的问题吗？

 现在几点了？

 我帮您查了一下当前时间：

现在是 2026年8月12日 22:59:44（星期三）

已经快到晚上11点了，时间不早了，提醒您注意休息哦！ 如果还有什么需要帮助的，随时告诉我～


 100 美元等于多少人民币？

 我帮您查询了最新的汇率转换结果：

100 美元 (USD) = 714.29 人民币 (CNY)

按照当前汇率，**1 美元 $\approx$ 7.14 人民币**。

温馨提示：

- 以上汇率仅供参考，实际兑换时银行或兑换机构可能会收取一定的手续费，汇率也会有细微浮动

- 建议在兑换前查看银行当天的实时牌价

还有其他需要帮助的吗？😊

=====

💬 进入交互模式

=====

再见！👋